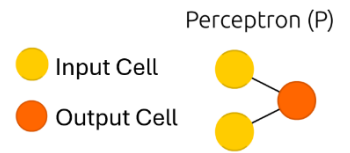


MACHINE LEARNING NOZIONI E APPUNTI

PERCEPTRONE

Il **perceptrone** è una delle forme più semplici di rete neurale. Si tratta di un modello di classificatore lineare che può essere utilizzato per distinguere tra due classi linearmente separabili. Questo significa che, data un'insieme di punti dati, il perceptrone cerca di tracciare una linea (o un iperpiano in dimensioni superiori) che separi i punti appartenenti a classi diverse.



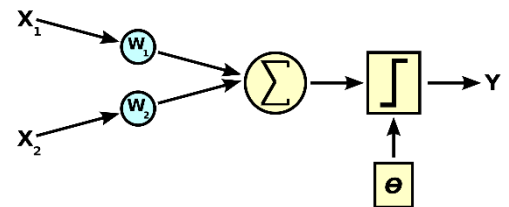
Struttura del Perceptrone

- **Input:** Un insieme di input numerici ($x_1, x_2, \dots, x_n$), ognuno dei quali è associato a un peso ($w_1, w_2, \dots, w_n$).
- **Somma Ponderata:** Calcola una somma ponderata degli input, che è la somma dei prodotti degli input con i loro pesi corrispondenti, più un bias (b).
- **Funzione di Attivazione:** Applica una funzione di attivazione al risultato della somma ponderata. Nel caso del perceptrone, questa funzione è tipicamente una funzione a gradino, che produce un output binario.

La formula di output del perceptrone è data da:

$$y = f\left(\sum_{i=1}^n w_i x_i + b\right)$$

Dove f è la funzione di attivazione, w_i sono i pesi, x_i sono gli input e b è il bias



Esempio:

Immagina di avere un semplice problema di classificazione: decidere se indossare una giacca basandoti sulla temperatura (T) e sulla possibilità di pioggia (P). Puoi codificare "Sì" come 1 e "No" come 0.

- **Input:** Temperatura (T) e Pioggia (P). Ad esempio, $T=18^\circ\text{C}$ (normalizzato come 0.5) e $P=\text{"No"}$ (codificato come 0).
- **Pesi:** Supponiamo che i pesi siano -0.7 per la temperatura e 1 per la pioggia, con un bias di 0.1.
- **Somma Ponderata:** Calcola la somma ponderata:
$$(-0.7 \times 0.5) + (1 \times 0) + 0.1 = -0.25(-0.7 \times 0.5) + (1 \times 0) + 0.1 = -0.25$$
- **Funzione di Attivazione:** Usando una funzione a gradino con soglia a 0, il risultato della somma ponderata è negativo; quindi, l'output della funzione di attivazione è 0 ("No").

In questo caso, il modello suggerirebbe di non indossare la giacca basandosi sulla temperatura e sulla mancanza di pioggia.

La fase di Training di un Percettrone vuol dire aggiustare i pesi dei collegamenti.

La seguente formula indica come effettivamente viene aggiornato il valore dei pesi dei collegamenti:

$$w_j^{(k+1)} = w_j^{(k)} + \lambda(y_i - \hat{y}_i^{(k)})x_{ij},$$

In modo molto intuitivo, il nuovo peso $w^{(k+1)}$ è la combinazione del vecchio peso w^k e un valore proporzionale all'errore di predizione ($y - \hat{y}$). Se la predizione è corretta (il risultato di $(y - \hat{y})$ è 0) allora il peso rimane invariato. Altrimenti viene modificato nel seguente modo:

- Se $y = +1$ e $\hat{y} = -1$: l'errore è dunque uguale a 2 e per compensare l'errore bisogna aumentare il peso dei link positivi e diminuire il peso dei link negativi.
- Se $y = -1$ e $\hat{y} = +1$: l'errore è dunque uguale a -2 e per compensare l'errore bisogna diminuire il peso dei link positivi e aumentare il peso dei link negativi.

Lambda è chiamato **Learning Rate**, che è un valore che varia tra 0 e 1 e serve per controllare quanto fini devono essere gli aggiustamenti durante il processo di learning.

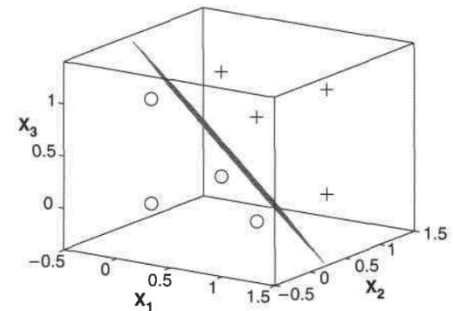
- Se lambda è più vicino a 0, i nuovi pesi variano meno rispetto a quelli precedenti.
- Se è più vicina ad 1, i nuovi pesi possono variare molto rispetto a quelli vecchi.

Alcune volte si può usare il valore lambda in modo adattivo: all'inizio sarà più vicino ad 1 in quanto "deve imparare di più" per poi avvicinarsi sempre più allo 0 per effettuare delle piccole modifiche per raggiungere la precisione.

Il percettrone sa fare operazioni di classificazione solo se i dati sono linearmente separabili, altrimenti è necessario aumentare la complessità del percettrone aggiungendo degli Hidden Layer.

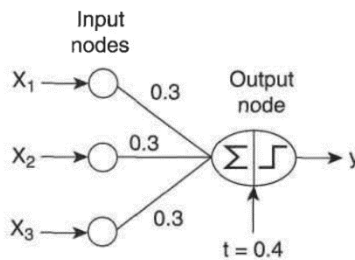
I set di dati linearmente separabili possono essere visti come un Hyperpiano che può essere separato da una retta. L'algoritmo di learning del percettrone converge in problemi linearmente separabili, altrove non converge.

La funzione XOR non è linearmente separabile.



x_1	x_2	x_3	y
1	0	0	-1
1	0	1	1
1	1	0	1
1	1	1	1
0	0	1	-1
0	1	0	-1
0	1	1	1
0	0	0	-1

(a) Data set.



(b) Perceptron.

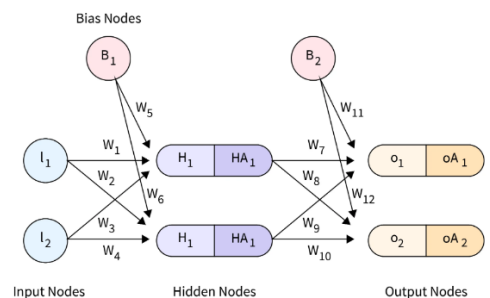
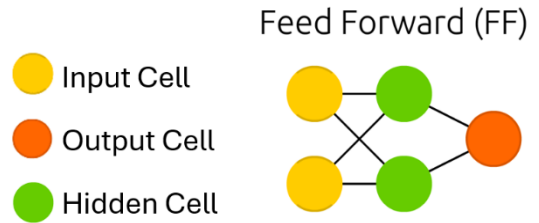
Figure 5.14. Modeling a boolean function using a perceptron.

RETI NEURALI FEED-FORDWARD

Le **Reti Neurali Feedforward**, note anche come **multi-layered network of neurons**, sono definite "feedforward" perché l'informazione fluisce in una direzione unica, dall'input Layer all'output Layer senza ritornare indietro o avere cicli.

L'architettura è composta da tre tipi di layer:

- **Input Layer:** l'input Layer accetta i dati di input e li passa al Layer successivo. Questo layer ha solo la funzione di accettare i dati di input e trasmetterli agli strati successivi. Non ha parametri apprendibili, quindi non è necessario aggiornarli. Agisce solo come un punto di partenza affinché la rete neurale possa operare, e il calcolo inizia negli strati successivi. Possiamo anche utilizzare l'input layer per aggiungere informazioni, come un termine di bias, ai dati di input. Ciò viene fatto aggiungendo un neurone di bias, che produce sempre 1.
- **Hidden Layer:** layer presente tra lo strato di input e lo strato di output. Viene chiamato nascosto perché non interagisce direttamente con l'ambiente esterno. Invece, riceve input solo dall'input layer o dagli layer nascosti precedenti, esegue calcoli interni prima di passare l'output al layer successivo. Ogni Hidden Layer ha un insieme di neuroni connessi ai neuroni del Layer precedente e a quello successivo. Questi Layer utilizzano funzioni di attivazione, come ReLU o sigmoid, per introdurre non-linearità nella rete, permettendole di apprendere e modellare relazioni più complesse tra gli input e gli output.
- **Output Layer:** A seconda del tipo di problema, il numero di neuroni nello strato di output può variare. Ad esempio, in un problema di classificazione binaria, avrebbe solo un neurone. Al contrario, un problema di classificazione multi-classe avrebbe tanti neuroni quanti sono le classi. Il layer di Output ha anche un insieme di parametri apprendibili, come pesi e bias, che vengono aggiornati durante l'addestramento per minimizzare la loss function scelta.



Nell'Immagine:

- **I:** Nodo di input (il punto di partenza per i dati che entrano nella rete neurale)
- **W:** Peso della connessione (utilizzato per determinare la forza della connessione tra i nodi)
- **H:** Nodo nascosto (uno strato all'interno della rete che elabora l'input)
- **HA:** Nodo nascosto attivato (il valore del nodo nascosto dopo essere passato attraverso una funzione predefinita)
- **O:** Nodo di output (l'output finale della rete, calcolato come somma pesata dell'ultimo strato nascosto)
- **OA:** Nodo di output attivato (l'output finale della rete dopo essere passato attraverso una funzione predefinita)
- **B:** Nodo di bias (un valore costante, tipicamente impostato a 1.0, utilizzato per regolare l'output della rete)

L'input alla rete è un vettore di valori, x , che passando attraverso la rete, layer dopo layer, viene trasformato in un output, y . L'output finale della rete predice la **funzione target** per l'input dato. La rete fa questa predizione utilizzando un insieme di parametri, θ (theta), regolati durante l'addestramento per minimizzare l'errore tra le previsioni della rete e la funzione target.

L'addestramento coinvolge l'aggiustamento dei valori di θ (theta) per minimizzare gli errori. Questo viene fatto presentando alla rete un insieme di coppie input-output (chiamate anche dati di addestramento) e calcolando l'errore tra la previsione della rete e l'output vero per ogni coppia.

Questo errore viene poi utilizzato per calcolare il gradiente dell'errore rispetto ai parametri, che indica come regolare i parametri per ridurre l'errore.

Questo viene fatto utilizzando tecniche di ottimizzazione come la discesa del gradiente

La **discesa del gradiente** è un algoritmo di ottimizzazione utilizzato per minimizzare una funzione di costo $J(\theta)$, dove θ rappresenta i parametri della funzione. La definizione formale dell'aggiornamento dei parametri θ in ogni iterazione dell'algoritmo è data da:

$$\theta_{\text{nuovo}} = \theta_{\text{vecchio}} - \alpha \cdot \nabla_{\theta} J(\theta)$$

dove:

- θ_{vecchio} sono i valori correnti dei parametri.
- α è il tasso di apprendimento, che determina la dimensione del passo nell'aggiornamento dei parametri.
- $\nabla_{\theta} J(\theta)$ è il gradiente della funzione di costo $J(\theta)$ rispetto ai parametri θ , ossia un vettore delle derivate parziali di J rispetto a ciascun parametro in θ .

L'obiettivo è iterare questo processo di aggiornamento dei parametri fino a quando non si raggiunge una convergenza, ovvero fino a quando i cambiamenti in θ non portano più a una diminuzione significativa di $J(\theta)$, indicando che l'algoritmo ha trovato un minimo (locale o globale) della funzione di costo.

Una volta che la rete memorizza il valore ottimale di θ (theta) nella sua memoria, può usarlo per predire nuovi input.

I **pesi** e i **bias** in una rete feedforward costituiscono i parametri apprendibili che vengono finemente regolati durante il processo di addestramento. L'obiettivo principale di questo addestramento è minimizzare una **loss function**, che misura la discrepanza tra l'output previsto dalla rete e l'output effettivo (o target). Questa funzione di perdita, quindi, funge da guida per l'aggiustamento dei pesi e dei bias nella direzione che riduce l'errore di previsione della rete.

Il processo di aggiornamento dei pesi e dei bias è noto come **backpropagation**, ed è un passo essenziale nell'addestramento di una rete neurale feedforward.

La **backpropagation**, o retropropagazione, è un algoritmo utilizzato per addestrare le reti neurali feedforward e si articola fondamentalmente in due fasi: la propagazione in avanti (**forward propagation**) e la propagazione all'indietro (**backward propagation**).

- **Propagazione in Avanti (Forward Propagation):** Durante la forward propagation, l'input viene passato attraverso la rete, da layer a layer, fino a generare un output. L'input X viene passato attraverso la rete. Per ogni neurone, l'input netto z e l'output $\hat{y}$ sono calcolati come segue:

$$z = \sum_i (w_i \cdot x_i) + b$$
$$\hat{y} = f(z)$$

Dove w_i sono i pesi, b è il bias, x_i sono gli input e f la funzione di attivazione del neurone.

L'input netto, spesso indicato come z nelle formule che riguardano le reti neurali, rappresenta il valore totale che entra in un neurone prima di essere trasformato dalla funzione di attivazione del neurone stesso. È il risultato della somma pesata degli input ricevuti da un neurone, a cui si aggiunge un termine noto come bias. In altre parole, l'input netto è la combinazione lineare degli input al neurone, ponderata dai rispettivi pesi, più il bias.

- **Calcolo dell'Errore (Loss function):** Dopo aver calcolato l'output, si procede alla valutazione dell'errore, ovvero la differenza tra l'output previsto dalla rete e il valore reale o target. L'errore E della previsione è valutato utilizzando una funzione di perdita, come la Mean Squared Error (MSE) per problemi di regressione:

$$E = \frac{1}{2} \sum (y - \hat{y})^2$$

Dove y è il valore di target

- **Propagazione all'Indietro (Backward Propagation):** La fase di backward propagation inizia con il calcolo dell'errore. Successivamente, quest'errore viene propagato all'indietro attraverso la rete, dal layer di output fino al layer di input. Durante questo processo, il gradiente della funzione di perdita rispetto a ciascun peso viene calcolato per determinare come i pesi contribuiscano all'errore complessivo.

Il cuore della backpropagation è il calcolo dei gradienti dell'errore rispetto ai pesi $\frac{\partial E}{\partial w}$ e ai bias $\frac{\partial E}{\partial b}$ e il loro aggiornamento. Utilizzando la regola della catena, i gradienti sono calcolati come segue:

- **Gradiente dell'errore rispetto all'output del neurone:**

$$\frac{\partial E}{\partial \hat{y}} = \hat{y} - y$$

- **Gradiente dell'errore rispetto all'input netto del neurone** (utilizzando la derivata della funzione di attivazione f'):

$$\frac{\partial E}{\partial z} = \frac{\partial E}{\partial \hat{y}} \cdot f'(z)$$

- **Gradiente dell'errore rispetto ai pesi:**

$$\frac{\partial E}{\partial w} = \frac{\partial E}{\partial z} \cdot \frac{\partial z}{\partial w} = \frac{\partial E}{\partial z} \cdot x$$

- **Gradiente dell'errore rispetto ai bias (dato che $\frac{\partial z}{\partial b} = 1$):**

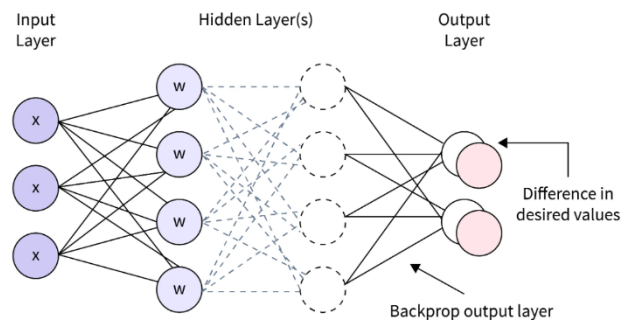
$$\frac{\partial E}{\partial b} = \frac{\partial E}{\partial z}$$

- **Aggiornamento dei Pesi:** Una volta calcolati i gradienti, i pesi sono aggiornati in direzione opposta rispetto al gradiente per minimizzare la funzione di perdita. Questo è tipicamente realizzato attraverso un metodo di ottimizzazione come il Gradient Descent, dove un tasso di apprendimento (learning rate) determina l'ampiezza dello step di aggiornamento.

I pesi e i bias vengono aggiornati sottraendo una frazione dei gradienti calcolati, moltiplicata per un tasso di apprendimento η , dai valori attuali:

$$w_{new} = w_{old} - \eta \cdot \frac{\partial E}{\partial w}$$

$$b_{new} = b_{old} - \eta \cdot \frac{\partial E}{\partial b}$$

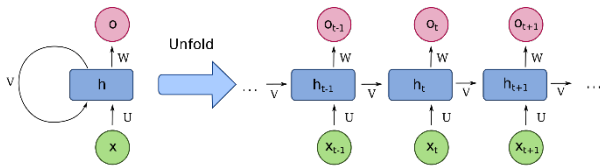


RETI NEURALI RICORRENTI (RNN)

Le Reti Neurali Ricorrenti (RNN) sono una classe di reti neurali progettate per elaborare sequenze di dati, mantenendo uno stato o una memoria dei dati passati. Questa caratteristica le rende ideali per applicazioni che richiedono la comprensione del contesto temporale o sequenziale, come la modellazione del linguaggio, il riconoscimento della voce

e la generazione di testo. A differenza delle reti neurali **feedforward**, dove l'informazione si muove in una sola direzione dall'input all'output, le RNN hanno connessioni cicliche che permettono l'elaborazione di input di lunghezze variabili e la memorizzazione di informazioni attraverso il tempo.

Le Reti Neurali Ricorrenti (RNN) sono reti neurali con un loop in un **hidden-state**, cioè l'output del hidden-state viene reinserito come input del hidden-state insieme all'input al prossimo timestamp.



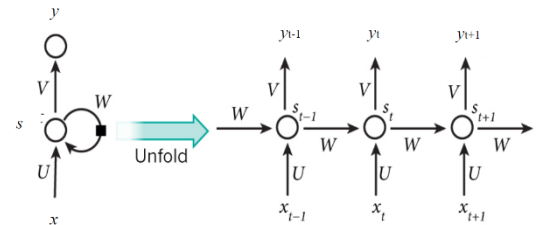
Le RNN possono essere "srotolate" in modelli di **feed-forward**, in cui l'output di ogni passaggio temporale diventa l'input del passaggio successivo. Una RNN srotolata può essere considerata come una deep-learning neural network con un numero infinito di strati.

Lo stato latente di una RNN dovrebbe attribuire pesi alle campioni in una serie temporale, ricordando eventi importanti.

Ad ogni passaggio temporale, lo stato interno viene aggiornato e le previsioni possono essere effettuate basandosi sui modelli a lungo termine. Lo stato latente agisce come una forma di memoria per memorizzare informazioni sulle osservazioni passate.

I parametri di una RNN sono tre matrici:

- **W**, basata sullo stato nascosto precedente.
- **U**, basata sull'input corrente.
- **V**, basata sulla relazione tra lo stato nascosto e l'output.



Durante l'elaborazione di una sequenza di dati, l'input corrente x_t viene combinato con il precedente stato nascosto h_{t-1} , moltiplicato per la matrice dei pesi W . Questo consente alla RNN di catturare le dipendenze a lungo termine nella sequenza.

Successivamente, l'input corrente x_t viene moltiplicato per la matrice dei pesi U , che cattura l'importanza dell'input corrente nel calcolo dello stato successivo.

L'output della somma di questi due prodotti viene quindi passato attraverso una funzione di attivazione non lineare, come la tangente iperbolica ($\tanh$), per ottenere il nuovo stato nascosto h_t .

Infine, lo stato nascosto h_t viene moltiplicato per la matrice dei pesi V per ottenere l'output y_t della RNN. Questo output può essere utilizzato per previsioni, classificazioni o altre operazioni a seconda del compito specifico.

In breve, utilizzando le matrici di pesi W , U e V , una RNN combina l'input corrente x_t con lo stato precedente h_{t-1} per generare un nuovo stato nascosto h_t e un output associato y_t . Questo processo viene ripetuto per ogni passaggio temporale, consentendo alla rete di catturare le dipendenze a lungo termine nella sequenza di dati.

L'equazione base di una RNN è:

$$h_t = \tanh(W * h_{t-1} + U * x_t)$$

Dove:

- h_t rappresenta lo stato nascosto al tempo "t".
- $\tanh()$ è la funzione di attivazione tangente iperbolica che viene applicata all'input sommato, garantisce che i valori rimangano compresi tra -1 e 1
- W è la matrice dei pesi che rappresenta l'interazione tra gli stati nascosti adiacenti h_{t-1} .
- U è la matrice dei pesi che rappresenta l'interazione tra l'input corrente x_t e lo stato nascosto h_{t-1} .
- x_t è l'input corrente al tempo "t".

Questa equazione descrive come l'input corrente x_t viene combinato con lo stato nascosto precedente h_{t-1} , moltiplicato rispettivamente per le matrici dei pesi U e W . Il risultato viene poi passato attraverso la funzione di attivazione tangente iperbolica per ottenere il nuovo stato nascosto h_t .

Mentre l'output di una RNN è definito come:

$$y_t = V * h_t$$

Dove:

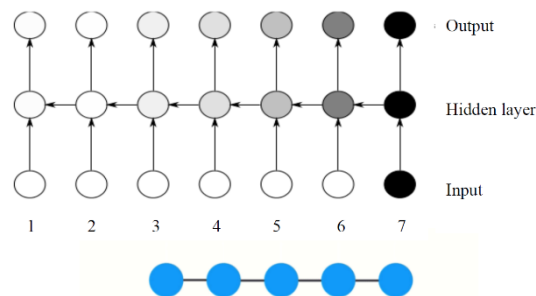
- y_t rappresenta l'output al tempo "t".
- V è la matrice dei pesi che rappresenta l'interazione tra lo stato nascosto h_t e l'output y_t .

Le RNN sono ampiamente utilizzate in applicazioni che coinvolgono dati sequenziali, come il riconoscimento del linguaggio naturale, la traduzione automatica, la previsione del testo, la previsione di serie temporali e molto altro ancora.

Tuttavia, le RNN possono incontrare problemi nel conservare informazioni a lungo termine, noti come "**vanishing gradient**" o "**exploding gradient**", che possono influire sulla capacità di apprendimento della rete su sequenze di lunga durata.

Il "problema del gradiente che tende a scomparire" (**vanishing gradient problem**) si riferisce ad un problema che si presenta durante l'addestramento di Reti Neurali Ricorrenti (RNN), in cui i valori dei gradienti diminuiscono in modo esponenziale. Durante la retropropagazione dell'errore attraverso i passaggi temporali di una RNN, i gradienti, che rappresentano l'informazione sull'errore, vengono calcolati e propagati all'indietro per aggiornare i pesi della rete. Tuttavia, a causa delle multiple moltiplicazioni di matrici coinvolte nel calcolo dei gradienti, i valori del gradiente tendono a ridursi drasticamente man mano che si retrocede nel tempo.

Di conseguenza, le contribuzioni dei gradienti provenienti dai passaggi temporali lontani diventano sempre più piccole e possono avvicinarsi a zero. Ciò significa che lo stato delle RNN in quei passaggi temporali non contribuisce significativamente all'apprendimento del modello. Di conseguenza, le dipendenze a lungo termine non vengono apprese in modo efficace dalle RNN, limitando la loro capacità di catturare correlazioni temporali a lungo raggio.



Nella pratica, questo problema comporta una limitazione nella capacità delle RNN standard di accedere a informazioni contestuali oltre un certo numero di passaggi temporali, solitamente intorno a 10. In altre parole, le RNN standard sono inerentemente instabili su lunghi intervalli di tempo e faticano ad apprendere e utilizzare dipendenze a lungo termine nei dati sequenziali.

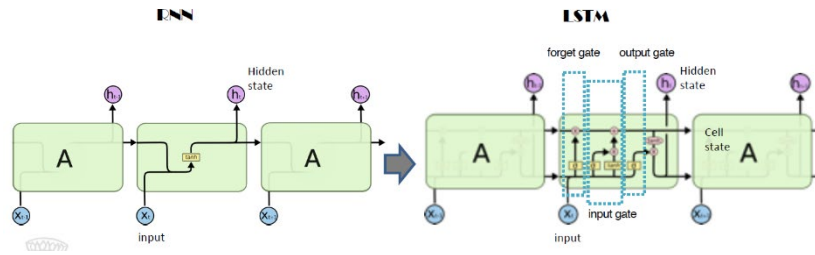
Al contrario l'"exploding gradient problem" (**problema del gradiente che esplode**) i valori dei gradienti aumentano rapidamente man mano che si propagano all'indietro attraverso i passaggi temporali della RNN. Ciò può portare a gradienti di dimensioni enormi che possono causare instabilità nell'addestramento e rendere difficile la convergenza del modello.

L'exploding gradient problem può essere causato da diverse ragioni, ad esempio, quando i pesi della RNN sono inizializzati in modo improprio, quando la funzione di attivazione utilizzata amplifica il segnale dei gradienti, o quando la rete è profonda e i gradienti si accumulano attraverso i numerosi strati.

Per mitigare l'exploding gradient problem, sono state sviluppate diverse tecniche. Una tecnica comune è la "**gradient clipping**", in cui i valori dei gradienti vengono limitati a un determinato intervallo per evitare che diventino troppo grandi.

Per superare il vanishing gradient problem, sono state sviluppate varianti di RNN come le **Long Short-Term Memory (LSTM)** e le **Gated Recurrent Unit (GRU)**, che incorporano meccanismi interni per controllare il flusso dei gradienti e mantenere stabile l'apprendimento su lunghe sequenze di dati. Queste varianti di RNN hanno dimostrato di essere più efficaci nel modellare dipendenze a lungo termine e superare il problema del gradiente che tende a scomparire.

1. Long Short-Term Memory (LSTM)



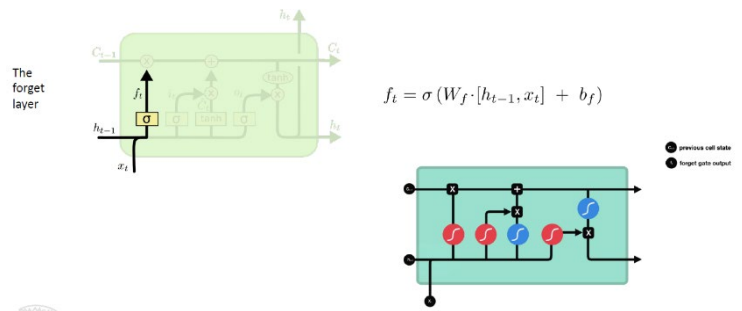
Una LSTM (Long Short-Term Memory) è un tipo di rete neurale ricorrente progettata per gestire dipendenze a lungo termine nelle sequenze di dati. Utilizza una struttura chiamata **cella di memoria** per memorizzare e controllare informazioni nel tempo. Le LSTM sono composte da **porte** (gate) che regolano il flusso di informazioni, consentendo di decidere cosa dimenticare, cosa ricordare e cosa produrre come output.

Le porte delle LSTM sono implementate tramite moltiplicazione elemento per elemento con sigmoide, che si trova nell'intervallo tra 0 e 1. Ciò consente alla rete di apprendere quali dati non sono importanti (da dimenticare) e quali dati sono importanti (da mantenere): ogni numero moltiplicato per 0 diventa 0 (e quindi scompare), ogni numero moltiplicato per 1 rimane lo stesso valore (e quindi viene mantenuto).

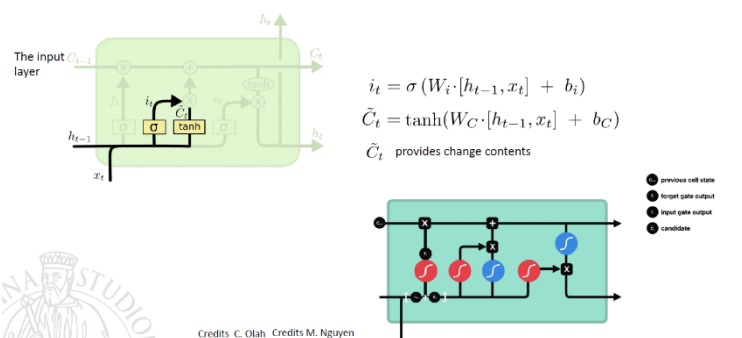
La sigmoide comprime i valori in un intervallo compreso tra 0 e 1. Questo ha il vantaggio di essere differenziabile e quindi adatto alla retropropagazione dell'errore, cioè al processo di calcolo dei gradienti e all'aggiornamento dei pesi durante l'addestramento della rete.

Ogni cella di memoria LSTM contiene quattro elementi principali:

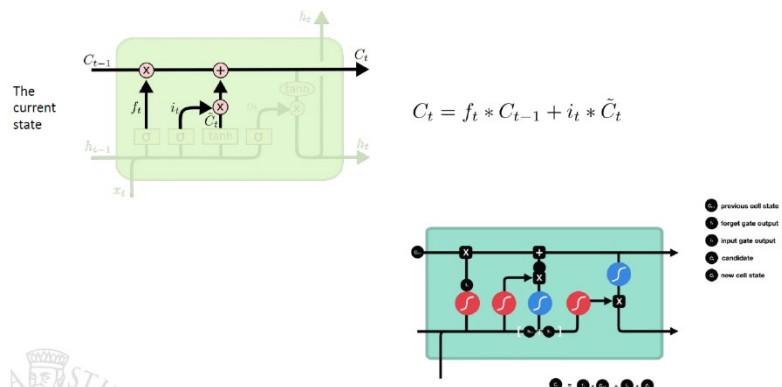
- **Forget gate:** Decide quali informazioni dovrebbero essere eliminate dalla cella di memoria, se mantenere la memoria o scartarla in base all'input corrente. Più l'output della sigmoide si avvicina a 0, significa dimenticare, mentre più si avvicina a 1, significa mantenere.



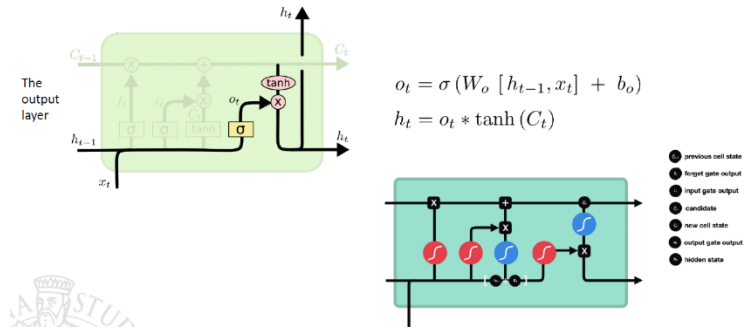
- **Input gate:** Decide quanto delle nuove informazioni verrà fatto passare attraverso la cella di memoria e quali valori della memoria aggiornare con le nuove osservazioni. Un output sigmoide pari a 0 significa che l'informazione non è importante, mentre un output pari a 1 significa che è importante. La funzione tangente iperbolica (tanh) comprime i valori tra -1 e 1 per aiutare a regolare la rete.



- **State update:** Aggiorna lo stato basandosi sullo stato precedente, soggetto a cambiamenti determinati dalle porte di dimenticanza e di input.

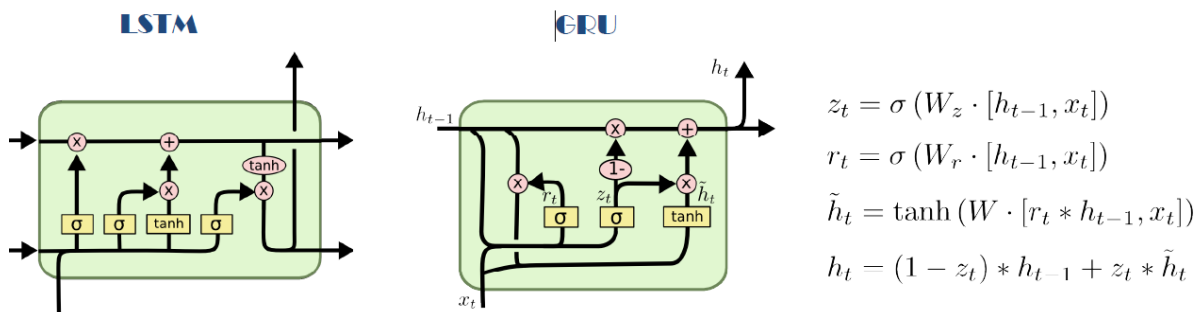


- **Output gate:** Decide quanto delle informazioni verrà passato per essere esposto al passaggio temporale successivo e quale parte dello stato latente verrà prodotta come output in base alla memoria attuale. Lo stato nascosto contiene informazioni sugli input precedenti.



Questi elementi coinvolgono sia attivazioni sigmoidali che tangenti iperboliche (tanh activations).

2. Gated Recurrent Unit (GRU)



Una GRU (Gated Recurrent Unit) è un tipo di rete neurale ricorrente (RNN) che funziona in modo simile a una LSTM (Long Short-Term Memory). La GRU è stata introdotta per semplificare l'architettura delle LSTM, riducendo il numero di porte e parametri.

A differenza delle LSTM, le GRU hanno solo due porte principali: la **porta di reset (Reset gate)** e la **porta di aggiornamento (Update gate)**. Queste porte consentono alle GRU di controllare il flusso di informazioni nel tempo e gestire le dipendenze a lungo termine nelle sequenze.

La GRU ha due componenti principali:

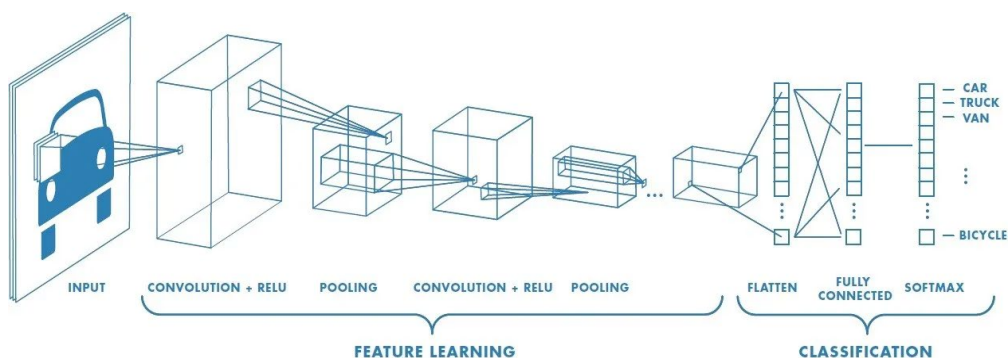
- **Porta di reset (Reset gate):**
 - Questa porta determina quanto dell'informazione passata deve essere dimenticata.
 - Utilizza una funzione sigmoide per generare un valore tra 0 e 1, che rappresenta l'importanza delle informazioni precedenti da scartare.
 - Un valore di reset gate vicino a 0 indica che l'informazione passata sarà completamente dimenticata, mentre un valore vicino a 1 indica che l'informazione passata sarà completamente mantenuta.
- **Porta di aggiornamento (Update gate):**
 - Questa porta regola quanto dell'informazione corrente deve essere incorporata nello stato nascosto aggiornato.
 - Utilizza una funzione sigmoide per generare un valore tra 0 e 1, che rappresenta l'importanza dell'informazione corrente da incorporare.
 - Un valore di update gate vicino a 0 indica che l'informazione corrente avrà poco effetto sull'aggiornamento dello stato nascosto, mentre un valore vicino a 1 indica che l'informazione corrente avrà un forte effetto sull'aggiornamento dello stato nascosto.

Il flusso di funzionamento di una GRU può essere riassunto nei seguenti passaggi:

1. All'inizio, lo stato nascosto viene inizializzato con un valore vuoto.
2. Per ogni elemento nella sequenza di input, la GRU esegue i seguenti passaggi:
 - Calcola il valore della porta di reset utilizzando una funzione sigmoide.
 - Utilizzando il valore della porta di reset, determina quanto dell'informazione passata scartare.
 - Calcola il valore della porta di aggiornamento utilizzando una funzione sigmoide.

- Utilizzando il valore della porta di aggiornamento, determina quanto dell'informazione corrente incorporare nello stato nascosto.
- Aggiorna lo stato nascosto combinando l'informazione passata (scartata o mantenuta) con l'informazione corrente.
- L'output finale può essere generato utilizzando lo stato nascosto aggiornato.

RETI NEURALI CONVOLUZIONALI (CNN)



Una Convolutional Neural Network (CNN) è un tipo di rete neurale artificiale progettata per l'elaborazione efficiente di dati strutturati in forma di griglia, come immagini o video.

Le CNN sono composte da **Convolution Layers**, che estraggono automaticamente caratteristiche dalle immagini (feature learning) tramite filtri (kernel) e **Pooling Layers** che riducono la dimensione spaziale dei dati.

Di solito, i Convolution Layers iniziali identificano le caratteristiche di basso livello, come le linee verticali e orizzontali. Nei layers più profondi, la rete è in grado di identificare caratteristiche complesse come volti e occhi. L'estrazione delle caratteristiche, come quella verticale nel primo layer, viene effettuata dalla rete durante la fase di addestramento.

Architettura CNN

1. **Input Layer:** L'immagine iniziale viene rappresentata come una matrice di pixel. Ogni pixel è rappresentato da un valore di intensità che varia da 0 a 255.
2. **Convolution Layer:** Questo layer utilizza un insieme di filtri (o kernel) per eseguire la convoluzione sull'immagine di input. Ogni filtro è una piccola matrice di pesi che scorre sull'immagine di input per produrre una mappa delle caratteristiche (feature map). Durante la convoluzione, il filtro identifica caratteristiche rilevanti come bordi, texture, o pattern all'interno dell'immagine.
3. **ReLU Activation Layer :** Dopo la convoluzione, una funzione di attivazione ReLU (Rectified Linear Unit) viene applicata a ciascun elemento della feature map. La funzione ReLU introduce non linearità all'interno della rete, aiutando a modellare pattern complessi.
4. **Pooling Layer :** Il pooling riduce la dimensione spaziale delle feature map mantenendo le caratteristiche più rilevanti. Quello comunemente utilizzato è il max-pooling, che prende il valore massimo all'interno di una finestra di pooling.
5. **Fully Connected Layer:** Le feature map ottenute dopo i passaggi precedenti vengono appiattite in un vettore unidimensionale. Questo vettore è quindi passato attraverso uno o più layer completamente connessi, simili a quelli di una rete neurale tradizionale. Questi layers finali trasformano le caratteristiche apprese dalle feature map in una forma adatta per la classificazione.
6. **Output Layer :** Contiene il numero di neuroni corrispondenti al numero di classi da classificare (ad esempio, 0-9 per le cifre MNIST). Utilizzando una funzione di attivazione appropriata (come softmax per la classificazione multiclasse), la rete produce la distribuzione di probabilità delle classi per l'input dato.

Al termine, abbiamo poi la fase di **Training** : durante l'addestramento, la rete è regolata tramite la discesa del gradiente per minimizzare la perdita tra le previsioni della rete e le etichette di classe reali. Questo processo di apprendimento permette alla rete di imparare a riconoscere e generalizzare i pattern presenti nei dati di addestramento.

Implementazione della CNN

Convolutional Layer (Sono un po' ripetizioni ma così è molto chiaro)

Immagina di avere un'immagine di dimensione 100x100 pixel come input. Ogni pixel dell'immagine è rappresentato da un valore di intensità, che potrebbe variare da 0 a 255 a seconda del colore e della scala di grigi. Ora, per comprendere il funzionamento del Convolutional Layer, consideriamo il seguente esempio:

Supponiamo di avere un filtro (o kernel) 3x3 che vogliamo applicare all'immagine di input. Questo filtro è solo una piccola matrice di pesi che può scorrere sull'immagine. Iniziamo con l'immagine di 100x100 pixel e applichiamo il filtro 3x3.

Il filtro viene posizionato sull'immagine di input, e viene calcolato il prodotto scalare tra i valori dei pixel sottostanti e i valori nel filtro. Questo prodotto scalare è calcolato per ogni posizione in cui il filtro può essere sovrapposto all'immagine. Il risultato di questo calcolo viene quindi memorizzato nella feature map, che è essenzialmente una nuova immagine che rappresenta le caratteristiche rilevanti dell'immagine originale.

Ad esempio, supponiamo di avere un filtro come il seguente:

Filtro:

[1 0 -1]

[1 0 -1]

[1 0 -1]

E immaginiamo di sovrapporre questo filtro sull'immagine di input 100x100. A ogni posizione del filtro, viene eseguito il prodotto scalare con i valori dei pixel corrispondenti nell'immagine. Ad esempio, nella prima posizione del filtro, il prodotto scalare può essere calcolato come:

$$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} * \begin{bmatrix} \text{Pixel1} & \text{Pixel2} & \text{Pixel3} \\ \text{Pixel4} & \text{Pixel5} & \text{Pixel6} \\ \text{Pixel7} & \text{Pixel8} & \text{Pixel9} \end{bmatrix}$$

Il risultato del prodotto scalare è poi sommato e il risultato viene posizionato nella feature map risultante. Questo processo viene ripetuto per ogni posizione in cui il filtro può essere sovrapposto sull'immagine.

In pratica, ci sono molti filtri che vengono applicati in parallelo per catturare diverse caratteristiche dell'immagine. Inoltre, durante l'addestramento della rete, i valori del filtro vengono appresi automaticamente attraverso il processo di retropropagazione dell'errore.

ReLU Activation Layer

Dopo l'applicazione del Convolutional Layer, la feature map risultante è soggetta all'applicazione della **funzione di attivazione ReLU** (Rectified Linear Unit).

La funzione ReLU è una funzione non lineare che introduce la non linearità nell'output del Convolutional Layer. La sua equazione è molto semplice:

$$f(x) = \max(0, x)$$

Dove x è l'input alla funzione. In termini semplici, la funzione ReLU restituisce l'input se l'input è maggiore di zero, altrimenti restituisce zero.

Quindi, per ogni valore nella feature map ottenuta dal Convolutional Layer, applichiamo la funzione ReLU. Ciò significa che se il valore è positivo, lo manteniamo invariato, altrimenti lo impostiamo a zero.

L'uso della funzione ReLU è importante per diverse ragioni:

- Introduce la non linearità nell'output del Convolutional Layer, consentendo alla rete di apprendere pattern più complessi e non lineari nelle immagini.
- Aiuta a risolvere il problema della scomparsa del gradiente durante la fase di retropropagazione, in quanto non presenta il problema del gradiente che si annulla per valori negativi.

Dopo l'applicazione della ReLU, la feature map risultante viene passata al layer successivo della rete.

Pooling Layer

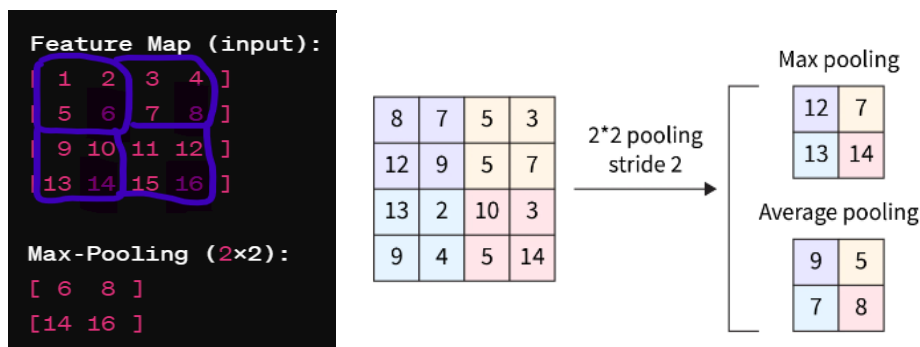
Dopo l'applicazione della funzione di attivazione ReLU, si procede con il **Pooling Layer**.

È progettato per ridurre la dimensione spaziale della rappresentazione e rendere l'output della rete più gestibile, riducendo al contempo il numero di parametri e di calcoli nella rete. Il tipo più comune di operazione di pooling è il max-pooling (ne esistono anche altri come average-pooling).

Ecco come funziona il max-pooling:

- Definizione della finestra di pooling: Si definisce una finestra (o kernel) di dimensione prefissata, ad esempio 2x2 o 3x3.
- Scorrimento della finestra: La finestra di pooling scorre sull'output dell'ultimo layer (che potrebbe essere l'output della funzione di attivazione ReLU del Convolutional Layer).
- Operazione di pooling: In ciascuna posizione della finestra, viene preso il valore massimo. Questo valore massimo diventa quindi l'elemento corrispondente nella nuova mappa di pooling.

Consideriamo un esempio con una finestra di pooling 2x2 e una feature map di dimensione 4x4:



Nell'esempio sopra, la finestra di pooling 2x2 scorre sull'input, e in ciascuna posizione, viene selezionato il valore massimo. Quindi, otteniamo una nuova mappa di pooling con dimensioni ridotte.

Il max-pooling aiuta a mantenere le caratteristiche più significative dell'immagine riducendo la dimensione dell'output e rendendo il modello più robusto alle piccole variazioni di posizione all'interno dell'immagine.

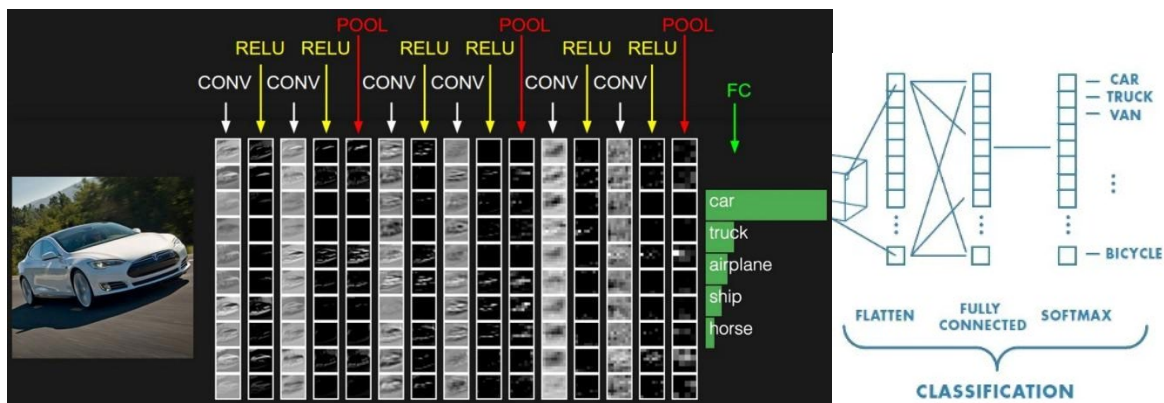
Fully Connected Layer

Si procede con uno o più layer completamente connessi, noti anche come **Fully Connected Layers**.

Il livello FC (Fully connected layer) è l'ultimo layer della CNN. Il layer di input dell' FC appiattisce tutte le immagini in una matrice di valori. Come suggerisce il nome, il layer FC ha tutti i neuroni connessi con tutti gli altri neuroni del layer successivo. Questo, completamente connesso, applica i pesi ai valori appiattiti per ottenere le previsioni delle etichette.

I layer completamente connessi trasformano le caratteristiche estratte dalle precedenti fasi della rete in una forma adatta per la classificazione o per la risoluzione del problema specifico. Questi strati svolgono un ruolo cruciale nel processo decisionale della rete neurale, combinando le informazioni estratte dalle feature map per generare previsioni finali.

L'ultimo layer FC utilizza una funzione di attivazione SoftMax alla fine per produrre valori nell'intervallo [0,1] per ottenere la probabilità dei valori di classificazione. Questo layer è specifico per il compito per cui la CNN viene utilizzata. Ad esempio, se stessimo implementando un'attività di classificazione di immagini, utilizzeremmo una funzione di attivazione Softmax per ottenere una probabilità in uscita dal modello. Se la classificazione è binaria, utilizzeremo una funzione di attivazione Sigmoid.



La differenza principale con le reti tradizionali è che anziché lavorare direttamente con l'input dell'immagine, essi lavorano con le feature map estratte dai Convolutional e Pooling Layers.

Ecco come funzionano gli strati completamente connessi:

1. **Appiattimento delle feature map:** Le feature map ottenute dai Pooling Layers sono appiattite in un vettore unidimensionale. Questo processo permette di trasformare le feature map in un formato adatto per l'input.
2. **Connessioni neurali:** Ogni neurone nei layer completamente connessi è connesso a ogni elemento del vettore appiattito delle feature map. Questo significa che ogni neurone riceve in input le informazioni da tutte le caratteristiche estratte dalle precedenti fasi della rete.
3. **Apprendimento dei pesi:** Durante la fase di addestramento della rete, i pesi delle connessioni tra i layer completamente connessi vengono aggiornati attraverso algoritmi di ottimizzazione come la discesa del gradiente. L'obiettivo è minimizzare la differenza tra le previsioni della rete e le etichette di classe reali dei dati di addestramento.
4. **Funzioni di attivazione:** Ogni neurone nei layer completamente connessi utilizza una funzione di attivazione, spesso la funzione ReLU o la funzione sigmoidea, per introdurre non linearità nelle previsioni della rete.
5. **Classificazione o output:** I layer completamente connessi possono terminare con uno strato di output che utilizza una funzione di attivazione appropriata per il tipo di problema che si sta affrontando. Ad esempio, per la classificazione multiclasse, l'output potrebbe essere passato attraverso una funzione softmax per ottenere una distribuzione di probabilità su tutte le classi.

Training

Questo è un passaggio fondamentale che consente alla CNN di apprendere dai dati di addestramento aggiornando i pesi delle sue connessioni neurali attraverso la retropropagazione dell'errore. Questo processo consente alla rete di migliorare gradualmente le sue prestazioni nel riconoscimento di pattern e nell'elaborazione delle immagini.

Ecco come avviene il processo di addestramento di una CNN:

1. **Alimentazione dei dati:** Durante l'addestramento, vengono presentati alla rete neurale i dati di addestramento, che consistono in coppie di input e output desiderati. Ad esempio, per un problema di classificazione di immagini, gli input sono le immagini e gli output sono le etichette di classe corrispondenti.
2. **Passaggio in avanti (Forward Pass):** Durante la fase di forward pass, i dati di input vengono alimentati attraverso la rete neurale. Le informazioni vengono propagate dai livelli iniziali della rete fino allo strato di output. Durante questo processo, vengono calcolate le previsioni della rete per i dati di input forniti.
3. **Calcolo della perdita:** Dopo aver ottenuto le previsioni della rete, viene calcolata la perdita (loss) tra le previsioni e i valori di output desiderati. La perdita rappresenta una misura dell'errore della rete nel predire le etichette di classe corrette per i dati di addestramento.

4. Retropropagazione (Backpropagation): Una volta calcolata la perdita, la rete utilizza l'algoritmo di retropropagazione per aggiornare i pesi delle connessioni neurali in modo da ridurre la perdita. Durante la retropropagazione, l'errore viene propagato all'indietro attraverso la rete, e i pesi vengono aggiornati utilizzando tecniche di ottimizzazione come la discesa del gradiente.
5. Ottimizzazione dei pesi: Gli algoritmi di ottimizzazione vengono utilizzati per regolare i pesi delle connessioni neurali in modo da ridurre progressivamente la perdita durante l'addestramento. L'obiettivo è trovare i valori dei pesi che minimizzano l'errore della rete sui dati di addestramento e che generalizzino bene su nuovi dati non visti.
6. Iterazioni: Il processo di forward pass, calcolo della perdita e retropropagazione viene ripetuto iterativamente su diverse epoche di addestramento. Durante ogni epoca, la rete neurale continua a migliorare le sue prestazioni, aggiornando i pesi delle connessioni neurali in modo da ridurre progressivamente l'errore.
7. Validazione e Test: Dopo l'addestramento, la CNN viene testata su un insieme separato di dati di validazione e test per valutare le sue prestazioni e la sua capacità di generalizzazione su nuovi dati non visti.

ResNet-50

ResNet-50 è una **rete neurale convoluzionale profonda (CNN, Convolutional Neural Network)** composta da **50 strati** (layers).

Fa parte della famiglia **ResNet (Residual Networks)** introdotta da **He et al. nel 2015** nel paper "*Deep Residual Learning for Image Recognition*".

È uno dei modelli più famosi usati in **computer vision**, in particolare per compiti di:

- classificazione di immagini,
- riconoscimento di oggetti,
- estrazione di feature visive.

Perché usare ResNet?

Prima dell'introduzione della ResNet, il progresso delle **reti neurali convoluzionali (CNN)** si basava principalmente sull'aumento della profondità del modello, cioè sul numero di strati (layers). Reti come **AlexNet** (2012) e **VGGNet** (2014) avevano dimostrato che reti più profonde ottenevano risultati migliori nella classificazione di immagini.

Tuttavia, aumentando ulteriormente la profondità, i ricercatori si sono scontrati con due problemi principali:

1. **Vanishing Gradient Problem** (gradiente che scompare):

Durante il processo di backpropagation, i gradienti che aggiornano i pesi diventano sempre più piccoli man mano che si propagano verso i primi strati. Questo comporta che i layer iniziali smettano di apprendere, bloccando l'allenamento della rete.

2. **Degradation Problem** (degrado delle prestazioni):

Contrariamente alle aspettative, aggiungere più strati non solo non migliorava le prestazioni, ma spesso le peggiorava: l'errore di training aumentava e la rete più profonda era meno accurata di una più "bassa".

Per risolvere questi limiti, He et al. (2015) introdussero il concetto di **Residual Learning**, cioè l'uso dei Residual Blocks con skip connections.

L'idea chiave è che, invece di imparare direttamente una funzione complessa $H(x)$, la rete impara la funzione residua $F(x) = H(x) - x$.

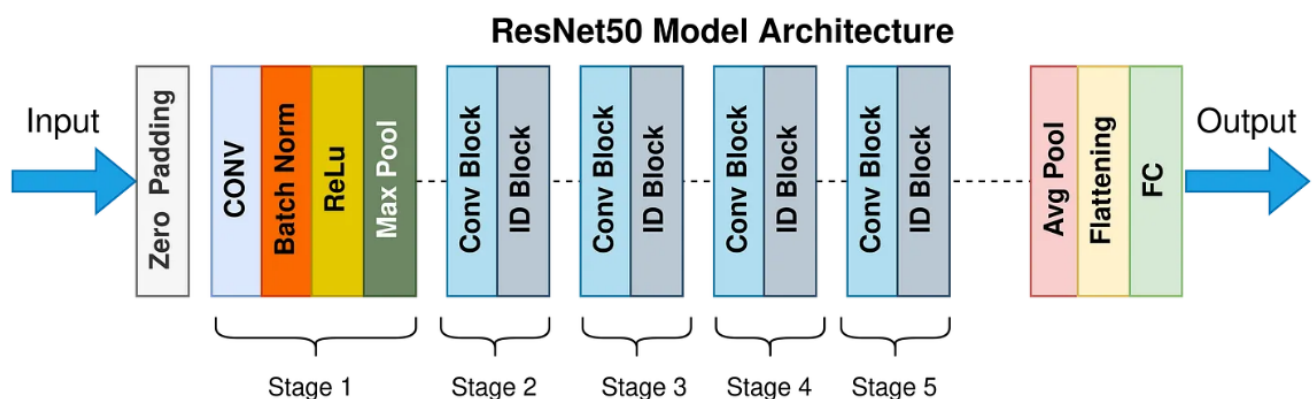
Il risultato finale si ottiene sommando input e output del blocco:

$$y = F(x, W) + x$$

Grazie a questo meccanismo:

- il gradiente può fluire più facilmente all'indietro, evitando il problema del vanishing gradient,
- la rete può "saltare" i layer non utili, semplificando l'ottimizzazione,
- diventa possibile addestrare reti molto profonde (50, 101, 152 layer e oltre) mantenendo alta la precisione.

Architettura ResNet-50



L'architettura della ResNet-50 è composta da una sequenza di stadi (Stage 1 → Stage 5) che trasformano progressivamente l'immagine in rappresentazioni sempre più astratte.

Stage 1 – Input e Preprocessing

- **Zero Padding:** aggiunge bordi di pixel nulli (0) per non perdere informazione ai margini durante le convoluzioni.

- **Conv (7×7 , stride 2, 64 filtri):** primo layer convoluzionale che estrae feature di base (bordi, linee, pattern semplici).
- **Batch Normalization:** normalizza le attivazioni per rendere più stabile e veloce l'allenamento.
- **ReLU:** attivazione non lineare che introduce complessità nelle funzioni apprese.
- **Max Pool (3×3 , stride 2):** riduce la dimensione spaziale dell'immagine, mantenendo solo le informazioni più importanti.

Dopo questo passaggio, l'immagine 224×224 è stata compressa in una mappa di $56 \times 56 \times 64$ con le prime feature di basso livello.

Stage 2 - Prime Residual Blocks

- Contiene 1 Conv Block +2 Identity Blocks.
- Ogni blocco segue la struttura bottleneck:
 1. convoluzione 1×1 (riduzione dimensione),
 2. convoluzione 3×3 (estrazione feature),
 3. convoluzione 1×1 (espansione dimensione).
- Nel Conv Block lo skip connection non è identità, ma viene adattato con una convoluzione 1×1 per allineare le dimensioni.
- Gli Identity Block invece mantengono le dimensioni e sommano direttamente input e output.
- Output Stage 2: $56 \times 56 \times 256$.

Le feature ora descrivono non solo bordi ma piccoli pattern locali.

Stage 3 - Rappresentazioni intermedie

- Struttura: 1 Conv Block +3 Identity Blocks.
- La profondità dei filtri aumenta: $128 \rightarrow 128 \rightarrow 512$.
- L'immagine viene ridotta in dimensione ma arricchita di canali.
- Output Stage 3: $28 \times 28 \times 512$.

Le feature catturano forme intermedie e combinazioni di pattern (es. contorni di oggetti).

Stage 4 - Profondità maggiore

- Struttura: 1 Conv Block +5 Identity Blocks (il più grande stadio della rete).
- Filtri: $256 \rightarrow 256 \rightarrow 1024$.
- Rappresentazioni più complesse e astratte, con focus su texture e parti di oggetti.
- Output Stage 4 : $14 \times 14 \times 1024$.

La mappa diventa più compatta, ma le feature hanno un significato molto più ricco.

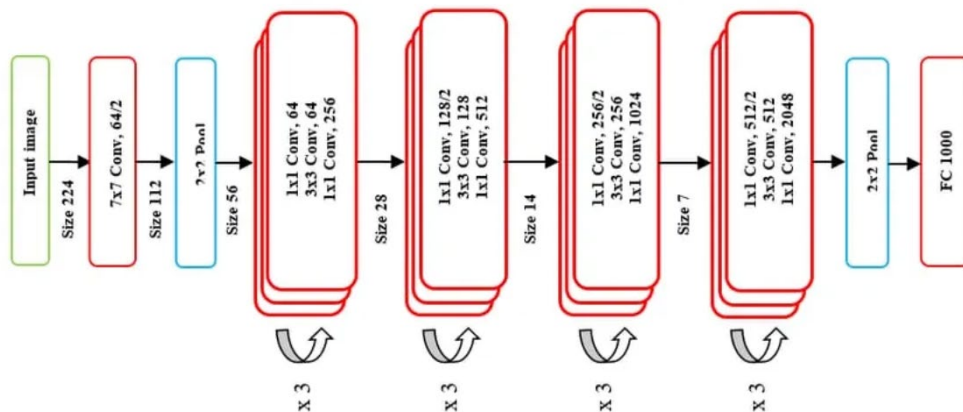
Stage 5 - Astrazione finale

- Struttura: 1 Conv Block +2 Identity Blocks.
- Filtri: $512 \rightarrow 512 \rightarrow 2048$.
- Rappresentazioni molto astratte: qui la rete cattura concetti come oggetti interi o scene.
- Output Stage 5 : $7 \times 7 \times 2048$.

Classificazione finale

- Average Pooling (7×7): comprime ogni canale (2048) a un singolo valore medio $\rightarrow$ output diventa $1 \times 1 \times 2048$.
- Flattening: trasforma in un vettore 1D di 2048 dimensioni.
- Fully Connected Layer (FC): proietta il vettore sulle classi (es. 1000 in ImageNet).
- Softmax: converte i valori in probabilità.

Output finale: distribuzione di probabilità $\rightarrow$ l'immagine viene assegnata alla classe più probabile.



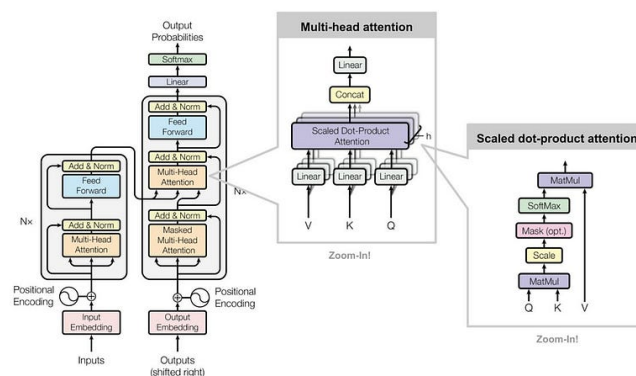
Confronto CNN tradizionale vs ResNet-50

Aspetto	CNN tradizionale	ResNet-50
Architettura generale	Sequenza lineare: Input $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ Pooling $\rightarrow$ (ripetizione) $\rightarrow$ Flatten $\rightarrow$ Fully Connected $\rightarrow$ Output.	Architettura a 5 stadi con Residual Blocks (Conv Block + Identity Block) + skip connections.
Flusso dell'informazione	Puramente sequenziale: ogni layer prende in input solo l'output del precedente.	Doppio flusso: (1) flusso sequenziale (conv $\rightarrow$ BN $\rightarrow$ ReLU) (2) scorciatoia (skip connection) che somma input e output.
Numero di strati	Tipicamente 5-20 (LeNet, AlexNet) o max 16-19 (VGG). Oltre diventa instabile.	50 strati pesati (conv/FC). Varianti: ResNet-101, ResNet-152.
Blocchi principali	- Convolutional Layer (kernel 3×3 o 5×5) - ReLU Activation - Pooling Layer - Fully Connected Layer	- Conv Block ($1 \times 1, 3 \times 3, 1 \times 1$ conv + skip adattato) - Identity Block ($1 \times 1, 3 \times 3, 1 \times 1$ conv + skip diretto) - Ogni blocco segue schema bottleneck.

Residual Learning	Non presente. Ogni layer deve imparare da zero una trasformazione complessa.	Presente: la rete impara solo la differenza residua $F(x) = H(x) - x$. Formula: $y = F(x) + x$.
Problemi di training	- Vanishing Gradient (gradiente che scompare) - Degradation Problem (più layer → errore maggiore).	Risolti: skip connections mantengono il gradiente e permettono l'addestramento di reti molto profonde.
Capacità di astrazione	Bordi e linee (layer iniziali) → forme e pattern (layer medi) → oggetti semplici (layer finali).	Stesso processo ma molto più potente e stabile: da feature di basso livello → parti di oggetti → oggetti interi → concetti astratti.
Efficienza di training	Allenare reti profonde è difficile e spesso inefficace.	Allenamento stabile anche con 50+ strati, meno sensibile a inizializzazione e iperparametri.
Prestazioni (ImageNet)	- AlexNet (2012): 15.3% top-5 error - VGG-16 (2014): 7.3% top-5 error	- ResNet-50 (2015): 3.57% top-5 error → nuovo stato dell'arte.
Uso tipico	Classificazione immagini base, dataset medi (MNIST, CIFAR, ImageNet "piccolo").	Grandi dataset (ImageNet, COCO), backbone per detection (YOLO, Faster R-CNN), segmentation (Mask R-CNN), transfer learning.

Transformer

Un Transformer è un modello di deep learning che, a differenza delle architetture precedenti come le reti neurali ricorrenti (RNN) e le [reti neurali convoluzionali \(CNN\)](#), si basa sul meccanismo di **auto-attenzione (self-attention)** per elaborare le sequenze di input. Viene utilizzato principalmente nei campi dell'elaborazione del linguaggio naturale (NLP) e della visione artificiale (CV). Come le reti neurali ricorrenti (RNN), i transformer sono progettati per elaborare dati di input sequenziali, come il linguaggio naturale, con applicazioni per compiti come la traduzione e la sintesi del testo. Tuttavia, a differenza delle RNN, i transformer elaborano l'intero input in una sola volta.



Architettura del Transformer

Elenco dell'architettura di un Transformer applicato a MANTRA:

1. **Input delle traiettorie:** Il controller di memoria MANTRA deve gestire i dati rappresentati in memoria dalla traiettoria passata e dalla traiettoria futura che rappresentano rispettivamente Key e Value. Nel nostro progetto questi due valori saranno concatenati l'input dell'encoder definendo la Source.
2. **Input embedding:** Le sequenze di posizioni vengono rappresentate come vettori di embedding attraverso un processo di codifica. Ogni posizione della traiettoria viene mappata in un vettore numerico, che può essere inizializzato casualmente o appreso durante il processo di addestramento del Transformer. Gli embeddings di input catturano le caratteristiche spaziali delle traiettorie.
3. **Positional encodings:** Poiché il Transformer è un modello basato su attenzione e non ha una struttura ricorrente o convoluzionale che preserva l'ordine, è necessario inserire delle informazioni sulla posizione relativa o assoluta dei token nella sequenza. Quindi vengono aggiunti vettori di posizione ai vettori di embedding di input per fornire informazioni sulla posizione relativa delle posizioni nella traiettoria. Questi vettori di posizione sono calcolati utilizzando funzioni sinusoidali. Per ogni posizione nella sequenza, si generano valori sinusoidali con frequenze diverse, per catturare le informazioni sulla posizione relativa. Questi valori sinusoidali possono essere calcolati utilizzando formule matematiche, come ad esempio:

$$PE_{pos,2i} = \sin(pos/10000^{2i/d_{model}}) \quad PE_{pos,2i+1} = \cos(pos/10000^{2i/d_{model}})$$

Dove pos rappresenta la posizione nella sequenza, i rappresenta l'indice della dimensione dell'embedding, e d_{model} rappresenta la dimensione degli embeddings di input.

Una volta calcolati i valori dei positional encodings per ogni posizione nella sequenza, vengono semplicemente sommati agli embeddings di input. Questo viene fatto per ciascuna dimensione dell'embedding, elemento per elemento.

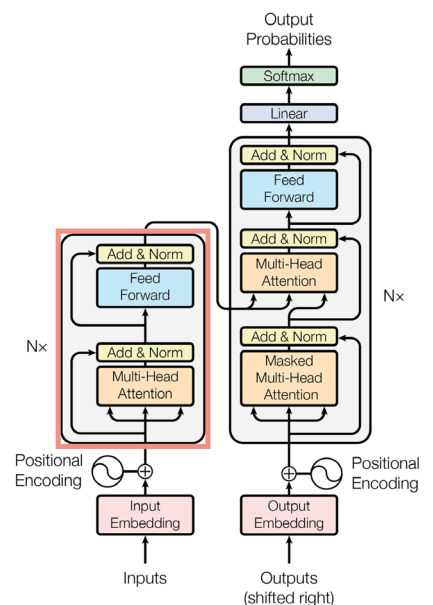
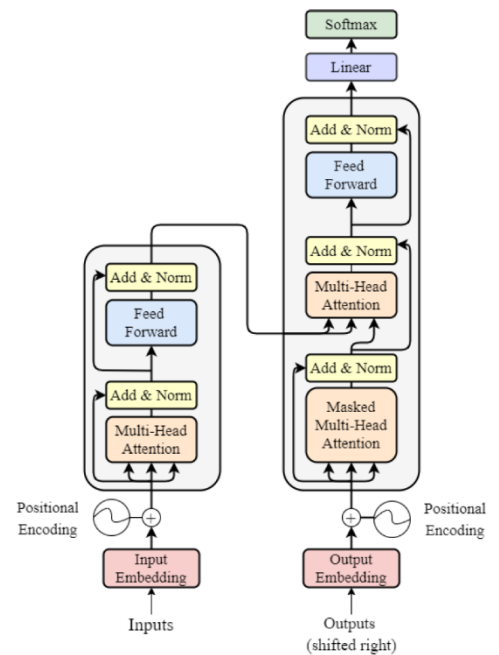
Il macro-blocco successivo rappresenta la parte dell'**Encoder**

L'encoder è composto da una pila di più strati identici. Ogni strato è formato da due sottolivelli. Il primo è un meccanismo di self-attention multi-head, mentre il secondo è una semplice rete fully connected feed-forward che agisce a livello di posizione. Successivamente ogni livello viene normalizzato. In altre parole, l'output di ciascun sottolivello è $LayerNorm(x + Sublayer(x))$, dove $Sublayer(x)$ rappresenta la funzione implementata dal sottolivello stesso. Per facilitare queste connessioni residue, tutti i sottolivelli del modello, così come i livelli di embedding, producono output di dimensione d_{model} .

1. **Multi-head Attention:** è una parte fondamentale dell'encoder. Prende in input la rappresentazione degli embeddings con i positional encodings dell'input e calcola l'attenzione tra i token della sequenza. La multi-head attention opera su più sottospazi dimensionali chiamati "teste", consentendo al modello di concentrarsi simultaneamente su diverse parti della sequenza.

Questa seguirà le seguenti fasi:

- a. **Preparazione:** Prima di applicare l'attenzione multi-head, l'input viene diviso in tre parti, chiamate query, chiavi e valori. Nel contesto di MANTRA, la chiave (key) rappresenta la traiettoria passata, il valore (value) rappresenta la traiettoria futura e la query rappresenta lo stato osservato. Queste parti

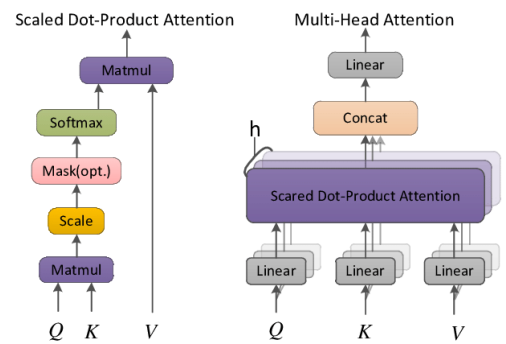


vengono ottenute da una trasformazione lineare dell'input, utilizzando pesi appresi durante l'addestramento del modello. L'obiettivo è creare rappresentazioni diverse degli input per le tre parti.

- b. **Calcolo dell'attenzione:** Per ogni testa di attenzione, vengono calcolati tre set di rappresentazioni: query, chiavi e valori. Le query corrispondono alle posizioni dei token correnti per cui si vuole calcolare l'attenzione, mentre le chiavi e i valori corrispondono alle posizioni di tutti i token nella sequenza. Per calcolare l'attenzione, vengono eseguite operazioni matematiche tra le query, le chiavi e i valori. In particolare, viene utilizzato il prodotto scalare tra le query e le chiavi normalizzato per ottenere i pesi di attenzione per ciascun token nella sequenza.
 - c. **Aggregazione delle attenzioni:** Dopo aver calcolato i pesi di attenzione per ciascun token nella sequenza, le attenzioni vengono combinate per ottenere un'unica rappresentazione aggregata. Questo viene fatto moltiplicando i pesi di attenzione per i valori corrispondenti e sommando i risultati pesati. In questo modo, i token rilevanti per una determinata query ottengono un peso maggiore, mentre i token meno rilevanti ottengono un peso minore.
 - d. **Concatenazione delle teste:** Dopo aver calcolato l'attenzione multi-testa per ogni testa, le rappresentazioni risultanti vengono concatenate lungo la dimensione dell'embedding. Questo consente al modello di considerare informazioni diverse da diverse prospettive e angoli di attenzione. La concatenazione delle teste di attenzione multi-testa porta ad avere una rappresentazione più ricca e complessa dell'input.
 - e. **Proiezione finale:** Infine, la rappresentazione aggregata ottenuta dalla concatenazione delle teste di attenzione multi-testa viene proiettata attraverso un'operazione lineare per ottenere l'output finale dell'attenzione. Questo output può essere ulteriormente elaborato dai successivi strati di normalizzazione del batch e feed-forward network nell'architettura del Transformer.
2. **Normalizzazione del batch:** Dopo l'attenzione multi-testa, viene applicata una normalizzazione del batch, come la Batch Normalization, per ridurre la covarianza tra le feature e stabilizzare il processo di apprendimento. La normalizzazione del batch viene applicata a ciascuna dimensione dell'embedding di input separatamente.
 3. **Feed-forward network:** Dopo la normalizzazione del batch, l'output passa attraverso un feed-forward network (rete a feed-forward) che opera su ciascun token indipendentemente. Il feed-forward network è un'operazione di rete neurale completamente connessa che applica una trasformazione lineare seguita da una funzione di attivazione, come ad esempio una funzione ReLU (Rectified Linear Unit). L'uso del feed-forward network permette al modello di apprendere relazioni complesse tra i token e di catturare informazioni non lineari nella sequenza.

La parte successiva è formata da:

1. **Output delle traiettorie:** Per quanto riguarda l'input del decodificatore, viene utilizzato il valore di query. Questo valore di query rappresenta fundamentalmente il valore dello stato osservato.
2. **Output embedding:** per dare diversità alle traiettorie passate facciamo un embedding, creando così del rumore
3. **Positional Encoding:** il positional encoding è comunque presente per gestire l'ordine con cui i dati vengono passati in input al decoder



Successivamente abbiamo il macro-blocco del **Decoder**

Il decoder ha una struttura simile all'encoder, ma con qualche differenza. Abbiamo sempre una struttura a layer, ma l'attenzione del decoder è guidata anche dall'output dell'encoder. Durante la generazione dell'output, il decoder utilizza l'output dell'encoder per calcolare l'attenzione multi-head che si concentra sugli aspetti rilevanti dell'input. Ciò consente al decoder di focalizzare l'attenzione su parti specifiche dell'input durante la generazione dell'output.

1. Masked Multi-Head attention:

La fase di self-attention nel layer del decoder è modificata mediante l'aggiunta del masking. L'uso del masking permette di garantire che durante la generazione dell'output del decoder, il modello non possa "guardare" le posizioni future nella traiettoria o sequenza di dati. In questo modo, il modello può basarsi solo sulle informazioni precedenti e generare l'output in modo coerente con l'ordine temporale, essendogli impedito di prestare attenzione alle posizioni successive. L'utilizzo della mascheratura e lo spostamento degli embedding di output consentono di garantire che durante la generazione delle previsioni per una determinata posizione i , queste dipendano solo dagli output conosciuti alle posizioni precedenti a i . In altre parole, l'attenzione si limita solo alle informazioni disponibili fino a quel punto nel processo di decodifica.

Quindi nel caso delle traiettorie, considerare solo le posizioni precedenti nella traiettoria per generare le previsioni correnti, lo spostamento di una posizione negli embedding di output assicurerebbe che le previsioni per una determinata posizione dipendano solo dalle informazioni delle posizioni precedenti nella traiettoria.

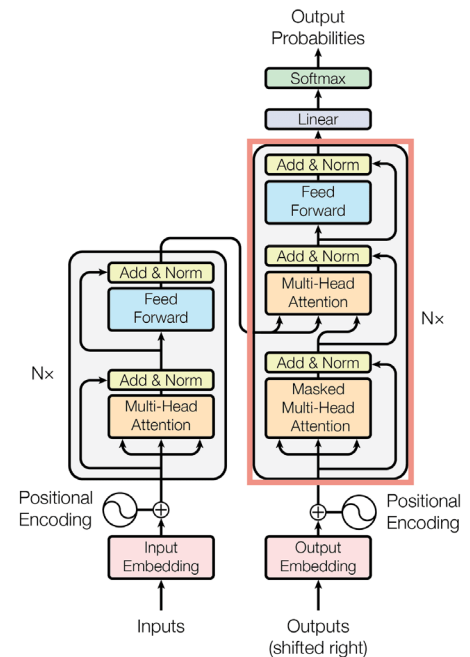
Ad esempio, se la traiettoria specifica comprende le posizioni [1, 3, 5, 7] durante il processo di generazione del testo, lo spostamento di una posizione negli embedding di output garantirebbe che la generazione della parola in posizione 5 dipenda solo dalle parole generate nelle posizioni 1, 3 e 5, ignorando le posizioni successive come 7. Questo aiuta a mantenere la corretta causalità ed è necessario per garantire che il modello generi output corretti e coerenti senza accesso diretto al contesto futuro durante l'addestramento.

Nel nostro codice la parte di Masking viene già gestita dal modulo *nn.Transformer*

Il modulo *nn.Transformer* di PyTorch gestisce il masking nel decoder attraverso l'uso di maschere durante il calcolo dell'attenzione. Questo avviene all'interno del metodo *forward* del modulo *nn.Transformer* e viene gestito automaticamente senza richiedere azioni specifiche da parte dell'utente.

- **Masking padding:** Viene applicato un masking padding per nascondere i token di padding nella sequenza di input. Questo masking garantisce che il modello non consideri i token di padding durante il calcolo dell'attenzione. Il valore assegnato ai token di padding è $-\infty$ (valore molto negativo) nella matrice di attenzione, in modo che, quando viene applicata la softmax, le probabilità associate a tali token diventino praticamente zero.
- **Masking causale:** Viene applicato un masking causale per nascondere le informazioni future nella sequenza di output durante il calcolo dell'attenzione. Questo masking è fondamentale per garantire che il modello non possa accedere alle informazioni future durante la generazione dell'output. Il valore assegnato alle posizioni future nella matrice di attenzione è $-\infty$, in modo che, quando viene applicata la softmax, le probabilità associate a tali posizioni diventino molto basse e le informazioni relative a quelle posizioni vengano praticamente ignorate durante il calcolo finale dell'attenzione.

Durante il calcolo dell'attenzione nel decoder, le maschere vengono applicate alla matrice di attenzione per nascondere le informazioni indesiderate. Questo viene fatto moltiplicando la matrice di attenzione per la maschera appropriata. In particolare, il masking padding viene applicato alle chiavi e ai valori, mentre il masking causale viene applicato alle chiavi, alle query e ai valori.

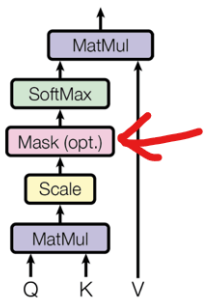


Dopo l'applicazione delle maschere, il calcolo dell'attenzione prosegue come di consueto, con il calcolo delle medie pesate dei valori utilizzando la matrice di attenzione normalizzata.

In sintesi, il modulo *nn.Transformer* di PyTorch gestisce il masking nel decoder applicando automaticamente il masking padding e il masking causale durante il calcolo dell'attenzione. Questo garantisce che il modello non possa accedere alle informazioni future durante la generazione dell'output, generando sequenze di output coerenti e rispettando l'ordine causale delle parole.

Questo riguardava la parte di **Masking**. Per quanto riguarda la **self-attention**, valgono gli stessi concetti dell'encoder, soltanto che aggiungiamo la parte di Masking, definita prima come opzionale.

Scaled Dot-Product Attention



2. Cross-Attention con Encoder nel Decoder:

La Cross-Attention è una fase chiave che facilita l'interazione tra l'encoder e il decoder nel modello Transformer. Essa consente al decoder di accedere alle informazioni generate dall'encoder durante la generazione dell'output sequenziale. La Cross-Attention avviene all'interno del decoder e coinvolge l'uso delle informazioni dell'encoder per guidare la generazione dell'output.

Dopo la fase di self-attention, il decoder esegue una cross-attention con l'output dell'encoder. In questa fase, la matrice di query (Q) viene ottenuta dallo stato attuale del decoder, mentre le chiavi (K) e i valori (V) sono ottenuti dall'output dell'encoder. Questo passaggio consente al decoder di accedere alle informazioni rilevanti dell'input durante la generazione dell'output. In MANTRA, la Cross-Attention viene utilizzata per interagire con la memoria delle traiettorie e ottenere un contesto significativo per la generazione delle traiettorie nel passo corrente del decoder. Questo meccanismo consente al decoder di accedere alle informazioni sulle traiettorie passate e future durante la generazione dell'output, facilitando la generazione di traiettorie coerenti e informative.

3. Normalizzazione:

Gli output della Cross-Attention vengono nuovamente normalizzati

4. Feed-Forward Neural Network:

Gli output normalizzati della Cross-Attention vengono passati attraverso un altro livello di rete neurale feed-forward

5. Altra Normalizzazione:

Gli output del feed-forward network vengono nuovamente normalizzati

6. Generazione dell'Output Probability:

Nei passaggi finali del decoder del Transformer, gli output normalizzati e trasformati attraverso l'attenzione e le reti feed-forward vengono elaborati per generare l'output sequenziale finale

- Proiezione Lineare:** Gli output normalizzati e trasformati vengono passati attraverso un livello di proiezione lineare. Questo livello consiste in una matrice di pesi appresa che proietta gli output dallo spazio latente a un nuovo spazio dimensionale corrispondente alle dimensioni d_{model} . La proiezione lineare aiuta a mappare le rappresentazioni interne del decoder nell'output finale.
- Funzione Softmax:** L'output del livello di proiezione lineare viene quindi passato attraverso una funzione softmax. La formula per il calcolo della funzione Softmax all'interno di un Transformer è la seguente:

$$Softmax(x)_i = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

Con x vettore di input, x_i rappresenta l' i -esimo elemento del vettore, n è la dimensione e e^{x_j} è l'elemento j -esimo del vettore di output normalizzato.

- c. **Generazione dell'Output:** L'output sequenziale viene generato selezionando il token con la probabilità più alta dalla distribuzione di probabilità generata dalla funzione softmax. Questo token selezionato viene quindi utilizzato come input per il passo successivo del decoder per generare il token successivo nella sequenza di output. Questo processo di generazione viene ripetuto fino a quando non si genera l'intera sequenza di output desiderata.

La proiezione lineare e la funzione softmax nell'output finale del decoder consentono di generare una distribuzione di probabilità su tutto il vocabolario di output, consentendo al modello di selezionare il token successivo in base alla sua probabilità. In questo modo, il decoder del Transformer può generare sequenze di output coerenti e significative sulla base delle informazioni raccolte durante i passaggi precedenti del processo di decoding.

ViT – Visual Image Transformer

Il **Vision Transformer (ViT)** è un'architettura di rete neurale che applica il meccanismo dei **Transformer**, nati per il linguaggio naturale (NLP), al mondo delle **immagini**.

Invece di usare convoluzioni (CNN), il ViT rappresenta l'immagine come una **sequenza di patch**, trattandole come "parole" e sfruttando il **self-attention** per capire le relazioni globali tra di esse.

Perché usare Vision Transformer (ViT)

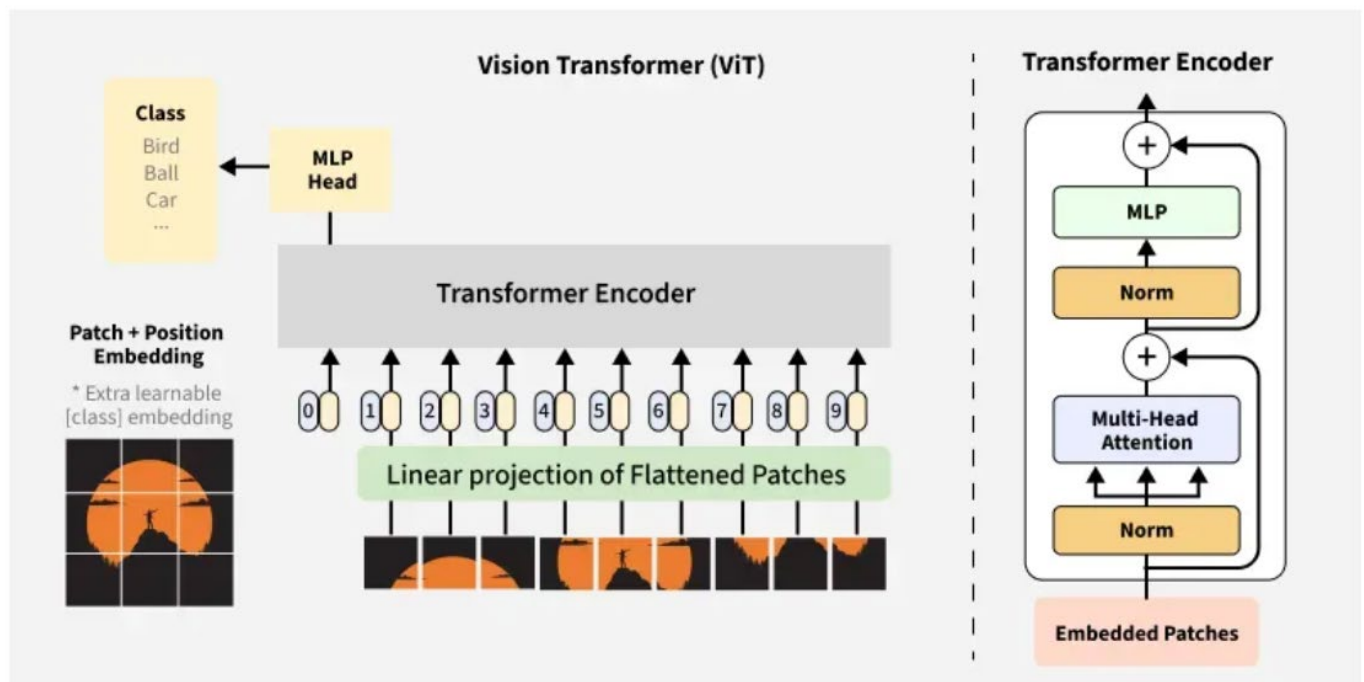
1. Capacità di cogliere relazioni globali

- Le **CNN** vedono il mondo con "finestre locali" (kernel), quindi all'inizio catturano solo dettagli piccoli (bordi, texture).
- Il **ViT** invece, grazie al **self-attention**, fa sì che **ogni patch guardi tutti gli altri contemporaneamente** → fin dall'inizio può considerare **relazioni a lungo raggio** (es. la zampa di un cane collegata alla testa, anche se distanti).

2. Architettura più semplice e modulare

- Una CNN è piena di **convoluzioni, pooling, padding, stride...**
- Il ViT ha un design molto più **pulito**: patch embedding → encoder Transformer → classificatore.
- Questo lo rende più facile da **scalare** e da adattare ad altri compiti (detection, segmentation, video, ecc.).

Architettura ViT



L'architettura del Vision Transformer comprende diversi stadi chiave:

1. **Suddivisione e incorporamento delle patch (Image Patching and Embedding)**
2. **Codifica posizionale (Positional Encoding)**
3. **Encoder del Transformer (Transformer Encoder)**
4. **Testa di classificazione (Classification Head, MLP Head)**

1 Suddivisione e Incorporamento delle Patch (Image Patching and Embedding)

Il primo e più critico passo nella pipeline del ViT è convertire l'immagine in una sequenza di patch, in modo analogo a come un modello NLP lavora con i token di un testo.

- **Suddivisione in patch (Patch Splitting):**

L'immagine di input, solitamente di dimensione $H \times W \times C$ (altezza, larghezza e canali), viene divisa in patch di dimensione fissa.

Ad esempio, un'immagine 224×224 può essere suddivisa in patch 16×16 non sovrapposte, ottenendo:

$$224/16 \times 224/16 = 14 \times 14 = 196 \text{ "patch"}$$

- **Appiattimento delle patch (Patch Flattening):**

Ogni patch viene appiattito in un vettore 1D.

Una patch di dimensione $P \times P \times C$ (es. $16 \times 16 \times 3$) viene trasformata in un vettore di dimensione $1 \times (P^2 \times C)$.

Per l'esempio dell'immagine di prima, si ottengono 196 vettori di patch, perché ogni patch diventa un vettore.

- **Incorporamento delle patch (Patch Embedding):**

Ogni patch appiattito viene proiettato in uno spazio a dimensione più alta (dimensione di embedding D) tramite una proiezione lineare apprendibile.

Questa trasformazione lineare permette al modello di imparare rappresentazioni più ricche per ogni patch.

Il risultato è una sequenza di embedding, ciascuno rappresentante una parte dell'immagine.

Ora l'immagine è rappresentata come una sequenza di **196 embedding vettoriali**, esattamente come una frase è rappresentata da una sequenza di parole in un Transformer NLP.

2 Positional Encoding

Il problema è che i **Transformer**, per natura, non hanno alcuna nozione dell'**ordine spaziale** dei dati in input.

- Nel linguaggio, non saprebbero distinguere se una parola viene prima o dopo.
- Nelle immagini, non saprebbero distinguere la posizione di un patch rispetto agli altri.

Per questo motivo serve aggiungere un **positional encoding**, cioè un'informazione che dica al modello **dove si trova ciascun patch**.

Positional Embedding:

I positional encoding vengono aggiunti a ciascun patch embedding per codificare informazioni sulla **posizione dei patch all'interno dell'immagine**.

Questo aiuta il modello a comprendere le **relazioni spaziali** tra i patch, in modo simile a come nei Transformer per NLP si codifica la posizione delle parole in una frase.

Supponiamo di avere un'immagine $224 \times 224 \times 3$, divisa in patch 16×16 .

- Numero di patch:

$$N = \frac{224}{16} \times \frac{224}{16} = 14 \times 14 = 196$$

- Ogni patch è trasformata in un embedding $x_i \in \mathbb{R}^D$, con $i = 0, 1, \dots, 195$.
- Formula generale del positional encoding

Per ogni patch i , l'input al Transformer diventa:

$$z_i = x_i + p_i$$

dove:

- x_i = embedding del patch (contenuto visivo)
- p_i = positional embedding (posizione del patch)
- z_i = embedding arricchito (contenuto + posizione)

Learned vs Fixed Positional Encoding:

Nei ViT, i positional encoding possono essere:

- **Learned Positional Encoding (usato nei ViT)**

In questo caso, ogni posizione i ha un vettore di embedding $p_i \in \mathbb{R}^D$ che viene appreso durante l'addestramento insieme agli altri pesi del modello.

La formula resta la stessa:

$$z_i = x_i + p_i$$

ma p_i non è calcolato con una funzione matematica $\rightarrow$ è un parametro del modello.

Nei ViT questa è la scelta più comune, perché i modelli possono imparare direttamente dai dati la rappresentazione posizionale più utile.

- **Fixed Positional Encoding (sinusoidale)**

Come nel Transformer originale (Vaswani et al., 2017), i positional encoding sono calcolati così:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/D}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/D}}\right)$$

dove:

- pos = indice della posizione del patch (da 0 a 195 nel nostro caso)
- i = indice della dimensione dell'embedding
- D = dimensione dell'embedding

Questo garantisce che ogni posizione abbia un pattern unico di valori sinusoidali

Nella maggior parte delle implementazioni di Vision Transformer si usano **learned positional encoding**.

Classification Token (CLS Token)

Nei Vision Transformer, viene introdotto all'inizio della sequenza di input un **token speciale di classificazione (CLS token)**. Questo token ha un ruolo fondamentale: raccoglie informazioni da **tutti i patch** durante i vari strati del Transformer. Il token CLS impara a rappresentare l'intera immagine prestando attenzione ai diversi patch tramite il meccanismo di self-attention.

All'uscita degli strati del Transformer, il token CLS viene estratto e passato a un classificatore per ottenere la predizione finale.

Dove si aggiunge

- All'inizio della sequenza di embedding dei patch.

Se abbiamo un'immagine 224×224 divisa in patch 16×16 :

- 196 patch $\rightarrow$ 196 embedding.
- A questi si aggiunge il [CLS].

La sequenza diventa:

$$Z_0 = [x_{CLS}, x_1 + p_1, x_2 + p_2, \dots, x_{196} + p_{196}]$$

dove:

- $x_{CLS} \in \mathbb{R}^D$ è il vettore del token di classificazione,
- x_i sono i patch embedding,
- p_i sono i positional encoding.

3. Strati dell'Encoder del Transformer

Una volta che i patch sono stati trasformati in embedding e arricchiti con informazioni posizionali, vengono passati attraverso una stack (pila) di strati encoder del Transformer.

Ogni encoder layer è composto da due blocchi principali:

1. Multi-Head Self-Attention (MSA)

Self-Attention

Il meccanismo di self-attention permette a ogni patch di "prestare attenzione" a tutti gli altri patch della sequenza.

- Questo consente al modello di catturare dipendenze a lungo raggio e relazioni tra parti diverse dell'immagine.
- In pratica, ogni patch calcola una combinazione pesata dei valori di tutti gli altri patch, in base alla loro somiglianza.

La formula è:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

dove:

- Q = Query (proiezione lineare degli embedding dei patch)
- K = Key (proiezione lineare degli embedding dei patch)
- V = Value (proiezione lineare degli embedding dei patch)
- d_k = dimensione delle Key (serve per la normalizzazione)

Interpretazione:

1. Si calcola il prodotto scalare QK^T , che misura la somiglianza tra un patch e gli altri.
2. Si normalizza con $\sqrt{d_k}$ e softmax $\rightarrow$ otteniamo pesi di attenzione.
3. Questi pesi vengono applicati ai valori $V \rightarrow$ producendo la rappresentazione aggiornata del patch.

Multi-Head Attention

- Invece di calcolare una sola self-attention, si calcolano più "teste" (heads) in parallelo.
- Ogni head apprende un modo diverso di guardare l'immagine (es. una può concentrarsi su colori, un'altra su forme, un'altra su bordi).
- Le uscite delle varie teste vengono concatenate e trasformate linearmente.

Formula:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

con:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

dove:

- h = numero di teste
- W_i^Q, W_i^K, W_i^V, W^O = matrici di proiezione apprendibili

2. Feed-Forward Neural Network (FFN)

Dopo il blocco di self-attention, ogni patch embedding passa attraverso una rete neurale feed-forward, applicata indipendentemente a ciascun token (stesso MLP per tutti i patch).

È composta da:

- due layer fully connected
- una funzione di attivazione non lineare (tipicamente GELU)

Formula con RELU:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

oppure con GELU:

$$\text{FFN}(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2$$

Questo passo permette al modello di applicare **trasformazioni non lineari ed estrarre feature più complesse**.

3. Residual Connections + Normalization

Ogni sotto-blocco (Multi-Head Self Attention e FFN) successivamente avrà:

- **Residual**: si somma l'input all'output del blocco → aiuta a non perdere informazioni.
- **Layer Normalization (LN)**: stabilizza il training.

Formula generica di un encoder layer:

$$z' = \text{LN}(x + \text{MultiHead Self Attention}(x))$$

$$z = \text{LN}(z' + \text{FFN}(z'))$$

Un ViT non usa un singolo encoder layer, ma una pila di L encoder (es. 12, 24), chiamata **Stacking Encoder Layers**.

Ogni layer successivo prende in input l'output del precedente, raffinando progressivamente la rappresentazione dei patch. In questo modo, il modello costruisce una rappresentazione gerarchica e astratta dell'immagine.

4. MLP Head (Classification Head)

- Dopo che i Transformer Encoder hanno elaborato la sequenza di patch e il token [CLS], si usa proprio l'output del token [CLS] per fare classificazione.
- Questo output viene passato in una rete MLP finale, composta in genere da 1 o 2 layer fully-connected.
- Alla fine si applica un softmax per produrre le probabilità delle classi (es. cane = 0.8, gatto = 0.1, auto = 0.1).

Formula finale:

$$y = \text{softmax}(W_{\text{cls}} \cdot z_{[\text{CLS}]} + b)$$

dove:

- $z_{[\text{CLS}]}$ = embedding finale del token [CLS]
- W_{cls}, b = parametri del classificatore
- y = distribuzione di probabilità sulle classi

Pipeline ViT:

1. Input immagine

- Immagine:

$$I \in \mathbb{R}^{224 \times 224 \times 3}$$

- Canali: 3 (RGB).
-

2. Patch splitting

- Patch size = 16×16 .
- Numero patch:

$$N = \frac{224}{16} \times \frac{224}{16} = 14 \times 14 = 196$$

 Ora abbiamo **196 patch**.

3. Flattening

Ogni patch:

$$16 \times 16 \times 3 = 768 \text{ valori}$$

- Ogni patch è appiattito in un vettore 1D:

$$p_i \in \mathbb{R}^{768}, \quad i = 1, \dots, 196$$

4. Linear Projection (Patch Embedding)

Si applica una matrice di embedding:

$$x_i = p_i W_E + b_E, \quad W_E \in \mathbb{R}^{768 \times D}, \quad b_E \in \mathbb{R}^D$$

 Con $D = 768$ (ViT-Base):

- Ogni patch diventa un embedding da **768 dimensioni**.
- Sequenza:

$$X = [x_1, x_2, \dots, x_{196}] \in \mathbb{R}^{196 \times 768}$$

📌 5. Aggiunta del [CLS] token

- Introduciamo il vettore apprendibile:

$$x_{CLS} \in \mathbb{R}^{768}$$

- Sequenza aggiornata:

$$X' = [x_{CLS}, x_1, x_2, \dots, x_{196}] \in \mathbb{R}^{197 \times 768}$$

📌 6. Positional Encoding

Ogni token riceve un positional embedding p_i :

$$Z^0 = X' + P, \quad P \in \mathbb{R}^{197 \times 768}$$

- 👉 Output iniziale al Transformer:

$$Z^0 \in \mathbb{R}^{197 \times 768}$$

📌 7. Transformer Encoder (12 layer)

Struttura di ogni layer

Ogni encoder layer ha:

1. LayerNorm
2. Multi-Head Self-Attention (MSA)
3. Residual connection
4. LayerNorm
5. Feed Forward Network (FFN, MLP)
6. Residual connection

a) Multi-Head Self-Attention

Per $Z^{l-1} \in \mathbb{R}^{197 \times 768}$:

1. Calcolo matrici:

$$Q = Z^{l-1}W^Q, \quad K = Z^{l-1}W^K, \quad V = Z^{l-1}W^V$$

con $W^Q, W^K, W^V \in \mathbb{R}^{768 \times 768}$.

2. Divisione in $h = 12$ teste:

- Ogni testa lavora su $d_k = 768/12 = 64$.
- Quindi $Q, K, V \in \mathbb{R}^{197 \times 768} \rightarrow \mathbb{R}^{197 \times 64}$ per testa.

3. Attenzione per una testa:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{64}}\right) V$$

- Matrice $QK^T \in \mathbb{R}^{197 \times 197}$: relazione tra tutti i token.
- Output per una testa: $\mathbb{R}^{197 \times 64}$.

4. Multi-Head:

$$\begin{aligned} \text{MHA}(Z^{l-1}) &= \text{Concat}(\text{head}_1, \dots, \text{head}_{12})W^O \\ &\in \mathbb{R}^{197 \times 768} \end{aligned}$$

b) Feed Forward Network (FFN)

Su ogni token $z \in \mathbb{R}^{768}$:

$$\text{FFN}(z) = \text{GELU}(zW_1 + b_1)W_2 + b_2$$

- $W_1 \in \mathbb{R}^{768 \times 3072}$ (espansione),
- $W_2 \in \mathbb{R}^{3072 \times 768}$ (compressione).
- Output: $\mathbb{R}^{768}$.

👉 Sequenza aggiornata: $\mathbb{R}^{197 \times 768}$.

🚩 8. Dopo 12 layer

Alla fine:

$$Z^{12} \in \mathbb{R}^{197 \times 768}$$

👉 La prima riga di Z^{12} è:

$$z_{CLS}^{(12)} = Z^{12}[0] \in \mathbb{R}^{768}$$

Questo è il riassunto globale dell'immagine.

🚩 9. MLP Head (Classificazione)

Il vettore $z_{CLS}^{(12)}$ viene passato al classificatore:

$$\hat{y} = \text{softmax}(W_{\text{cls}} z_{CLS}^{(12)} + b)$$

- $W_{\text{cls}} \in \mathbb{R}^{K \times 768}$ (dove K = numero classi).
- Output: $\hat{y} \in \mathbb{R}^K$.

👉 Esempio: se $K = 1000$ (ImageNet), otteniamo un vettore di 1000 probabilità.

🔗 Backpropagation (come funziona il training)

1. Loss function:

Se la classe vera è y , usiamo cross-entropy:

$$\mathcal{L} = - \sum_{k=1}^K y_k \log \hat{y}_k$$

2. Gradiente sull'output:

$$\frac{\partial \mathcal{L}}{\partial z_{CLS}^{(12)}} = W_{\text{cls}}^T (\hat{y} - y)$$

3. Propagazione all'indietro:

- L'errore si propaga dai pesi $W_{\text{cls}} \rightarrow$ al vettore $z_{CLS}^{(12)}$.
- Da $z_{CLS}^{(12)} \rightarrow$ indietro attraverso **12 layer encoder**:
 - aggiorna i parametri di **MSA** (W^Q, W^K, W^V, W^O),
 - aggiorna i parametri del **FFN** (W_1, W_2),
 - aggiorna gli embedding (W_E),
 - aggiorna il token **[CLS]** e i positional embedding se sono apprendibili.

4. Aggiornamento parametri:

Tutti i parametri si aggiornano via **SGD o Adam**:

$$\theta \leftarrow \theta - \eta \frac{\partial \mathcal{L}}{\partial \theta}$$

1. Local vs. Global Attention

- **CNN (Convolutional Neural Network):**
 - usa **filtri convoluzionali** (es. 3×3 , 5×5) che guardano solo una **piccola regione locale** dell'immagine.
 - il contesto globale si costruisce **gradualmente**, passando da strati superficiali (bordi, texture) a profondi (oggetti interi).
 - es: un kernel 3×3 vede solo 9 pixel alla volta.
- **ViT (Vision Transformer):**
 - usa **self-attention globale**: ogni patch può "parlare" con tutti gli altri fin dal primo layer.
 - → cattura direttamente relazioni **a lungo raggio** (es. collega la testa e la coda di un animale già nel primo strato).

Esempio:

- Una CNN vede prima **orecchie**, poi **occhi**, poi combina → "gatto".
- Un ViT può direttamente correlare "orecchie" e "coda" → "gatto" già nei primi layer.

2. Inductive Biases

- **CNN:**
 - hanno **bias induttivi forti**:
 - **località** (il contenuto vicino è più importante),
 - **invarianza alla traslazione** (un gatto in alto o in basso viene trattato uguale).
 - Questi bias rendono le CNN **efficienti**: imparano bene anche con pochi dati.
- **ViT:**
 - hanno **pochi bias**: il modello deve imparare da zero concetti come località o invarianza.
 - Questo le rende più **flessibili** (possono adattarsi a diversi tipi di dati), ma anche **più affamate di dati**.

3. Training Data Requirements

- **CNN:**
 - performano bene anche con dataset medio-piccoli (es. CIFAR-10, MNIST).
 - i bias incorporati aiutano ad apprendere pattern anche con pochi esempi.
- **ViT:**
 - richiedono dataset molto grandi (es. ImageNet-21k, JFT-300M) per allenarsi da zero.
 - Soluzione comune: **pretraining** su grandi dataset e **fine-tuning** su quello target.

Per questo, in pratica:

- CNN si usano ancora tanto quando i dati sono pochi.
- ViT dominano quando si hanno dataset enormi.

PERCEIVER

Il **Perceiver** è un modello di deep learning che nasce per **unificare l'elaborazione di input multimodali** (immagini, testo, audio, ecc.) usando **un'unica architettura basata sul Transformer**, senza dover costruire reti specializzate per ogni tipo di dato.

Idee Chiave

1. **Fondamenta: Transformer**
 - I Perceiver ereditano dai Transformer la capacità di lavorare con dati di natura diversa.
 - Non richiedono modifiche architetturali per cambiare dominio (immagine, testo, suono).
2. **Bias Induttivo Generico**
 - Il modello **non assume regole specifiche sul tipo di input**.
 - Può imparare direttamente dai dati reali senza vincoli strutturali forti (al contrario delle CNN, che si basano sulla località spaziale).
3. **Attenzione Globale**
 - I Transformer considerano **tutto il contesto dell'input** contemporaneamente.
 - Questo rende possibile catturare relazioni lunghe e complesse tra elementi anche distanti.
4. **Trattamento Flessibile della Posizione**
 - Nei Perceiver, la posizione non è rigidamente codificata come nelle CNN (dove convoluzione e pooling preservano la struttura spaziale).
 - La posizione viene aggiunta come informazione, e il modello **impara da sé le relazioni spaziali/temporali**.
5. **Condivisione dei Pesi**
 - L'uso di pesi condivisi riduce il costo computazionale.
 - Questo li rende efficienti anche su dataset grandi e adatti a GPU/TPU.

Problemi che Risolve

1. **Limite della Multimodalità**
 - I modelli classici sono progettati *ad hoc* (es. BERT per il testo, ResNet per le immagini).
 - Il Perceiver invece permette di usare **un'unica architettura per tutti i tipi di dati**.
2. **Collo di Bottiglia Quadratico dei Transformer**
 - Nei Transformer standard, il calcolo dell'attenzione cresce **$O(N^2)$** rispetto alla lunghezza dell'input (impraticabile per immagini ad alta risoluzione o audio lunghi).
 - Il Perceiver introduce un **set di variabili latenti** che interagiscono con l'input tramite attenzione.
 - Gli input "scrivono" in questi latenti.
 - Le operazioni successive avvengono nello spazio latente molto più piccolo → **complessità ridotta**, ma con capacità di rappresentare input grandi.

L'architettura **Perceiver** nasce come compromesso tra due mondi:

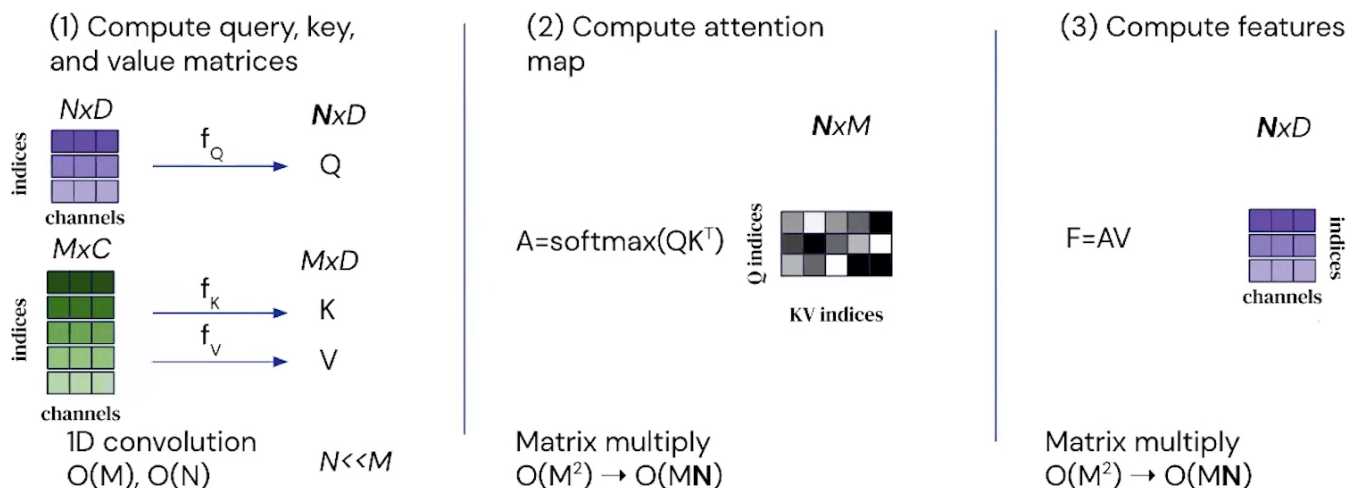
- le **ConvNet (CNN)** → molto **scalabili**, ma poco generali;
- i **Transformer** → molto **generali**, ma poco scalabili.

Il Perceiver cerca di prendere il meglio da entrambi:

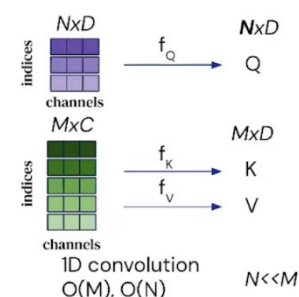
- la **scalabilità** tipica delle CNN (costo lineare con l'input);
- la **flessibilità** dei Transformer (capacità di gestire dati multimodali).

	ConvNets	Transformer
Applicabilità	Ottimale per dati strutturati a griglia, come immagini	Adatto per una vasta gamma di dati non strutturati e sequenziali, come testo, audio e immagini
Scalabilità	$O(MKL)$, lineare rispetto alla dimensione dell'input	$O(M^2 L)$, quadratica rispetto alla lunghezza della sequenza di input

Modifiche che sono state attuate per il Perceiver



Per prima cosa viene sostituito il concetto classico di **self-attention** con uno strato di **cross-attention** all'ingresso. Invece di calcolare i punteggi di attenzione confrontando ogni elemento dell'input con sé stesso (self-attention), la cross-attention confronta l'input con un set di latenti (o inducing points), che sono componenti appresi durante l'addestramento. I **latenti** funzionano come uno stato iniziale appreso simile a quello di una Rete Neurale Ricorrente (RNN), ma con una differenza fondamentale. Mentre in una RNN lo stato viene aggiornato passo dopo passo attraverso la sequenza, nei Perceiver i latenti vengono aggiornati attraverso iterazioni di elaborazione che riflettono l'intero input simultaneamente, sfruttando la cross-attenzione.



Quindi rispetto al concetto tradizionale dei Transformer:

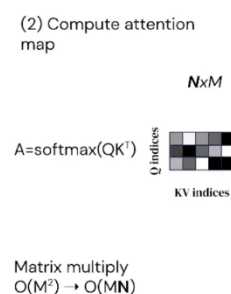
- **Input:**
 - o $M \times C$; come nei transformer con M numero di unità di input e C dimensione dell'embedding
 - o $N \times D$; introdotti dai Perceiver, dove N è il numero di latenti e D è la dimensione di ciascun latente
- **Configurazione di Q, K e V**
 - o **Query (Q)** $N \times D$: Le matrici delle query provengono dai latenti nel modello Perceiver, di numero fisso N , indipendentemente dalla dimensione dell'input M . La dimensione D rappresenta lo spazio di caratteristiche in cui i latenti (e quindi le query) esistono. Avere N diverso da M consente al modello di mantenere la complessità computazionale gestibile, poiché N è solitamente molto più piccolo di M , riducendo così il numero di calcoli necessari per l'attenzione.
 - o **Key (K) e Value (V)** $M \times D$: Le matrici delle chiavi e dei valori derivano direttamente dall'input, che ha M elementi, ognuno con una rappresentazione in uno spazio di caratteristiche di dimensione D . Questo significa che ogni elemento dell'input viene considerato sia per la generazione delle chiavi che dei valori, permettendo al modello di valutare l'intera estensione dell'input quando determina l'attenzione e produce l'output.

La complessità diventa da $O(M)$ a $O(M), O(N)$ in quanto la convoluzione 1D viene su entrambe le matrici in input. Ricordandoci che $N \ll M$.

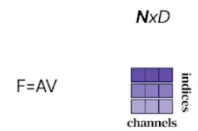
Quando viene calcolata la mappa di attenzione invece di essere all'interno di un unico insieme di dati (come i token di testo in un Transformer standard), il Perceiver calcola l'attenzione tra due insiemi distinti: i latenti e l'input.

Questo porta un cambiamento da una matrice quadratica ad una rettangolare.

Il calcolo dell'attenzione si fa quindi tra questi due insiemi, riducendo la complessità del problema da $O(M^2)$ a $O(MN)$, dove N (numero di latenti) è solitamente molto più piccolo di M .



Per quanto riguarda il calcolo delle feature abbiamo lo stesso comportamento del calcolo della mappa di attenzione. Questo riduce la complessità del problema da $O(M^2)$ a $O(MN)$

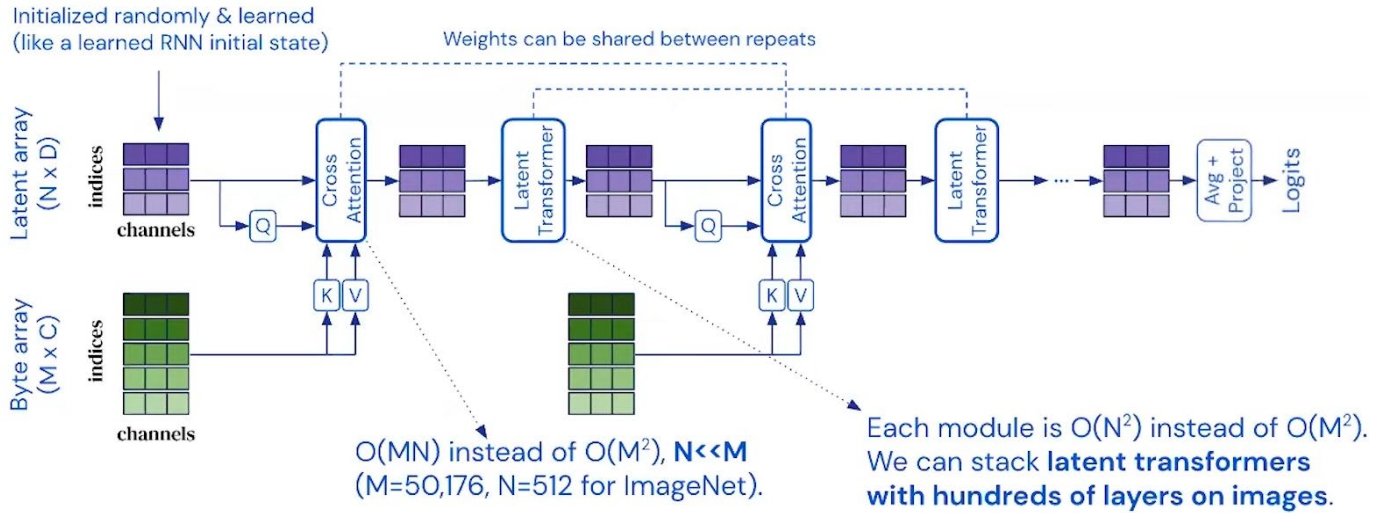


Matrix multiply
 $O(M^2) \rightarrow O(MN)$

Queste modifiche portano a questi vantaggi:

- **Complessità Lineare rispetto alla Dimensione dell'Input:** Uno dei principali vantaggi del Perceiver è che la complessità del calcolo dell'attenzione diventa lineare rispetto alla dimensione dell'input, $O(MN)$. Questo è reso possibile dalla trasformazione della matrice di attenzione da una matrice quadrata, tipica dei Transformer, in una matrice rettangolare. La dimensione dei latenti N può essere molto più piccola rispetto alla dimensione dell'input, riducendo così drasticamente il numero di calcoli necessari.
- **Mappatura Indipendente dalla Dimensione dell'Input:** Il Perceiver permette di trattare input che possono essere molto grandi trasformandoli in una rappresentazione più compatta attraverso i latenti. Questa capacità di condensare informazioni complesse in una forma gestibile è particolarmente vantaggiosa per applicazioni che richiedono l'elaborazione di dati ad alta dimensionalità come immagini, video o dati sensoriali complessi. L'Output è molto più piccolo rispetto all'immagine in input, quindi è più facile costruire deep network partendo dai latenti.
- **Controllo Completo tramite Iperparametri:** I latenti fungono da iperparametri che possono essere ottimizzati per bilanciare tra efficienza e prestazione. Questa flessibilità permette agli sviluppatori di adattare il modello in base alle specifiche esigenze dell'applicazione, ottimizzando per vari scenari di utilizzo senza dover ristrutturare l'architettura di base del modello.
- **Possibilità di Costruire Reti Profonde:** Grazie al controllo sui latenti e alla loro dimensione ridotta, è possibile impilare più layer di attenzione o altri processi di elaborazione senza incorrere in un aumento esponenziale della complessità computazionale. Questo permette al Perceiver di eseguire elaborazioni complesse e profonde, una capacità essenziale per apprendere rappresentazioni astratte e profonde dai dati.

Architettura Perceiver



Il Perceiver è così definito:

FORWARD PASS

Input → Fourier → Embedding → Latent Array → Cross-Attention → Residual + MLP → Latent Transformer → Residual + MLP → (ripetizione T volte) → Pooling → Classifier → Softmax → Loss

Input Processing

1. Immagine di Input (Byte Array)

L'input è un'immagine (o qualsiasi altro dato) trasformata in una matrice:

$$X \in \mathbb{R}^{M \times C}$$

- M = numero di elementi (per ImageNet: $224 \times 224 = 50.176$ pixel).
- C = canali (per un'immagine RGB, $C=3$).

Positional Encoding (Fourier features)

- Aggiungiamo $P = 256$ Fourier features:

$$X_{enc} = [X || \text{FourierFeatures}] \in \mathbb{R}^{M \times (C+P)}$$

- Nuova dimensione: $[M \times (259)]$.

Proiezione lineare input

- Peso:

$$W_{in} \in \mathbb{R}^{(C+P) \times D}, b_{in} \in \mathbb{R}^D$$

- Output:

$$X_{emb} = X_{enc}W_{in} + b_{in}, X_{emb} \in \mathbb{R}^{M \times D}$$

2. Latent Array

- Latenti:

$$L^{(0)} \in \mathbb{R}^{N \times D}$$

- Inizializzati con Truncated Normal:

$$L_{ij}^{(0)} \sim \mathcal{N}(0, \sigma^2), \text{ troncata in } [-2\sigma, 2\sigma]$$

Esempio ImageNet: $N = 512, D = 512$

Cross-Attention Block

1.Pre-Norm

$$\tilde{L} = \text{LayerNorm}(L^{(t)})$$

2. Proiezioni Q, K, V

- Query dai latenti:

$$Q = \tilde{L}W_Q, \quad W_Q \in \mathbb{R}^{D \times D}$$

- Keys e Values dall'input:

$$K = X_{emb}W_K, \quad V = X_{emb}W_V$$

$$\text{con } W_K, W_V \in \mathbb{R}^{D \times D}$$

3. Scaled Dot-Product Attention

a. MatMul tra Q e K = Scores

$$\text{Scores} = QK^T$$

b. Scaling per $\sqrt{D}$

$$\text{Scaled Scores} = \frac{\text{Scores}}{\sqrt{D}}$$

c. Masking (opzionale)

d. Softmax

$$\text{Attention Weights}_{ij} = \frac{e^{\text{Scaled Scores}_{ij}}}{\sum_{k=1}^M e^{\text{Scaled Scores}_{ik}}}$$

$$A = \text{Softmax}\left(\frac{QK^T}{\sqrt{D}}\right), A \in \mathbb{R}^{N \times M}$$

e. MatMul

$$Z = AV, Z \in \mathbb{R}^{N \times D}$$

Proiezione lineare dopo Scaled Dot-Product

$$Z' = ZW_O, W_O \in \mathbb{R}^{D \times D}$$

Residual

$$L^{(t)'} = \tilde{L} + Z'$$

Feed-Forward MLP

Pre-Norm:

$$\tilde{L} = \text{LayerNorm}(L^{(t)'})$$

Espansione:

$$H = (\tilde{L}W_1 + b_1), \quad W_1 \in \mathbb{R}^{D \times D_{ff}}$$

Per esempio in ImageNet $W_1 \in \mathbb{R}^{512 \times 2048}$

Attivazione non lineare:

RELU/GELU

$$H' = \sigma(H)$$

Compressione:

$$F = H'W_2 + b_2, \quad W_2 \in \mathbb{R}^{D_{ff} \times D}$$

Per esempio in ImageNet $W_1 \in \mathbb{R}^{2048 \times 512}$

Residual:

$$L^{(t)''} = L_{lat} = L^{(t)'} + F$$

Latent Transformer Block**Pre-Norm**

$$\tilde{L}_{lat} = \text{LayerNorm}(L^{(t)'})$$

Self-Attention (Q, K, V tutti dai latenti):

$$\begin{aligned} Q &= \tilde{L}_{lat} W_Q, & K &= \tilde{L}_{lat} W_K, & V &= \tilde{L}_{lat} W_V \\ A &= \text{Softmax}\left(\frac{QK^T}{\sqrt{D}}\right) & Z &= AV \end{aligned}$$

Proiezione finale

$$Z' = ZW_O, \quad W_O \in \mathbb{R}^{D \times D}$$

Residual:

$$\tilde{L}_{lat}' = \tilde{L}_{lat} + Z'$$

MLP + Residual:

$$\tilde{L}_{lat}'' = \tilde{L}_{lat}' + \text{MLP}\left(\text{LayerNorm}(\tilde{L}_{lat}')\right)$$

Ripetizione

- Alternanza Cross-Attention $\leftrightarrow$ Latent Transformer per T blocchi.
- Weight Sharing: stessi pesi riutilizzati $\rightarrow$ meno parametri.

Output Aggregation

- Pooling sui latenti:

$$h = \frac{1}{N} \sum_{i=1}^N L_i^{(T)}$$

Nel caso di ImageNet $N = 512$

Classification Head

- Lineare + Softmax:

$$y = \text{Softmax}(hW_c + b_c)$$

Loss

$$\mathcal{L} = - \sum_{k=1}^K y_k^{\text{true}} \log(y_k^{\text{pred}})$$

BACKWARD PASS

Loss → Gradiente su yyy → Classifier (W_c, b_c) → Pooling → Latenti finali → (Latent Transformer Block → Cross-Attention Block, al contrario T volte) → Input Embedding (W_{in}, b_{in}) → Latent Array iniziale

Loss → Gradiente

La derivata parte dalla funzione di costo (Cross-Entropy).

- Gradiente iniziale:

$$\frac{\partial \mathcal{L}}{\partial y}$$

Classification Head

- Parametri: W_c, b_c .
- Gradiente rispetto all'output del pooling h :

$$\frac{\partial \mathcal{L}}{\partial h} = \frac{\partial \mathcal{L}}{\partial y} W_c^T$$

- Gradiente sui pesi:

$$\frac{\partial \mathcal{L}}{\partial W_c} = h^T \frac{\partial \mathcal{L}}{\partial y}, \frac{\partial \mathcal{L}}{\partial b_c} = \frac{\partial \mathcal{L}}{\partial y}$$

Aggregation (Pooling)

Se usiamo average pooling sui latenti:

$$h = \frac{1}{N} \sum_{i=1}^N L_i^{(T)}$$

Il gradiente si distribuisce uniformemente:

$$\frac{\partial \mathcal{L}}{\partial L_i^{(T)}} = \frac{1}{N} \frac{\partial \mathcal{L}}{\partial h}$$

Latent Transformer Block

Il gradiente ora passa dentro ogni blocco Transformer sui latenti:

- **Residual**

Il gradiente si sdoppia: una parte passa dal ramo identità, l'altra dal ramo attivo (attenzione o MLP).

- **LayerNorm**

Backprop della normalizzazione: si calcolano derivate rispetto a media, varianza e input.

- **Self-Attention sui latenti**

Si calcolano:

$$\frac{\partial \mathcal{L}}{\partial Q}, \frac{\partial \mathcal{L}}{\partial K}, \frac{\partial \mathcal{L}}{\partial V}$$

e i gradienti sui pesi W_Q, W_K, W_V, W_O .

- **MLP**

Backprop classica di due layer lineari con attivazione ReLU/GELU:

$$\frac{\partial \mathcal{L}}{\partial W_1}, \frac{\partial \mathcal{L}}{\partial W_2}$$

Cross-Attention Block

- Gradiente su Q (latenti)

L'errore torna dentro L .

- Gradiente su K, V (input)

L'errore si propaga agli embedding dell'input X_{emb} .

- Aggiornamento parametri

Si calcolano i gradienti rispetto ai pesi del blocco:

$$\frac{\partial \mathcal{L}}{\partial W_Q}, \frac{\partial \mathcal{L}}{\partial W_K}, \frac{\partial \mathcal{L}}{\partial W_V}, \frac{\partial \mathcal{L}}{\partial W_O}$$

Qui i latenti restituiscono informazione ai dati originali.

Input Embedding

Dall'errore sui X_{emb} , il gradiente si propaga:

$$\frac{\partial \mathcal{L}}{\partial W_{in}}, \frac{\partial \mathcal{L}}{\partial b_{in}}$$

Gli embedding imparano a mappare meglio i dati grezzi nello spazio latente.

Latent Array iniziale

Anche i latenti iniziali $L^{(0)}$ sono trainabili.

$$\frac{\partial \mathcal{L}}{\partial L^{(0)}} \neq 0$$

Questo significa che non sono solo "slot vuoti": si ottimizzano insieme al resto.

Weight Sharing

Se i blocchi riusano gli stessi pesi:

- I gradienti calcolati in ogni copia vengono sommati.
- L'aggiornamento finale avviene su un'unica matrice di parametri condivisi.

L'architettura di un Perceiver parte dai due array in input:

Input → Fourier → Embedding → Latent Array → Cross-Attention → Residual + MLP → Latent Transformer → Residual + MLP → (ripetizione T volte) → Pooling → Classifier → Softmax → Loss

Immagine di Input (Byte Array)

L'input, rappresentato come un byte array $M \times C$, rappresenta un modo uniforme per gestire input eterogenei, indipendentemente dalla loro origine o modalità (immagini, testo, audio, ecc.).

Nel caso studiato per **ImageNet** abbiamo un'immagine RGB di dimensione 224×224 questa verrà processata:

- Ogni pixel dell'immagine è rappresentato da tre valori (canali RGB), quindi l'immagine può essere vista come una matrice di dimensioni $[224 \times 224](M) \times [3](C)$
- Il numero totale di pixel sarà $[224 \times 224] = 50176$
- Il numero totale di valori (byte) sarà $[224 \times 224] \times [3] = 150528$

L'immagine viene convertita in un array lineare o "appiattito" (mediante una **Convoluzione1D**).

Ogni pixel (con i suoi 3 canali RGB) dell'immagine viene linearizzato e questa viene trasformata in un array monodimensionale di lunghezza 150528

$$\text{Byte Array} = [r_1, g_1, b_1, r_2, g_2, b_2, \dots, r_{224 \times 224}, g_{224 \times 224}, b_{224 \times 224}]$$

Input → **Fourier** → Embedding → Latent Array → Cross-Attention → Residual + MLP → Latent Transformer → Residual + MLP → (ripetizione T volte) → Pooling → Classifier → Softmax → Loss

1. Positional Encoding (Fourier features)

Nota bene: Il Perceiver non lavora direttamente solo con i 3 canali RGB, ma arricchisce ciascun pixel con informazioni di posizione (x,y). Infatti, un pixel con valore [R,G,B] contiene solo informazione di colore, ma **non contiene informazione di posizione** (es: "questo rosso è in alto a sinistra o in basso a destra?"). Senza posizione, il modello non potrebbe distinguere due immagini identiche ma traslate.

Per fare ciò utilizziamo le **Fourier Features**. Queste sono una tecnica di mappatura che trasforma un input scalare o vettoriale $x \in \mathbb{R}^d$ in uno spazio a dimensionalità più alta, applicando funzioni seno e coseno a frequenze multiple.

Formalmente, data una matrice di frequenze $B \in \mathbb{R}^{m \times d}$, la mappatura delle Fourier features è definita come:

dove:

$$\gamma(x) = [\cos(2\pi Bx), \sin(2\pi Bx)] \in \mathbb{R}^{2m}$$

- x è il vettore di input (d-dimensionale),
- B contiene i vettori di frequenze (scelti in modo deterministico o campionati da una distribuzione, ad esempio gaussiana),
- il risultato è un embedding di dimensione $2m$, in cui ogni componente dell'input viene rappresentato tramite combinazioni sinusoidali a diverse scale.

Sia (x, y) la posizione normalizzata di un pixel (tipicamente in $[-1,1] \circ [0,1]$).

Per una serie di frequenze $f_1, f_2, \dots, f_L$, calcoliamo:

$$\phi(x) = [\sin(2\pi f_1 x), \cos(2\pi f_1 x), \dots, \sin(2\pi f_L x), \cos(2\pi f_L x)]$$

$$\phi(y) = [\sin(2\pi f_1 y), \cos(2\pi f_1 y), \dots, \sin(2\pi f_L y), \cos(2\pi f_L y)]$$

- Se usiamo $L = 64$ frequenze, avremo per $\phi(x)$ 64 sin e 64 cos, per $\phi(y)$ 64 sin e 64 cos quindi generiamo per due coordinate $(x, y) \rightarrow 256$ valori in totale (128 per $\phi(x)$, 128 per $\phi(y)$)

Si scelgono L frequenze che crescono in maniera esponenziale:

$$f_k = 2^k, k = 0, 1, \dots, L - 1$$

- Esempio: 1,2,4,8,16, ...

Con $L = 64$, hai 64 frequenze per coordinata $\rightarrow 128$ valori (sin + cos).

Vantaggio: cattura sia **scale globali che scale locali**.

Quindi ogni pixel diventa:

$$[R, G, B, \text{pos}_1, \text{pos}_2, \dots, \text{pos}_{256}]$$

Per questo C passa da essere con valori di input grezzi da 3 a 259 (R,G,B + posizioni)

Quindi attualmente abbiamo un Byte Array di dimensione, $M \times C$ (50176×259)

$$\text{Byte Array} \in \mathbb{R}^{50176 \times 259}$$

Input → **Fourier** → **Embedding** → Latent Array → Cross-Attention → Residual + MLP → Latent Transformer → Residual + MLP → (ripetizione T volte) → Pooling → Classifier → Softmax → Loss

2. Proiezione lineare input

Proiezione Lineare 259 → 512

Applichiamo una **proiezione lineare** per passare dallo spazio C (259) alla dimensione D (512) per K e V

Perché passare da uno spazio 259 → 512?

- **Uniformità del modello**
Tutti i blocchi successivi (Q, K, V, latenti) lavorano con embedding fissi $D=512$. Usare 512 evita incoerenze dimensionali.
- **Maggiore capacità rappresentativa**
Proiettare a 512 offre più spazio per codificare pattern complessi, rispetto ai 259 originali.
- **Compatibilità con i latenti**
I latenti hanno dimensione 512: serve che anche l'input sia mappato in 512 per poter calcolare QK^T .
- **Stabilità del training**
La proiezione lineare (con Xavier init) distribuisce meglio l'informazione, riducendo instabilità.
- **Combinazione colore + posizione**
Ogni dimensione in 512 è una miscela appresa di RGB e Fourier, che integra informazione visiva e posizionale.
Sia x il vettore riga di C quindi

$$x \in \mathbb{R}^{259}$$

Si applica:

$$W_{\text{embed}} \in \mathbb{R}^{512 \times 259}, b \in \mathbb{R}^{512}$$

$$x_{\text{embed}} = W_{\text{embed}} \cdot x + b$$

Così ogni pixel passa da:

$$x \in \mathbb{R}^{259} \rightarrow x_{\text{embed}} \in \mathbb{R}^{512}$$

Input → **Fourier** → **Embedding** → **Latent Array** → Cross-Attention → Residual + MLP → Latent Transformer → Residual + MLP → (ripetizione T volte) → Pooling → Classifier → Softmax → Loss

Latent Array:

È una **matrice di vettori latenti** $L \in \mathbb{R}^{N \times D}$ **interamente appresa** (parametri del modello).

- **N** = numero di latenti (slot/ "unità di memoria").
- **D** = dimensione del canale/embedding dei latenti

Questi vettori **non derivano** direttamente dall'input e **non hanno un legame spaziale** con token/pixel specifici: sono una base compatta che il modello usa per riassumere l'input.

Nel caso specifico del Perceiver per ImageNet, l'array latente è $N = 512$ ($N \ll M(50176)$) e $D = 512$.

L'array latente $L \in \mathbb{R}^{512 \times 512}$ (per ImageNet) è inizializzato campionando ciascun elemento da una distribuzione:

$$L_{ij} \sim \mathcal{N}(0, 0.02^2), \text{ troncata a } [-2, 2].$$

In pratica i numeri dentro L sono quasi tutti piccolissimi, tipicamente compresi fra -0.05 e +0.05 (perché la gaussiana è molto stretta attorno a 0).

Il troncamento a $[-2, 2]$ esiste per sicurezza, ma con $\sigma = 0.02$ è rarissimo che si producano valori oltre 0.1 - quindi quasi mai entra davvero in gioco.

Questi numeri sono i vettori di partenza dei 512 slot latenti: ognuno è un vettore di 512 valori inizializzati così.

$$L = \begin{bmatrix} 0.012 & -0.004 & 0.021 & \dots & -0.017 \\ -0.009 & 0.015 & 0.003 & \dots & 0.008 \\ 0.020 & 0.001 & -0.014 & \dots & 0.027 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ -0.006 & 0.022 & -0.010 & \dots & 0.004 \end{bmatrix}_{512 \times 512}$$

Perché?

- **Evitare gradienti instabili:** se i latenti partissero con valori grandi, i prodotti QK^T sarebbero enormi, la softmax saturerebbe, e i gradienti diventerebbero instabili.
- **Garantire neutralità:** media zero significa che nessun latente ha un “vantaggio” iniziale su un altro.
- **Sicurezza numerica:** il troncamento assicura che non ci siano valori anomali che potrebbero falsare il training nelle prime epoche.
- **Permettere specializzazione graduale:** partendo con valori piccoli e neutri, i latenti vengono “riempiti” progressivamente dall’informazione dell’input, e col tempo ciascuno si specializza (bordo, colore, texture, pattern globali...).

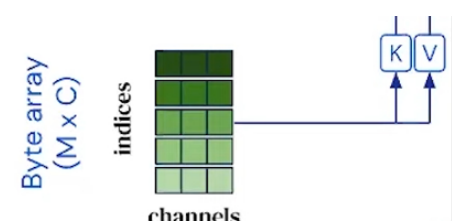
Input → **Fourier** → **Embedding** → **Latent Array** → **Cross-Attention** → Residual + MLP → Latent Transformer → Residual + MLP → (ripetizione T volte) → Pooling → Classifier → Softmax → Loss

Cross-Attention Block

Keys (K) e Values (V):

Il **byte array** ha dimensione $M \times C$ (50176×512) dopo la trasformazione mediante proiezione lineare di C da 259 a 512.

Per passare dal byte array alle matrici **K (Key)** e **V (Value)** utilizziamo due matrici di peso apprese W_k e W_v di dimensione $C \times D$ (512×512)



$$W_k \in \mathbb{R}^{C \times D} = \mathbb{R}^{512 \times 512}$$

$$W_v \in \mathbb{R}^{C \times D} = \mathbb{R}^{512 \times 512}$$

Vengono definire **matrici di peso** apprese in quanto i valori ottenuti derivano da trasformazioni lineari

$$y = W \cdot x + b$$

dove W è una **matrice di pesi** e b un **vettore di bias**.

Queste sono definire “**apprese**” in quanto W e b non contengono conoscenza della rete, ma sono inizializzati con piccoli numeri casuali (es. Xavier Uniform).

Quando creiamo le matrici di peso in PyTorch (es. nn.Linear), se non diciamo nulla, vengono inizializzate automaticamente con **Xavier Uniform**. Questa scelta serve ad **evitare che i valori nei layer esplodano o collassino a zero** man mano che i segnali attraversano la rete.

Xavier Uniform (per W_k e W_v)

- Formula:

$$W_{ij} \sim U(-a, a), \quad a = \sqrt{\frac{6}{fan_{in} + fan_{out}}}$$

- Dove:

- $fan_{in} = C$ (dimensione input),
- $fan_{out} = D$ (dimensione output).

$$U(-a, a)$$

Significa **distribuzione uniforme** tra $-a$ e $+a$.

- Nel nostro caso:

$$C = D = 512 \Rightarrow a = \sqrt{\frac{6}{1024}} \approx 0.076$$

- Quindi ogni peso di $W_k, W_v \in \mathbb{R}^{512 \times 512}$ è scelto **uniformemente** tra:

$$[-0.076, +0.076]$$

Solo successivamente mediante backpropagation e ottimizzazione allora avrò conoscenza sulla rete. Questo viene fatto mediante la riduzione della differenza tra predizione e loss.

Quindi per calcolare K e V :

$$\text{Byte Array} \in \mathbb{R}^{50176 \times 259}$$

$$W_k \in \mathbb{R}^{259 \times 512}, \quad W_v \in \mathbb{R}^{259 \times 512}$$

$$K = \text{Byte Array} \cdot W_k \in \mathbb{R}^{50176 \times 512}$$

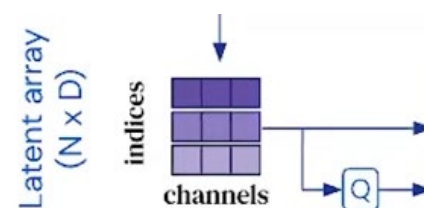
$$V = \text{Byte Array} \cdot W_v \in \mathbb{R}^{50176 \times 512}$$

Il risultato è che K, V hanno dimensione $M \times D$

Query (Q):

Ogni elemento del latent array viene trasformato in Query (Q) tramite una matrice di peso W_q di dimensione $D \times D$:

$$Q = W_q \cdot \text{Latent Array}$$



Latent Array L

$$L \in \mathbb{R}^{N \times D} = \mathbb{R}^{512 \times 512}$$

Queries (Q)

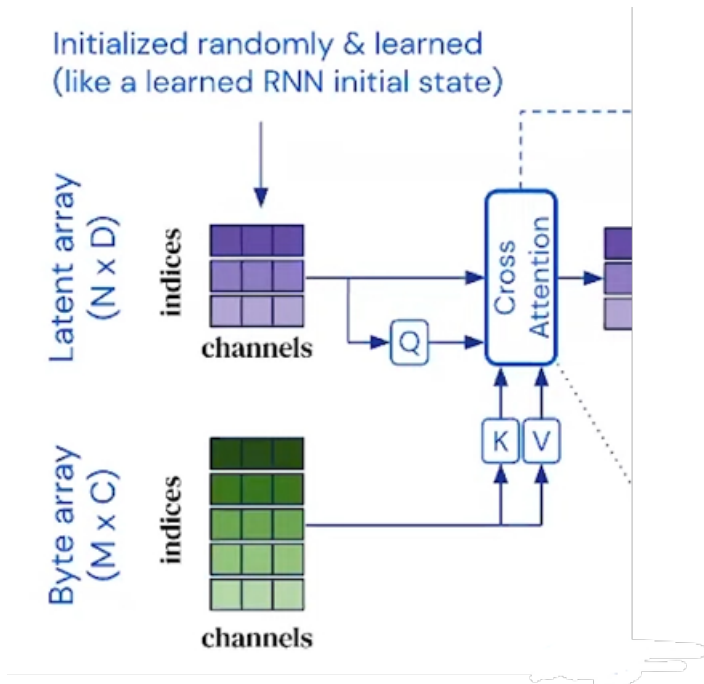
$$W_q \in \mathbb{R}^{D \times D} = \mathbb{R}^{512 \times 512}$$

$$Q = L \cdot W_q^T + b_q$$

$$L \in \mathbb{R}^{512 \times 512}$$

$$W_q^T \in \mathbb{R}^{512 \times 512}$$

$$Q \in \mathbb{R}^{512 \times 512}$$



- **Byte Array:** $\mathbb{R}^{50176 \times 259}$
- **Embedding lineare** ($W_{\text{embed}} \in \mathbb{R}^{512 \times 259}$):
 $X_{\text{embed}} \in \mathbb{R}^{50176 \times 512}$
- **Latent Array L:** $\mathbb{R}^{512 \times 512}$
- **Queries Q** ($W_q \in \mathbb{R}^{512 \times 512}$):
 $Q \in \mathbb{R}^{512 \times 512}$
- **Keys K** ($W_k \in \mathbb{R}^{512 \times 512}$):
 $K \in \mathbb{R}^{50176 \times 512}$
- **Values V** ($W_v \in \mathbb{R}^{512 \times 512}$):
 $V \in \mathbb{R}^{50176 \times 512}$
- **Attn matrix** (QK^T):
 $\mathbb{R}^{512 \times 50176}$
- **Output cross-attn** ($\text{Attn} \cdot V$):
 $\mathbb{R}^{512 \times 512}$

L'interazione tra il **byte array** (l'immagine) e il **latent array** avviene attraverso il modulo di **cross-attention**.

Scaled-Dot Product Attention

Perché facciamo la scaled-dot product attention (ovvero calcoliamo i punteggi d'attenzione)?

1. Ogni latente è una Query (Q): rappresenta una "domanda" del tipo: "Quale parte dell'immagine mi serve?".
2. Ogni token d'input è una Key (K): rappresenta una "descrizione" del contenuto del token.
3. Il prodotto scalare $Q_i \cdot K_j$ misura quanto la domanda del latente i-esimo è compatibile con la descrizione dell'input j-esimo.

Se è grande → il latente "è interessato" a quel token.

Se è piccolo → il latente lo ignora.

Supponiamo:

- Latente 1 cerca "colore rosso".
- Input ha 3 pixel:
 - Pixel A: rosso
 - Pixel B: verde
 - Pixel C: rosso+verde

I punteggi di attenzione funzionano così:

- Latente 1 calcola similarità con A, B, C.
 - A è molto rosso → punteggio alto.
 - B non è rosso → punteggio basso.
 - C è un po' rosso → punteggio medio.

Dopo softmax:

- Latente 1 guarda soprattutto A, un po' C, quasi per nulla B.

Risultato: il latente si aggiorna con informazione **mirata ai token che gli servono**.

Pipeline della scaled-dot product attention

1. **Input** Q, K e V

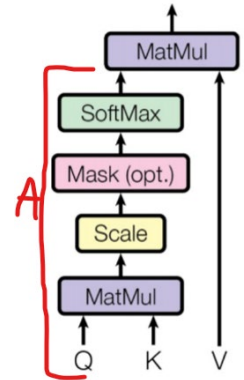
2. **MatMul** tra Q e K = Scores

$$\text{Scores} = QK^T$$

- $Q \in \mathbb{R}^{512 \times 512}$
- $K^T \in \mathbb{R}^{512 \times 50176}$
- 🐼 Scores $\in \mathbb{R}^{512 \times 50176}$

Interpretazione: ogni riga = un latente, ogni colonna = un token dell'immagine.

Il prodotto scalare $Q_i \cdot K_j$ misura quanto la domanda del latente i-esimo è compatibile con la descrizione dell'input j-esimo.



3. **Scaling** per $\sqrt{D}$

$$\text{Scaled Scores} = \frac{\text{Scores}}{\sqrt{512}} \approx \frac{\text{Scores}}{22.6}$$

🐼 Dimensioni rimangono $\mathbb{R}^{512 \times 50176}$.

Serve solo per evitare che i valori enormi saturino la softmax.

4. **Masking (opzionale)**, applico una maschera ai punteggi prima di passare i risultati al SoftMax

$$\text{Masked Scaled Scores}_{ij} = \begin{cases} \text{Scaled Scores}_{ij}, & \text{se Mask}_{ij} = 1 \\ -\infty, & \text{se Mask}_{ij} = 0 \end{cases}$$

Dove ho 1 se l'elemento è rilevante per la query e 0 se l'elemento deve essere ignorato

5. Calcolo della **SoftMax**, per ogni riga i calcolo la probabilità per ogni colonna j .

Perché serve la Softmax?

I valori di Scaled Scores sono numeri qualsiasi (anche negativi).

Noi vogliamo trasformarli in pesi di attenzione con le seguenti proprietà:

- Essere positivi (≥ 0)
- Sommare a 1 per ogni latente

In questo modo diventano una distribuzione di probabilità in cui il latente i "decide" quanto guardare ciascun token j

$$Attention\,Weights_{ij} = \frac{e^{Scaled\,Scores_{ij}}}{\sum_{k=1}^M e^{Scaled\,Scores_{ik}}}$$

Dove ij rappresenta quanto la query Q_i presta attenzione alla key K_j e ik è il resto della riga per normalizzare il valore e serve a calcolare la somma totale delle probabilità per la query Q_i

6. Secondo **MatMul** tra *Attention Weights* (A) e V , cioè $Z = AV$.

Questo serve a prendere i contenuti V degli input e combinarli **secondo i pesi di attenzione** calcolati prima. È il passo in cui i latenti “assorbono” davvero l’informazione dagli input.

- **A**: i pesi di attenzione, cioè *quanto ogni latente guarda ogni token dell’input*.
→ A_{ij} = quanto il latente i “si fida” del token j .
- **V**: i Value (contenuto informativo dei token input).
→ V_j = cosa contiene il token j .

👉 Lo scopo ora è: **costruire la nuova rappresentazione dei latenti** prendendo davvero l’informazione dai Value, pesata dall’attenzione calcolata.

$$Z_{512 \times 512} = A_{512 \times 50176} \cdot V_{50176 \times 512}$$

📌 Quindi:

- $A \in \mathbb{R}^{512 \times 50176}$ (pesi di attenzione)
- $V \in \mathbb{R}^{50176 \times 512}$ (valori dei token)
- $Z \in \mathbb{R}^{512 \times 512}$ (nuovi latenti)

Proiezione lineare dopo Scaled Dot-Product

$$Z' = ZW_o, \quad W_o \in \mathbb{R}^{512 \times 512}$$

👉 Output: $Z' \in \mathbb{R}^{512 \times 512}$

Perché serve anche qui (anche se le dimensioni non cambiano)

- ◆ **Flessibilità**: il modello può ricombinare le feature di Z .
- ◆ **Coerenza**: la struttura rimane la stessa usata nel multi-head → più modulare.
- ◆ **Preparazione al residual**: assicura che Z' e L abbiano stessa dimensione per poter fare $L + Z'$.

In single-head, la proiezione è **ridondante dal punto di vista dimensionale**, ma **fondamentale per l’apprendimento**

Caso Multi-Head

Ogni testa produce:

$$Z_h = A_h V_h \in \mathbb{R}^{512 \times 64}$$

Concatenando le 8 teste:

$$Z = \text{Concat}(Z_1, \dots, Z_8) \in \mathbb{R}^{512 \times 512}$$

Proiezione lineare

Anche qui applichi:

$$Z' = Z W_o$$

con $W_o \in \mathbb{R}^{512 \times 512}$.

Dimensioni:

$$Z' \in \mathbb{R}^{512 \times 512}$$

Perché è indispensabile

- ♦ **Normalizzazione della concatenazione:** senza W_o , avresti un "blocco" di feature derivanti dalle diverse teste, non rimescolate tra loro.
- ♦ **Mixing delle teste:** W_o permette di combinare l'informazione proveniente dalle varie teste in un'unica rappresentazione coerente.
- ♦ **Compatibilità:** porta l'output nello stesso spazio dei latenti L per la residual connection.

• Nel multi-head, la proiezione non è solo utile: è **necessaria** per unire davvero le informazioni delle teste.

Input → **Fourier** → **Embedding** → **Latent Array** → **Cross-Attention** → **Residual** + MLP → Latent Transformer → Residual + MLP → (ripetizione T volte) → Pooling → Classifier → Softmax → Loss

Residual

A questo punto abbiamo:

$$Z' \in \mathbb{R}^{512 \times 512}$$

Latent array originale normalizzato $\tilde{L} \in \mathbb{R}^{512 \times 512}$

Facciamo la somma:

$$L^{(t)'} = \tilde{L} + Z'$$

Con $t = 0$ essendo il primo blocco di cross-attention

Questo step serve a:

- Non perdere l'informazione originaria dei latenti.
- Previene che l'output degeneri se l'attenzione "sbaglia": nel peggiore dei casi, il modello può imparare a far sì che $Z' \approx 0$, mantenendo intatto L .

- Rende il flusso del gradiente più stabile (evita problemi di vanishing gradient nei modelli molto profondi).

$L^{(t)'}$ è il residuo: il modello mantiene l'informazione originale di L e ci aggiunge solo una correzione/aggiornamento data dall'attenzione

Input → **Fourier** → **Embedding** → **Latent Array** → **Cross-Attention** → **Residual** + **MLP** → Latent Transformer → Residual + MLP → (ripetizione T volte) → Pooling → Classifier → Softmax → Loss

Feed Forward Network (MLP)

Pre-Norm:

$$\tilde{L} = \text{LayerNorm}(L^{(t)'})$$

La LayerNorm prende ogni vettore latente (cioè ogni riga di L') e:

1. Calcola la media e la deviazione standard dei suoi valori.
2. Normalizza ogni valore sottraendo la media e dividendo per la deviazione standard (aggiungendo ϵ per stabilità numerica).
3. Applica due parametri appresi (γ, β) per scalare e traslare il risultato.

Formula

$$\tilde{L}_{ij} = \gamma_j \cdot \frac{L'_{ij} - \mu_i}{\sigma_i + \epsilon} + \beta_j$$

Definizioni:

- L'_i = i-esimo latente (vettore di dimensione D)
- $\mu_i = \frac{1}{D} \sum_{j=1}^D L'_{ij}$ = media
- $\sigma_i = \sqrt{\frac{1}{D} \sum_{j=1}^D (L'_{ij} - \mu_i)^2}$ = deviazione standard
- $\epsilon \in \mathbb{R}$ = piccolo valore costante (es. 10^{-5}) per stabilità numerica
- $\gamma, \beta \in \mathbb{R}^D$ = parametri appresi

Ciò che ottengo è

$$\tilde{L} \in \mathbb{R}^{512 \times 512},$$

cioè, i latenti stabilizzati e pronti a entrare nello step successivo

Perché serve una MLP:

- Evita che i valori nei latenti crescano senza controllo o collassino a 0.
- Rende più stabile il training (la distribuzione dei valori resta bilanciata).
- Permette al modello di riutilizzare i latenti in più blocchi senza "deriva numerica".

Perché aggiungiamo una MLP?

- L'attenzione ($Z = AV$) serve a mescolare informazione tra token/latenti, pesando quanto ogni elemento deve guardare gli altri.
- Però, matematicamente, **l'attenzione è una combinazione lineare dei valori V** .
- Se ci fermassimo lì, ogni latente sarebbe solo un "mix lineare" di altri → potere espressivo limitato.

- Con $W_1, W_2 + \text{ReLU/GELU}$, il **modello diventa capace di rappresentare funzioni non lineari complesse**.

Il **Feed Forward** è un sottoblocco applicato **indipendentemente ad ogni latente** (cioè riga per riga del latent array).

Non c'è interazione tra i latenti: ognuno viene trasformato nello stesso modo da una rete neurale *pointwise*.

Formula generale:

$$F = \text{MLP}(\tilde{L}) = \sigma(\tilde{L}W_1 + b_1)W_2 + b_2$$

- $\tilde{L} \in \mathbb{R}^{N \times D} \rightarrow \text{input (qui } 512 \times 512\text{)}.$
- $W_1 \in \mathbb{R}^{512 \times 2048}, b_1 \in \mathbb{R}^{2048}.$
- $W_2 \in \mathbb{R}^{2048 \times 512}, b_2 \in \mathbb{R}^{512}.$
- σ = funzione di attivazione non lineare (ReLU o GELU).

1. Espansione (Linear 1)

- Ogni vettore latente da 512 dimensioni viene proiettato in uno spazio più ampio (2048).
- Formula:

$$H = \tilde{L}W_1 + b_1$$

- Dimensioni: $H \in \mathbb{R}^{512 \times 2048}.$
- Scopo: permettere più combinazioni lineari (più capacità rappresentativa).

2. Attivazione non lineare

- Si applica ReLU/GELU elemento per elemento:

$$H' = \sigma(H)$$

- Mantiene dimensioni: $512 \times 2048.$
- Scopo: introdurre non linearità $\rightarrow$ il modello non si limita a combinazioni lineari, ma può modellare funzioni complesse.

3. Compressione (Linear 2)

- Si proietta di nuovo nello spazio originale di 512 dimensioni:

$$F = H'W_2 + b_2$$

- Dimensioni: $F \in \mathbb{R}^{512 \times 512}.$
- Scopo: riportare l'output nella stessa dimensione dei latenti per poter fare residual connection.

Residual connection

In output dalla MLP: $F \in \mathbb{R}^{512 \times 512}$, mentre come input avevo (prima della MLP) $\tilde{L} \in \mathbb{R}^{512 \times 512}$

- Residual = somma:

$$L'' = L_{lat} = \tilde{L} + F$$

Significa che **non buttiamo via l'input originale, ma aggiungiamo il contributo appreso dalla MLP**. Così se la MLP non impara niente di utile, almeno il flusso dell'informazione rimane intatto.

Quindi alla fine del blocco di MLP ottengo $L'' \in \mathbb{R}^{512 \times 512}$

Layer Normalization

Dopo la residual, applichiamo di nuovo una LayerNorm:

$$\hat{L} = \text{LayerNorm}(L'')$$

- Mantiene la dimensione: 512×512 .
- Serve per stabilizzare l'allenamento, evitando che i valori crescano troppo o collassino.

Riassunto Blocco Cross Attention - con formule e dimensioni numeriche

Input

- Latent array

$$L \in \mathbb{R}^{512 \times 512}$$

- Byte array (immagine proiettata)

$$X \in \mathbb{R}^{50176 \times 512}$$

1. Proiezioni lineari

$$\begin{aligned} Q &= LW_q, W_q \in \mathbb{R}^{512 \times 512}, Q \in \mathbb{R}^{512 \times 512} \\ K &= XW_k, W_k \in \mathbb{R}^{512 \times 512}, K \in \mathbb{R}^{50176 \times 512} \\ V &= XW_v, W_v \in \mathbb{R}^{512 \times 512}, V \in \mathbb{R}^{50176 \times 512} \end{aligned}$$

2. Attention Scores

$$S = \frac{QK^T}{\sqrt{512}}, S \in \mathbb{R}^{512 \times 50176}$$

3. Softmax (pesi di attenzione)

$$A_{ij} = \frac{e^{S_{ij}}}{\sum_{k=1}^{50176} e^{S_{ik}}}, A \in \mathbb{R}^{512 \times 50176}$$

4. Aggregazione

$$Z = AV, Z \in \mathbb{R}^{512 \times 512}$$

5. Proiezione lineare finale

$$Z' = ZW_o, W_o \in \mathbb{R}^{512 \times 512}, Z' \in \mathbb{R}^{512 \times 512}$$

6. Residual Connection (post-attention)

$$L' = L + Z', L' \in \mathbb{R}^{512 \times 512}$$

7. Layer Normalization (post-attention)

$$\begin{aligned} \tilde{L}_{ij} &= \gamma_j \cdot \frac{L'_{ij} - \mu_i}{\sigma_i + \epsilon} + \beta_j \\ \tilde{L} &\in \mathbb{R}^{512 \times 512} \end{aligned}$$

8. Feed Forward (MLP a 2 layer)

Espansione

$$H = \tilde{L}W_1 + b_1, W_1 \in \mathbb{R}^{512 \times 2048}, H \in \mathbb{R}^{512 \times 2048}$$

Attivazione (ReLU/GELU)

$$H' = \sigma(H), H' \in \mathbb{R}^{512 \times 2048}$$

Compressione

$$F = H'W_2 + b_2, W_2 \in \mathbb{R}^{2048 \times 512}, F \in \mathbb{R}^{512 \times 512}$$

9. Residual Connection (post-MLP)

$$L'' = \tilde{L} + F, L'' \in \mathbb{R}^{512 \times 512}$$

10. Layer Normalization (post-MLP)

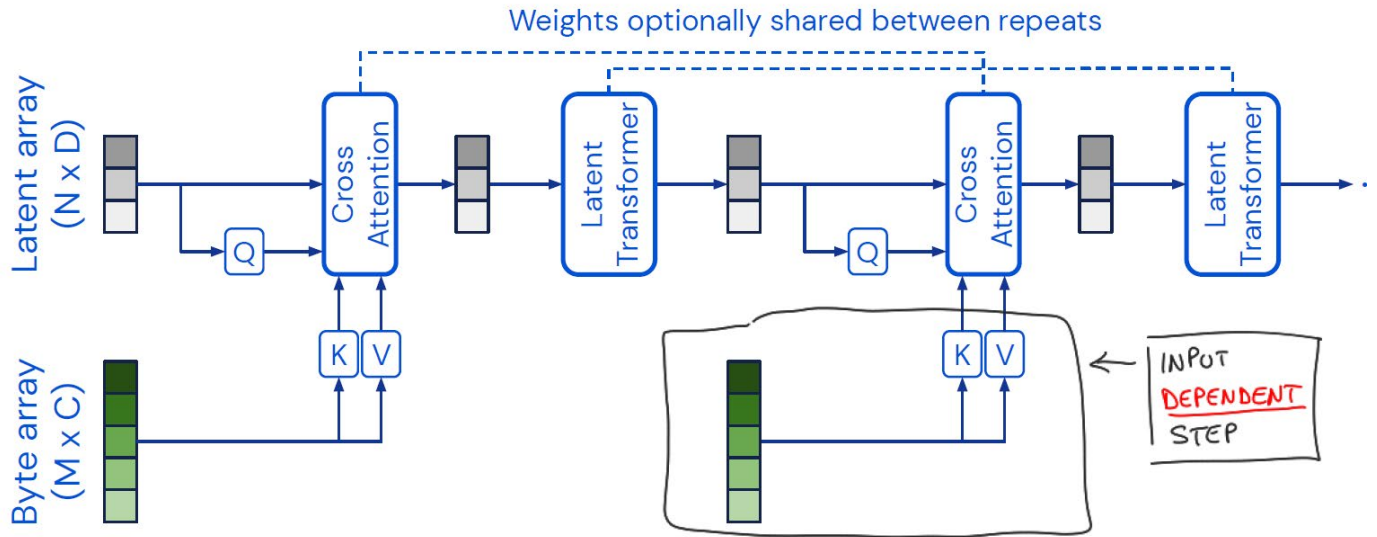
$$\begin{aligned} \hat{L}_{ij} &= \gamma_j \cdot \frac{L''_{ij} - \mu_i}{\sigma_i + \epsilon} + \beta_j \\ \hat{L} &\in \mathbb{R}^{512 \times 512} \end{aligned}$$

Output del blocco

$$\hat{L} \in \mathbb{R}^{512 \times 512}$$

Input → Fourier → Embedding → Latent Array → Cross-Attention → Residual + MLP → Latent Transformer → Residual + MLP → (ripetizione T volte) → Pooling → Classifier → Softmax → Loss

Latent Transformer



Input

$$\hat{L} \in \mathbb{R}^{512 \times 512}$$

(latenti normalizzati usciti dal Cross Attention Block).

1. Layer Normalization (pre-self-attention)

$$\tilde{L}_{ij} = \gamma_j \cdot \frac{\hat{L}_{ij} - \mu_i}{\sigma_i + \epsilon} + \beta_j, \tilde{L} \in \mathbb{R}^{512 \times 512}$$

- Normalizziamo ogni latente.
- Dimensione invariata.

La normalizzazione viene rieseguita perché prima di ogni blocco di latent e cross si normalizza in quanto il blocco di latent non si fida del cross attention

2. Calcolo di Q,K,V (provenienti dai latenti)

Ora andranno ricalcolati Q, K, V, ma essendo ora in un blocco di Latent Transformer, ci comporteremo come nel caso della self-attention dove come input abbiamo $\tilde{L}$, quindi avremo:

$$\begin{aligned} Q &= \tilde{L}W_q, & W_q &\in \mathbb{R}^{512 \times 512}, & Q &\in \mathbb{R}^{512 \times 512} \\ K &= \tilde{L}W_k, & W_k &\in \mathbb{R}^{512 \times 512}, & K &\in \mathbb{R}^{512 \times 512} \\ V &= \tilde{L}W_v, & W_v &\in \mathbb{R}^{512 \times 512}, & V &\in \mathbb{R}^{512 \times 512} \end{aligned}$$

3. Self-Attention sui latenti

Dopo aver calcolato Q, K, V :

$$\begin{aligned} A &= \text{Softmax}\left(\frac{QK^T}{\sqrt{D}}\right), A \in \mathbb{R}^{512 \times 512} \\ Z &= AV, Z \in \mathbb{R}^{512 \times 512} \end{aligned}$$

4. Proiezione + Residual (post-attention)

$$\begin{aligned} Z' &= ZW_o, & W_o &\in \mathbb{R}^{512 \times 512} \\ L' &= \tilde{L} + Z', & L' &\in \mathbb{R}^{512 \times 512} \end{aligned}$$

5. Feed-Forward MLP

Pre-Norm:

$$\tilde{L}' = \text{LayerNorm}(L')$$

Espansione:

$$H = \tilde{L}'W_1 + b_1, W_1 \in \mathbb{R}^{512 \times 2048}, H \in \mathbb{R}^{512 \times 2048}$$

Attivazione non lineare:

$$H' = \sigma(H) \text{ (ReLU/GELU)}$$

Compressione:

$$F = H'W_2 + b_2, W_2 \in \mathbb{R}^{2048 \times 512}, F \in \mathbb{R}^{512 \times 512}$$

Residual finale

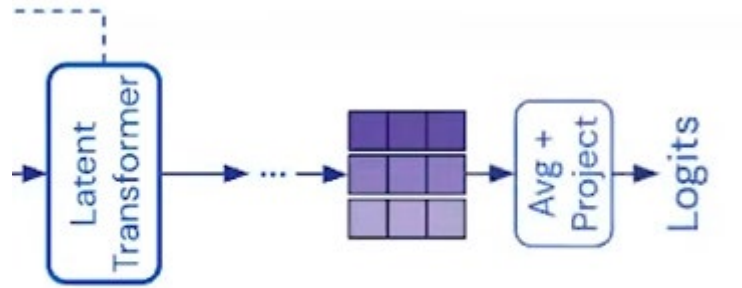
$$L'' = L' + F, L'' \in \mathbb{R}^{512 \times 512}$$

Output del blocco

$$L^{\text{out}} = L'' \in \mathbb{R}^{512 \times 512}$$

Input → **Fourier** → **Embedding** → **Latent Array** → **Cross-Attention** → **Residual + MLP** → **Latent Transformer** → **Residual + MLP** → (ripetizione T volte) → **Pooling** → **Classifier** → **Softmax** → **Loss**

Classification Head



Pooling

Il pooling è un'operazione che serve a riassumere più vettori in uno solo. Nel caso del Perceiver, i vettori sono i latenti finali:

$$L^{(\text{finale})} \in \mathbb{R}^{N \times 512}$$

dove hai N latenti, ciascuno di dimensione 512.

Il pooling medio li combina facendo la media elemento per elemento:

$$h = \frac{1}{N} \sum_{i=1}^N L_i^{(\text{finale})}, h \in \mathbb{R}^{512}$$

Risultato: un unico vettore compatto che rappresenta l'intero input.

Perché facciamo Pooling?

1. Compressione delle informazioni

- Hai N latenti → troppi per passare direttamente al classificatore.
- Il pooling produce un solo vettore gestibile (h).

2. Uniformità

- Indipendentemente da quanti latenti usi, dopo il pooling hai sempre un vettore di dimensione fissa (512).
- Questo permette di collegarlo facilmente al classificatore lineare.

3. Evita parametri aggiuntivi

- La media non introduce nuovi pesi.
- È un metodo semplice ed efficiente per combinare i latenti.

4. Analogia con altri modelli

- Nei Transformer (BERT, GPT), si usa un token speciale (CLS) per riassumere.
- Nel Perceiver non c'è un token speciale: il pooling medio è la soluzione più diretta.

Input → Fourier → Embedding → Latent Array → Cross-Attention → Residual + MLP → Latent Transformer → Residual + MLP → (ripetizione T volte) → Pooling → Classifier → Softmax → Loss

Classificatore Fully Connected Layer (Logits)

Il vettore h entra in un layer lineare:

$$z = hW_c + b_c$$

- $h \in \mathbb{R}^{512} \rightarrow$ rappresentazione compatta dell'input.
- $W_c \in \mathbb{R}^{512 \times 1000} \rightarrow$ matrice di pesi che proietta lo spazio dei latenti (512 dimensioni) nello spazio delle classi (1000 per ImageNet).
- $b_c \in \mathbb{R}^{1000} \rightarrow$ vettore di bias.
- $z \in \mathbb{R}^{1000} \rightarrow$ vettore dei logit, cioè punteggi grezzi per ogni classe.

Concretamente: da un vettore di 512 numeri ne ottieni uno di 1000 numeri, ognuno corrispondente a una classe.

Perché serve questo passaggio?

- Il pooling ti ha dato un riassunto dell'input.
- Ma per decidere a quale classe appartiene, serve uno strato che traduce quel riassunto in punteggi di classificazione.
- Il layer lineare è la funzione più semplice: una combinazione lineare + bias.
- Ogni colonna di W_c può essere vista come un "filtro" che misura quanto h è compatibile con una determinata classe.
- Il bias b_c sposta il punteggio di base per ciascuna classe.
- L'output z contiene i punteggi ancora non normalizzati.

Input $\rightarrow$ Fourier $\rightarrow$ Embedding $\rightarrow$ Latent Array $\rightarrow$ Cross-Attention $\rightarrow$ Residual + MLP $\rightarrow$ Latent Transformer $\rightarrow$ Residual + MLP $\rightarrow$ (ripetizione T volte) $\rightarrow$ Pooling $\rightarrow$ Classifier $\rightarrow$ Softmax $\rightarrow$ Loss

Softmax sui logits

Prendi il vettore dei logits z e applichi la funzione softmax:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{1000} e^{z_j}}$$

Questo trasforma i logits in probabilità normalizzate:

- Tutti i valori $p_i \geq 0$.
- La somma di tutte le probabilità = 1.
- Risultato: un vettore $p \in \mathbb{R}^{1000}$, che rappresenta la probabilità di appartenenza a ciascuna classe.

Scelta della classe predetta

Si prende l'indice della probabilità massima:

$$\hat{y} = \arg \max_i p_i$$

$\hat{y}$ è la classe predetta (ad esempio, "gatto" se la classe 281 corrisponde al gatto in ImageNet).

Input $\rightarrow$ Fourier $\rightarrow$ Embedding $\rightarrow$ Latent Array $\rightarrow$ Cross-Attention $\rightarrow$ Residual + MLP $\rightarrow$ Latent Transformer $\rightarrow$ Residual + MLP $\rightarrow$ (ripetizione T volte) $\rightarrow$ Pooling $\rightarrow$ Classifier $\rightarrow$ Softmax $\rightarrow$ Loss

Loss

La **loss** è il modo in cui “misuri” quanto la predizione del modello è distante dalla verità, e serve a guidare l’aggiornamento dei pesi durante il training.

Esempio su ImageNet

- Il modello produce i logits $z \in \mathbb{R}^{1000}$.
- Dopo la softmax, ottieni le probabilità predette:

$$p_i^{pred} = \frac{e^{z_i}}{\sum_{j=1}^{1000} e^{z_j}}$$

con $i = 1, \dots, 1000$.

- L'etichetta vera y^{true} (la classe corretta) viene rappresentata come one-hot vector:

$$y^{true} = (0, 0, \dots, 1, \dots, 0)$$

dove c'è un solo 1 nella posizione della classe giusta.

- Se sei in fase di training, hai anche l'etichetta vera y .
- Usi la cross-entropy loss:

$$\mathcal{L} = - \sum_{i=1}^K y_i^{true} \log(p_i^{pred})$$

Nel nostro caso:

$$\mathcal{L} = - \sum_{i=1}^{1000} y_i^{true} \log(p_i^{pred})$$

- Dove y_i è 1 per la classe corretta e 0 per tutte le altre (one-hot encoding).
- Questa loss viene poi usata per aggiornare i pesi tramite **backpropagation**.

Noi vogliamo che la probabilità della classe corretta sia la più alta possibile:

$$p_{corretta} \rightarrow 1$$

Quindi serve una funzione di **costo** che:

- sia **bassa** quando $p_{corretta}$ è vicina a 1,
- sia **alta** quando $p_{corretta}$ è vicina a 0.

Vediamo perché c'è il log:

1. Monotonia

- Il logaritmo è crescente: se p cresce, $\log(p)$ cresce.
- Quindi massimizzare p equivale a massimizzare $\log(p)$.
- Usiamo il -log perché stiamo minimizzando la loss (anziché massimizzare la probabilità).

2. Penalità forte per probabilità basse

- Se $p_{corretta} = 0.9$:

$$-\log(0.9) \approx 0.105 \rightarrow \text{loss bassa.}$$

- Se $p_{corretta} = 0.01$:

$$-\log(0.01) \approx 4.6 \rightarrow \text{loss altissima.}$$

- Il log "esplode" quando $p \rightarrow 0$, così punisce molto le predizioni sbagliate e sicure.

Intuizione visiva

La funzione $-\log(p)$:

- Quando $p = 1$: $-\log(1) = 0 \rightarrow$ perfetta predizione.
- Quando $p \rightarrow 0$: $-\log(p) \rightarrow +\infty \rightarrow$ predizione pessima.

È esattamente il comportamento che vogliamo!

Backward pass (flusso dei gradienti)

Alla fine del forward pass abbiamo la loss $\mathcal{L}$, cioè una misura numerica di quanto la predizione del modello (p) è lontana dalla verità (y). Questa loss però, da sola, non basta a migliorare i pesi: per sapere come modificare i parametri della rete dobbiamo calcolare quanto la loss dipende dai valori interni del modello (logits, latenti, pesi, ecc.). Per fare ciò utilizziamo il gradiente.

$$\mathcal{L} = - \sum_{i=1}^K y_i^{\text{true}} \log(p_i^{\text{pred}})$$

1 Perché serve il gradiente?

- L'obiettivo del training è ridurre la loss $\mathcal{L}$ aggiustando i pesi della rete.

- L'algoritmo che usiamo (tipicamente **discesa del gradiente**) aggiorna i parametri nella direzione che diminuisce la loss più velocemente.
- Per sapere in che direzione muovere i pesi, dobbiamo sapere quanto la loss cambia se cambiano i valori interni della rete (logits, pesi, attivazioni, ecc.).

Questa informazione ce la dà il gradiente.

Il backward pass funziona con la **regola della catena**: se hai una funzione composta, per aggiornare i pesi devi risalire gradualmente dall'uscita (loss) fino ai parametri interni.

- Nel nostro caso:

$$h \xrightarrow{W_c, b_c} z \xrightarrow{\text{softmax}} p \xrightarrow{\text{cross-entropy}} \mathcal{L}$$

- Per arrivare a W_c e h , bisogna passare per $z \rightarrow$ Quindi il primo gradiente utile da calcolare è:

$$\frac{\partial \mathcal{L}}{\partial z}$$

$$\text{Con } z = hW_c + b_c$$

2. Perché proprio $\frac{\partial \mathcal{L}}{\partial z}$?

La loss $\mathcal{L}$ dipende dai logits z attraverso softmax + cross-entropy

$$\mathcal{L} = - \sum_{k=1}^K y_k \log p_k$$

dove:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

Se sostituiamo p_k nella loss

$$\mathcal{L} = - \sum_{k=1}^K y_k \log \left(\frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}} \right)$$

Semplificando il logaritmo (Ricorda che $\log \frac{a}{b} = \log a - \log b$)

Quindi:

$$\mathcal{L} = - \sum_{k=1}^K y_k \left(z_k - \log \sum_{j=1}^K e^{z_j} \right)$$

Separando i due termini

$$\mathcal{L} = - \sum_{k=1}^K y_k z_k + \sum_{k=1}^K y_k \log \sum_{j=1}^K e^{z_j}$$

Ma siccome y è one-hot (cioè $\sum_k y_k = 1$):

$$\mathcal{L} = - \sum_{k=1}^K y_k z_k + \log \sum_{j=1}^K e^{z_j}$$

Derivando rispetto a z_i calcoliamo $\frac{\partial \mathcal{L}}{\partial z_i}$.

- Primo termine:

$$\frac{\partial}{\partial z_i} \left(- \sum_{k=1}^K y_k z_k \right) = -y_i$$

- Secondo termine:

$$\frac{\partial}{\partial z_i} \left(\log \sum_{j=1}^K e^{z_j} \right) = \frac{1}{\sum_{j=1}^K e^{z_j}} \cdot e^{z_i} = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} = p_i$$

Risultato finale

$$\frac{\partial \mathcal{L}}{\partial z_i} = p_i - y_i$$

oppure, in forma vettoriale:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}} = \mathbf{p} - \mathbf{y}$$

Per aggiornare $\mathbf{W}_c$, $\mathbf{b}_c$ e $\mathbf{h}$ devo sapere quanto cambia la loss se cambiano loro.
Con la **regola della catena**:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \mathbf{h}} &= \frac{\partial \mathcal{L}}{\partial \mathbf{z}} \cdot \frac{\partial \mathbf{z}}{\partial \mathbf{h}} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{W}_c} &= \frac{\partial \mathcal{L}}{\partial \mathbf{z}} \cdot \frac{\partial \mathbf{z}}{\partial \mathbf{W}_c} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{b}_c} &= \frac{\partial \mathcal{L}}{\partial \mathbf{z}} \cdot \frac{\partial \mathbf{z}}{\partial \mathbf{b}_c} \end{aligned}$$

- **Gradiente rispetto a $\mathbf{h}$**

Formula

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}} \cdot \frac{\partial \mathbf{z}}{\partial \mathbf{h}}$$

Dove:

- $\mathbf{z} = \mathbf{h} \mathbf{W}_c + \mathbf{b}_c$
- La derivata di $\mathbf{z}$ rispetto a $\mathbf{h}$ è:

$$\frac{\partial \mathbf{z}}{\partial \mathbf{h}} = \mathbf{W}_c$$

- Quindi:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}} = (\mathbf{p} - \mathbf{y}) \mathbf{W}_c^\top$$

- Esplicitando indici:

$$\frac{\partial \mathcal{L}}{\partial h_j} = \sum_{k=1}^K (p_k - y_k) W_{c,jk} \quad j = 1, \dots, D$$

Significato: ogni componente di $\mathbf{h}$ riceve un contributo da tutte le classi, pesato dai corrispondenti elementi della matrice $\mathbf{W}_c$.

- **Gradiente rispetto a W_c**

Formula

$$\frac{\partial \mathcal{L}}{\partial W_c} = \frac{\partial \mathcal{L}}{\partial z} \cdot \frac{\partial z}{\partial W_c}$$

Dove:

- $z = hW_c + b_c$.
- Ogni logit è:

$$z_k = \sum_{j=1}^D h_j W_{c,jk} + b_{c,k}$$

- Derivata di z_k rispetto a $W_{c,jk}$:

$$\frac{\partial z_k}{\partial W_{c,jk}} = h_j$$

- Quindi:

$$\frac{\partial \mathcal{L}}{\partial W_{c,jk}} = (p_k - y_k) h_j$$

- Forma compatta matriciale:

$$\frac{\partial \mathcal{L}}{\partial W_c} = h^T (p - y)$$

Significato: ogni peso $W_{c,jk}$ viene aggiornato in base al valore della componente h_j e all'errore della classe k .

- **Gradiente rispetto a b_c**

Formula

$$\frac{\partial \mathcal{L}}{\partial b_c} = \frac{\partial \mathcal{L}}{\partial z} \cdot \frac{\partial z}{\partial b_c}$$

Dove:

- $z = hW_c + b_c$.
- Derivata di z_k rispetto a $b_{c,k}$:

$$\frac{\partial z_k}{\partial b_{c,k}} = 1$$

- Quindi:

$$\frac{\partial \mathcal{L}}{\partial b_{c,k}} = (p_k - y_k)$$

- Forma compatta:

$$\frac{\partial \mathcal{L}}{\partial b_c} = p - y$$

Significato: ogni bias riceve direttamente l'errore associato alla propria classe.

Quindi:

- **Verso h :**

$$\frac{\partial \mathcal{L}}{\partial h} = (p - y)W_c^\top$$

→ distribuisce l'errore nello spazio dei latenti.

- **Verso W_c :**

$$\frac{\partial \mathcal{L}}{\partial W_c} = h^\top (p - y)$$

→ aggiorna la matrice dei pesi del classificatore.

- **Verso b_c :**

$$\frac{\partial \mathcal{L}}{\partial b_c} = p - y$$

→ ogni bias prende l'errore della sua classe.

Ricordiamo cosa succede nel forward

- Hai N latenti finali $L^{(\text{finale})} \in \mathbb{R}^{N \times D}$.
- Applichi average pooling per ottenere un unico vettore compatto $h \in \mathbb{R}^D$:

$$h = \frac{1}{N} \sum_{i=1}^N L_i$$

Quindi ogni decisione di classificazione dipende da tutti i latenti, mediati insieme.

Nel backward pooling

Il backward serve a propagare l'errore (il gradiente della loss) non solo a h , ma a ciascun latente.

Perché?

- Se ci fermassimo a h , sapremmo solo "quanto l'errore dipende dal vettore medio", ma non sapremmo come aggiornare i singoli latenti L_i .
- La rete, invece, ha bisogno di sapere quanto ciascun latente ha contribuito all'errore, per poterne correggere i valori attraverso i pesi che l'hanno generato.

Il backward pooling è quindi il meccanismo che distribuisce l'errore dai logit $\rightarrow$ al vettore medio $h \rightarrow$ ai latenti L_i .
Sappiamo che:

$$\frac{\partial \mathcal{L}}{\partial h} \in \mathbb{R}^D$$

Ora, per ogni latente L_i :

$$h = \frac{1}{N} \sum_{i=1}^N L_i$$

Quando scrivi

$$\frac{\partial h}{\partial L_i}$$

non stai derivando un numero rispetto a un numero, ma un vettore rispetto a un altro vettore. Il risultato non è uno scalare, ma una **matrice jacobiana**.

Dato che

$$h = \frac{1}{N} (L_1 + L_2 + \dots + L_N)$$

prendiamo un singolo L_i .

Il contributo di L_i a h è:

$$h = \dots + \frac{1}{N} L_i + \dots$$

cioè, semplicemente L_i moltiplicato per $\frac{1}{N}$

Se guardo una componente h_j :

$$h_j = \frac{1}{N} L_{i,j} + (\text{altre parti})$$

quindi:

$$\frac{\partial h_j}{\partial L_{i,k}} = \begin{cases} \frac{1}{N} & \text{se } j = k \\ 0 & \text{se } j \neq k \end{cases}$$

Derivata di h rispetto a un singolo latente L_i :

$$\frac{\partial h}{\partial L_i} = \frac{1}{N} I_D$$

Dovendo propagare l'errore da h ad ogni latente L_i , applicando la catena:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial L_i} &= \frac{\partial \mathcal{L}}{\partial h} \cdot \frac{\partial h}{\partial L_i} \\ \frac{\partial \mathcal{L}}{\partial L_i} &= \frac{1}{N} \frac{\partial \mathcal{L}}{\partial h} \end{aligned}$$

Cioè, l'errore della Loss viene ridistribuita ad ogni latente in modo uniforme (ovvero riceve la stessa quota $\frac{1}{N}$ del gradiente)

Facciamo la media perché i latenti, dopo molti strati di self-attention, sono già stati “mescolati” e contengono informazioni simili → trattarli uguali non è un grosso problema.

Quindi: con la media “accusi tutti allo stesso modo”. Se vuoi dare più colpa a chi pesa di più, devi introdurre **pooling pesato** (ad esempio attention pooling).

Backward nel Latent Transformer

Forward

Input (X) → Input Embedding → Cross-Attention → LayerNorm → Q/K/V Projection → Scaled Dot-Product Attention → Output Projection → Residual → MLP → Residual → Latent Transformer Block × T → Latenti finali → Pooling → Classificatore lineare → Softmax → Loss

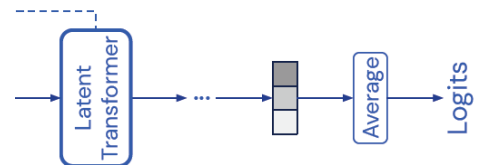
Backward

Loss ← Softmax ← Classificatore lineare ← Pooling ← Latenti i-esimi ← Latent Transformer Block × T ← Residual ← MLP ← Residual ← Output Projection ← Scaled Dot-Product Attention ← Q/K/V Projection ← LayerNorm ← Cross-Attention ← Input Embedding ← Input (X)

Attualmente abbiamo fatto loss → softmax → classificatore lineare → pooling → latenti i-esimi

Ed abbiamo un gradiente su ciascun latente finale:

$$\frac{\partial \mathcal{L}}{\partial L_i^{(\text{finale})}} = \frac{1}{N} \frac{\partial \mathcal{L}}{\partial h} \in \mathbb{R}^{512}, i = 1, \dots, N$$



Ora dobbiamo risalire dentro al Latent Transformer Module (cioè, i blocchi self-attention + MLP).

L'ultimo step del Latent Transformer era il residual quindi faccio il gradiente sul Residual post-MLP

$$L^{(\text{finale})} = \tilde{L} + F$$

dove:

- $\tilde{L}$ = latenti normalizzati,
- F = output dell'MLP.

Quindi una somma elemento per elemento.

Dovendo fare una derivata rispetto a una somma, la regola generale:

$$y = a + b$$

allora:

$$\frac{\partial y}{\partial a} = 1 \qquad \frac{\partial y}{\partial b} = 1$$

Applicazione al caso nostro

$$L^{(\text{finale})} = \tilde{L} + F$$

- Ogni elemento di $L^{(\text{finale})}$ dipende linearmente sia dal corrispondente elemento di $\tilde{L}$ che da quello di F .
- Non c'è nessun coefficiente moltiplicativo diverso da 1.

Quindi:

$$\frac{\partial L^{(\text{finale})}}{\partial \tilde{L}} = I \quad \frac{\partial L^{(\text{finale})}}{\partial F} = I$$

dove I è la matrice identità.

Applicando la regola della catena:

$$\frac{\partial \mathcal{L}}{\partial \tilde{L}} = \frac{\partial \mathcal{L}}{\partial L^{(\text{finale})}} \cdot \frac{\partial L^{(\text{finale})}}{\partial \tilde{L}} = \frac{\partial \mathcal{L}}{\partial L^{(\text{finale})}} \cdot I = \frac{\partial \mathcal{L}}{\partial L^{(\text{finale})}}$$

stessa cosa per F :

$$\frac{\partial \mathcal{L}}{\partial F} = \frac{\partial \mathcal{L}}{\partial L^{(\text{finale})}}$$

Risultato: il gradiente in ingresso si copia uguale nei due rami perché la residual connection è una semplice somma

Backward su MLP

Nel Forward il blocco MLP era:

1. Espansione

$$H = \tilde{L}W_1 + b_1 \quad (W_1 \in \mathbb{R}^{512 \times 2048}, H \in \mathbb{R}^{N \times 2048})$$

2. Attivazione (ReLU/GELU)

$$H' = \sigma(H) \quad (\text{stessa forma } N \times 2048)$$

3. Compressione

$$F = H'W_2 + b_2 \quad (W_2 \in \mathbb{R}^{2048 \times 512}, F \in \mathbb{R}^{N \times 512})$$

4. Residual connection

$$L^{(\text{finale})} = \tilde{L} + F$$

Nel Backward Partiamo dal gradiente che arriva da

$$\frac{\partial \mathcal{L}}{\partial L^{(\text{finale})}}$$

Come hai visto:

$$\frac{\partial \mathcal{L}}{\partial \tilde{L}} = \frac{\partial \mathcal{L}}{\partial L^{(\text{finale})}} \quad \frac{\partial \mathcal{L}}{\partial F} = \frac{\partial \mathcal{L}}{\partial L^{(\text{finale})}}$$

Ora concentriamoci sul ramo F

1. Compressione:

$$F = H'W_2 + b_2$$

- Gradiente rispetto a W_2 :

$$\frac{\partial \mathcal{L}}{\partial W_2} = H'^T \frac{\partial \mathcal{L}}{\partial F}$$

- Gradiente rispetto a b_2 :

$$\frac{\partial \mathcal{L}}{\partial b_2} = \sum_{i=1}^N \frac{\partial \mathcal{L}}{\partial F_i}$$

- Gradiente che torna su H' :

$$\frac{\partial \mathcal{L}}{\partial H'} = \frac{\partial \mathcal{L}}{\partial F} W_2^T$$

2. Attivazione:

$$H' = \sigma(H)$$

Qui entra in gioco la derivata della funzione di attivazione:

- Se ReLU:

$$\frac{\partial H'}{\partial H} = \begin{cases} 1 & \text{se } H > 0 \\ 0 & \text{se } H \leq 0 \end{cases}$$

- Se GELU: derivata più complessa ma stesso concetto (una maschera continua).

Quindi:

$$\frac{\partial \mathcal{L}}{\partial H} = \frac{\partial \mathcal{L}}{\partial H'} \odot \sigma'(H)$$

(dove $\odot$ è prodotto elemento per elemento).

3. Espansione:

$$H = \tilde{L}W_1 + b_1$$

- Gradiente rispetto a W_1 :

$$\frac{\partial \mathcal{L}}{\partial W_1} = \tilde{L}^T \frac{\partial \mathcal{L}}{\partial H}$$

- Gradiente rispetto a b_1 :

$$\frac{\partial \mathcal{L}}{\partial b_1} = \sum_{i=1}^N \frac{\partial \mathcal{L}}{\partial H_i}$$

- Gradiente che torna a $\tilde{L}$ (prima della residual connection):

$$\frac{\partial \mathcal{L}^{(\text{MLP})}}{\partial \tilde{L}} = \frac{\partial \mathcal{L}}{\partial H} W_1^T$$

4. Riassunto "flow"

Il gradiente segue questo percorso:

$$\frac{\partial \mathcal{L}}{\partial L^{(\text{finale})}} \rightarrow \frac{\partial \mathcal{L}}{\partial F} \rightarrow \frac{\partial \mathcal{L}}{\partial H'} \rightarrow \frac{\partial \mathcal{L}}{\partial H} \rightarrow \frac{\partial \mathcal{L}}{\partial \tilde{L}}$$

e in parallelo aggiorna i pesi W_2, b_2, W_1, b_1

Layer Normalization

Forward

Input (X) → Input Embedding → Cross-Attention → LayerNorm → Q/K/V Projection → Scaled Dot-Product Attention → Output Projection → Residual → MLP → Residual → Latent Transformer Block × T → Latenti finali
→ Pooling → Classificatore lineare → Softmax → Loss

Backward

Loss ← Softmax ← Classificatore lineare ← Pooling ← Latenti i-esimi ← Latent Transformer Block × T ← Residual
← MLP ← Residual ← Output Projection ← Scaled Dot-Product Attention ← Q/K/V Projection ← LayerNorm
← Cross-Attention ← Input Embedding ← Input (X)

Nel forward:

$$\tilde{L}_{ij} = \gamma_j \cdot \frac{L'_{ij} - \mu_i}{\sigma_i + \epsilon} + \beta_j$$

- μ_i : media sulla riga i
- σ_i : deviazione standard sulla riga i
- γ, β : parametri appresi (gain e bias della normalizzazione)
- Backward della LayerNorm

Per risalire da $\frac{\partial \mathcal{L}}{\partial \tilde{L}}$ a L' :

1. Gradiente rispetto a γ e β :

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \gamma_j} &= \sum_i \frac{\partial \mathcal{L}}{\partial \tilde{L}_{ij}} \cdot \hat{L}_{ij} \\ \frac{\partial \mathcal{L}}{\partial \beta_j} &= \sum_i \frac{\partial \mathcal{L}}{\partial \tilde{L}_{ij}} \end{aligned}$$

dove $\hat{L}_{ij} = \frac{L'_{ij} - \mu_i}{\sigma_i + \epsilon}$

2. Gradiente rispetto a L' :

Qui si usa la catena di derivate, e la formula finale è:

$$\frac{\partial \mathcal{L}}{\partial L'_{ij}} = \frac{1}{\sigma_i} \left(\frac{\partial \mathcal{L}}{\partial \tilde{L}_{ij}} \cdot \gamma_j - \frac{1}{D} \sum_k \frac{\partial \mathcal{L}}{\partial \tilde{L}_{ik}} \cdot \gamma_k - \hat{L}_{ij} \cdot \frac{1}{D} \sum_k \frac{\partial \mathcal{L}}{\partial \tilde{L}_{ik}} \cdot \gamma_k \cdot \hat{L}_{ik} \right)$$

con $D = 512$ (la dimensione del canale).

Residual Connection (post-attention o post-MLP)

Forward

Input (X) → Input Embedding → Cross-Attention → LayerNorm → Q/K/V Projection → Scaled Dot-Product Attention → Output Projection → Residual → MLP → Residual → Latent Transformer Block × T → Latenti finali
→ Pooling → Classificatore lineare → Softmax → Loss

Backward

Loss ← Softmax ← Classificatore lineare ← Pooling ← Latenti i-esimi ← Latent Transformer Block × T ← Residual
← MLP ← Residual ← Output Projection ← Scaled Dot-Product Attention ← Q/K/V Projection ← LayerNorm
← Cross-Attention ← Input Embedding ← Input (X)

Forward:

$$L' = L + Z'$$

(dove L è l'input originale e Z' è l'output trasformato, ad esempio da self-attention o MLP).

Backward:

Il gradiente si divide in due rami identici perché la somma è lineare:

$$\frac{\partial \mathcal{L}}{\partial L} = \frac{\partial \mathcal{L}}{\partial L'} \frac{\partial L'}{\partial Z'} = \frac{\partial \mathcal{L}}{\partial L'}$$

Interpretazione:

- Il gradiente in uscita da L' viene copiato sia sul percorso "skip" (L) sia sul percorso trasformato (Z').
- In questo modo, la rete può imparare sia a mantenere informazione grezza (via skip connection), sia a modificarla (via ramo trasformato).

Da qui, andando ancora indietro, tocchiamo la proiezione lineare finale dello Z' :

$$Z' = ZW_o$$

Backward - Output Projection

Forward

Input (X) → Input Embedding → Cross-Attention → LayerNorm → Q/K/V Projection → Scaled Dot-Product Attention → Output Projection → Residual → MLP → Residual → Latent Transformer Block × T → Latenti finali → Pooling → Classificatore lineare → Softmax → Loss

Backward

Loss ← Softmax ← Classificatore lineare ← Pooling ← Latenti i-esimi ← Latent Transformer Block × T ← Residual ← MLP ← Residual ← Output Projection ← Scaled Dot-Product Attention ← Q/K/V Projection ← LayerNorm ← Cross-Attention ← Input Embedding ← Input (X)

Forward

Dopo l'attenzione otteniamo:

$$Z' = ZW_o + b_o$$

- $Z \in \mathbb{R}^{N \times D}$
- $W_o \in \mathbb{R}^{D \times D}$
- $b_o \in \mathbb{R}^D$
- $Z' \in \mathbb{R}^{N \times D}$

Backward

Sappiamo che arriva il gradiente della loss rispetto all'uscita:

$$\frac{\partial \mathcal{L}}{\partial Z'} \in \mathbb{R}^{N \times D}$$

Dobbiamo propagare questo gradiente indietro verso Z , W_o e b_o .

1. Gradiente rispetto a W_o

Ogni riga di Z contribuisce linearmente a Z' .

Per la derivata matriciale:

$$\frac{\partial \mathcal{L}}{\partial W_o} = Z^T \frac{\partial \mathcal{L}}{\partial Z'}$$

- $Z^T \in \mathbb{R}^{D \times N}$
- $\frac{\partial \mathcal{L}}{\partial Z'} \in \mathbb{R}^{N \times D}$
- Risultato:

$$\frac{\partial \mathcal{L}}{\partial W_o} \in \mathbb{R}^{D \times D}$$

2. Gradiente rispetto a b_o

Il bias b_o viene aggiunto ad ogni riga di Z' .

La derivata è:

$$\frac{\partial \mathcal{L}}{\partial b_o} = \sum_{i=1}^N \frac{\partial \mathcal{L}}{\partial Z'_i}$$

- Ogni $\frac{\partial \mathcal{L}}{\partial Z'_i} \in \mathbb{R}^D$
- La somma su $i = 1..N$ restituisce:

$$\frac{\partial \mathcal{L}}{\partial b_o} \in \mathbb{R}^D$$

3. Gradiente rispetto a Z

La derivata di $Z' = ZW_o + b_o$ rispetto a Z è semplicemente:

$$\frac{\partial Z'}{\partial Z} = W_o$$

Quindi:

$$\frac{\partial \mathcal{L}}{\partial Z} = \frac{\partial \mathcal{L}}{\partial Z'} W_o^T$$

- $\frac{\partial \mathcal{L}}{\partial Z'} \in \mathbb{R}^{N \times D}$
- $W_o^T \in \mathbb{R}^{D \times D}$
- Risultato:

$$\frac{\partial \mathcal{L}}{\partial Z} \in \mathbb{R}^{N \times D}$$

Riassunto backward output projection

- Verso Z :

$$\frac{\partial \mathcal{L}}{\partial Z} = \frac{\partial \mathcal{L}}{\partial Z'} W_o^T \in \mathbb{R}^{N \times D}$$

- Verso W_o :

$$\frac{\partial \mathcal{L}}{\partial W_o} = Z^T \frac{\partial \mathcal{L}}{\partial Z'} \in \mathbb{R}^{D \times D}$$

- Verso b_o :

$$\frac{\partial \mathcal{L}}{\partial b_o} = \sum_{i=1}^N \frac{\partial \mathcal{L}}{\partial Z'_i} \in \mathbb{R}^D$$

Backward - Scaled Dot-Product Attention

Forward

Input (X) → Input Embedding → Cross-Attention → LayerNorm → Q/K/V Projection → Scaled Dot-Product Attention → Output Projection → Residual → MLP → Residual → Latent Transformer Block × T → Latenti finali → Pooling → Classificatore lineare → Softmax → Loss

Backward

Loss ← Softmax ← Classificatore lineare ← Pooling ← Latenti i-esimi ← Latent Transformer Block × T ← Residual ← MLP ← Residual ← Output Projection ← Scaled Dot-Product Attention ← Q/K/V Projection ← LayerNorm ← Cross-Attention ← Input Embedding ← Input (X)

Forward (richiamo)

L'attenzione calcola:

$$\text{Attention}(Q, K, V) = AV$$

con

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

- $Q \in \mathbb{R}^{N \times d_k}$ (queries)
- $K \in \mathbb{R}^{M \times d_k}$ (keys)
- $V \in \mathbb{R}^{M \times d_v}$ (values)
- $QK^T \in \mathbb{R}^{N \times M}$ (score matrix)
- $A \in \mathbb{R}^{N \times M}$ (matrice delle attenzioni, dopo softmax su ogni riga)
- Output:

$$Z = AV \in \mathbb{R}^{N \times d_v}$$

Backward

Supponiamo di ricevere in ingresso il gradiente:

$$\frac{\partial \mathcal{L}}{\partial Z} \in \mathbb{R}^{N \times d_v}$$

Dobbiamo risalire a Q, K, V .

1. Gradiente verso V

$$Z = AV$$

- Derivata:

$$\frac{\partial \mathcal{L}}{\partial V} = A^T \frac{\partial \mathcal{L}}{\partial Z}$$

- Dimensioni:

- $A^T \in \mathbb{R}^{M \times N}$
- $\frac{\partial \mathcal{L}}{\partial Z} \in \mathbb{R}^{N \times d_v}$

- Risultato:

$$\frac{\partial \mathcal{L}}{\partial V} \in \mathbb{R}^{M \times d_v}$$

2. Gradiente verso A

$$Z = AV \Rightarrow \frac{\partial \mathcal{L}}{\partial A} = \frac{\partial \mathcal{L}}{\partial Z} V^\top$$

- Dimensioni:

- $\frac{\partial \mathcal{L}}{\partial Z} \in \mathbb{R}^{N \times d_v}$

- $V^\top \in \mathbb{R}^{d_v \times M}$

- Risultato:

$$\frac{\partial \mathcal{L}}{\partial A} \in \mathbb{R}^{N \times M}$$

3. Backward attraverso Softmax

Abbiamo:

$$A = \text{softmax}(S), S = \frac{QK^\top}{\sqrt{d_k}}$$

La derivata della softmax è:

$$\frac{\partial A_{ij}}{\partial S_{ik}} = A_{ij}(\delta_{jk} - A_{ik})$$

Quindi, per ogni riga i :

$$\frac{\partial \mathcal{L}}{\partial S_{i,:}} = \left(\frac{\partial \mathcal{L}}{\partial A_{i,:}} \right) J_{\text{softmax}}(S_{i,:})$$

dove J_{softmax} è la jacobiana della softmax.

- Dimensioni:

$$\frac{\partial \mathcal{L}}{\partial S} \in \mathbb{R}^{N \times M}$$

4. Gradiente verso Q e K

Sappiamo:

$$S = \frac{QK^\top}{\sqrt{d_k}}$$

Verso Q

$$\frac{\partial \mathcal{L}}{\partial Q} = \frac{\partial \mathcal{L}}{\partial S} K \cdot \frac{1}{\sqrt{d_k}}$$

- Dimensioni:

- $\frac{\partial \mathcal{L}}{\partial S} \in \mathbb{R}^{N \times M}$

- $K \in \mathbb{R}^{M \times d_k}$

- Risultato:

$$\frac{\partial \mathcal{L}}{\partial Q} \in \mathbb{R}^{N \times d_k}$$

Verso K

$$\frac{\partial \mathcal{L}}{\partial K} = \left(\frac{\partial \mathcal{L}}{\partial S} \right)^\top Q \cdot \frac{1}{\sqrt{d_k}}$$

- Dimensioni:
- $\left(\frac{\partial \mathcal{L}}{\partial S} \right)^\top \in \mathbb{R}^{M \times N}$
- $Q \in \mathbb{R}^{N \times d_k}$
- Risultato:

$$\frac{\partial \mathcal{L}}{\partial K} \in \mathbb{R}^{M \times d_k}$$

Riassunto backward scaled dot-product attention

- Verso V :

$$\frac{\partial \mathcal{L}}{\partial V} = A^\top \frac{\partial \mathcal{L}}{\partial Z} \in \mathbb{R}^{M \times d_v}$$

- Verso A :

$$\frac{\partial \mathcal{L}}{\partial A} = \frac{\partial \mathcal{L}}{\partial Z} V^\top \in \mathbb{R}^{N \times M}$$

- Verso S (pre-softmax):

$$\frac{\partial \mathcal{L}}{\partial S} \in \mathbb{R}^{N \times M}$$

- Verso Q :

$$\frac{\partial \mathcal{L}}{\partial Q} = \frac{\partial \mathcal{L}}{\partial S} K \cdot \frac{1}{\sqrt{d_k}} \in \mathbb{R}^{N \times d_k}$$

- Verso K :

$$\frac{\partial \mathcal{L}}{\partial K} = \left(\frac{\partial \mathcal{L}}{\partial S} \right)^\top Q \cdot \frac{1}{\sqrt{d_k}} \in \mathbb{R}^{M \times d_k}$$

Backward - Proiezioni lineari Q, K, V

Forward

Input (X) → Input Embedding → Cross-Attention → LayerNorm → Q/K/V Projection → Scaled Dot-Product Attention → Output Projection → Residual → MLP → Residual → Latent Transformer Block × T → Latenti finali → Pooling → Classificatore lineare → Softmax → Loss

Backward

Loss ← Softmax ← Classificatore lineare ← Pooling ← Latenti i-esimi ← Latent Transformer Block × T ← Residual ← MLP ← Residual ← Output Projection ← Scaled Dot-Product Attention ← Q/K/V Projection ← LayerNorm ← Cross-Attention ← Input Embedding ← Input (X)

Forward (richiamo)

Nel Perceiver otteniamo:

$$Q = LW_Q + b_Q$$

$$K = XW_K + b_K$$

$$V = XW_V + b_V$$

- $L \in \mathbb{R}^{N \times D}$ = latenti (per Q)
- $X \in \mathbb{R}^{M \times D}$ = input embedding (per K, V)
- $W_Q, W_K, W_V \in \mathbb{R}^{D \times d_k}$ oppure $D \times d_v$
- $b_Q, b_K, b_V \in \mathbb{R}^{d_k}$ o d_v
- $Q \in \mathbb{R}^{N \times d_k}, K \in \mathbb{R}^{M \times d_k}, V \in \mathbb{R}^{M \times d_v}$

Backward

Abbiamo già calcolato dai passi precedenti:

- $\frac{\partial \mathcal{L}}{\partial Q} \in \mathbb{R}^{N \times d_k}$
- $\frac{\partial \mathcal{L}}{\partial K} \in \mathbb{R}^{M \times d_k}$
- $\frac{\partial \mathcal{L}}{\partial V} \in \mathbb{R}^{M \times d_v}$

Ora risaliamo a $W_Q, W_K, W_V, b_Q, b_K, b_V$ e agli input L, X .

1. Proiezione delle Query

$$Q = LW_Q + b_Q$$

- Verso W_Q :

$$\frac{\partial \mathcal{L}}{\partial W_Q} = L^T \frac{\partial \mathcal{L}}{\partial Q}$$

- Dimensioni:

- $L^T \in \mathbb{R}^{D \times N}$

- $\frac{\partial \mathcal{L}}{\partial Q} \in \mathbb{R}^{N \times d_k}$

- Risultato:

$$\frac{\partial \mathcal{L}}{\partial W_Q} \in \mathbb{R}^{D \times d_k}$$

- Verso b_Q :

$$\frac{\partial \mathcal{L}}{\partial b_Q} = \sum_{i=1}^N \frac{\partial \mathcal{L}}{\partial Q_i} \in \mathbb{R}^{d_k}$$

- Verso L :

$$\frac{\partial \mathcal{L}}{\partial L} = \frac{\partial \mathcal{L}}{\partial Q} W_Q^\top$$

- Dimensioni:

- $\frac{\partial \mathcal{L}}{\partial Q} \in \mathbb{R}^{N \times d_k}$
- $W_Q^\top \in \mathbb{R}^{d_k \times D}$

- Risultato:

$$\frac{\partial \mathcal{L}}{\partial L} \in \mathbb{R}^{N \times D}$$

2. Proiezione delle Key

$$K = XW_K + b_K$$

- Verso W_K :

$$\frac{\partial \mathcal{L}}{\partial W_K} = X^\top \frac{\partial \mathcal{L}}{\partial K}$$

- Dimensioni:

- $X^\top \in \mathbb{R}^{D \times M}$
- $\frac{\partial \mathcal{L}}{\partial K} \in \mathbb{R}^{M \times d_k}$

- Risultato:

$$\frac{\partial \mathcal{L}}{\partial W_K} \in \mathbb{R}^{D \times d_k}$$

- Verso b_K :

$$\frac{\partial \mathcal{L}}{\partial b_K} = \sum_{i=1}^M \frac{\partial \mathcal{L}}{\partial K_i} \in \mathbb{R}^{d_k}$$

- Verso X (dal ramo K):

$$\left. \frac{\partial \mathcal{L}}{\partial X} \right|_K = \frac{\partial \mathcal{L}}{\partial K} W_K^\top$$

- Dimensioni:

- $\frac{\partial \mathcal{L}}{\partial K} \in \mathbb{R}^{M \times d_k}$
- $W_K^\top \in \mathbb{R}^{d_k \times D}$

- Risultato:

$$\left. \frac{\partial \mathcal{L}}{\partial X} \right|_K \in \mathbb{R}^{M \times D}$$

3. Proiezione delle Value

$$V = XW_V + b_V$$

- Verso W_V :

$$\frac{\partial \mathcal{L}}{\partial W_V} = X^T \frac{\partial \mathcal{L}}{\partial V}$$

- Dimensioni:

- $X^T \in \mathbb{R}^{D \times M}$

- $\frac{\partial \mathcal{L}}{\partial V} \in \mathbb{R}^{M \times d_v}$

- Risultato:

$$\frac{\partial \mathcal{L}}{\partial W_V} \in \mathbb{R}^{D \times d_v}$$

- Verso b_V :

$$\frac{\partial \mathcal{L}}{\partial b_V} = \sum_{i=1}^M \frac{\partial \mathcal{L}}{\partial V_i} \in \mathbb{R}^{d_v}$$

- Verso X (dal ramo V):

$$\left. \frac{\partial \mathcal{L}}{\partial X} \right|_V = \frac{\partial \mathcal{L}}{\partial V} W_V^T$$

- Dimensioni:

- $\frac{\partial \mathcal{L}}{\partial V} \in \mathbb{R}^{M \times d_v}$

- $W_V^T \in \mathbb{R}^{d_v \times D}$

- Risultato:

$$\left. \frac{\partial \mathcal{L}}{\partial X} \right|_V \in \mathbb{R}^{M \times D}$$

D Riassunto backward proiezioni lineari

- Verso W_Q :

$$\frac{\partial \mathcal{L}}{\partial W_Q} = L^T \frac{\partial \mathcal{L}}{\partial Q} \in \mathbb{R}^{D \times d_k}$$

- Verso b_Q :

$$\frac{\partial \mathcal{L}}{\partial b_Q} = \sum_{i=1}^N \frac{\partial \mathcal{L}}{\partial Q_i} \in \mathbb{R}^{d_k}$$

- Verso L :

$$\frac{\partial \mathcal{L}}{\partial L} = \frac{\partial \mathcal{L}}{\partial Q} W_Q^T \in \mathbb{R}^{N \times D}$$

- Verso W_K :

$$\frac{\partial \mathcal{L}}{\partial W_K} = X^T \frac{\partial \mathcal{L}}{\partial K} \in \mathbb{R}^{D \times d_k}$$

- Verso b_K :

$$\frac{\partial \mathcal{L}}{\partial b_K} = \sum_{i=1}^M \frac{\partial \mathcal{L}}{\partial K_i} \in \mathbb{R}^{d_K}$$

- Verso X dal ramo K :

$$\frac{\partial \mathcal{L}}{\partial X} \Big|_K = \frac{\partial \mathcal{L}}{\partial K} W_K^T \in \mathbb{R}^{M \times D}$$

- Verso W_V :

$$\frac{\partial \mathcal{L}}{\partial W_V} = X^T \frac{\partial \mathcal{L}}{\partial V} \in \mathbb{R}^{D \times d_V}$$

- Verso b_V :

$$\frac{\partial \mathcal{L}}{\partial b_V} = \sum_{i=1}^M \frac{\partial \mathcal{L}}{\partial V_i} \in \mathbb{R}^{d_V}$$

- Verso X dal ramo V :

$$\frac{\partial \mathcal{L}}{\partial X} \Big|_V = \frac{\partial \mathcal{L}}{\partial V} W_V^T \in \mathbb{R}^{M \times D}$$

- Nota importante:

Il gradiente totale su X è la somma dei contributi che arrivano dal ramo K e dal ramo V :

$$\frac{\partial \mathcal{L}}{\partial X} = \frac{\partial \mathcal{L}}{\partial X} \Big|_K + \frac{\partial \mathcal{L}}{\partial X} \Big|_V$$

BACK Backward - Layer Normalization (su Q/K/V)

Forward (richiamo)

Dato un input $H \in \mathbb{R}^{N \times D}$ (dove ogni riga ha dimensione D), la LayerNorm calcola:

$$\tilde{H}_{ij} = \gamma_j \cdot \hat{H}_{ij} + \beta_j$$

dove:

- $\hat{H}_{ij} = \frac{H_{ij} - \mu_i}{\sigma_i + \epsilon}$
- $\mu_i = \frac{1}{D} \sum_{j=1}^D H_{ij}$ (media della riga i)
- $\sigma_i = \sqrt{\frac{1}{D} \sum_{j=1}^D (H_{ij} - \mu_i)^2}$ (deviazione standard della riga i)
- $\gamma, \beta \in \mathbb{R}^D$ (parametri appresi: gain e bias)
- Output: $\tilde{H} \in \mathbb{R}^{N \times D}$

Backward

Arriva in ingresso:

$$\frac{\partial \mathcal{L}}{\partial \tilde{H}} \in \mathbb{R}^{N \times D}$$

Dobbiamo risalire a γ, β, H .

1. Gradiente rispetto a γ e β

- Verso γ :

$$\frac{\partial \mathcal{L}}{\partial \gamma_j} = \sum_{i=1}^N \frac{\partial \mathcal{L}}{\partial \tilde{H}_{ij}} \cdot \hat{H}_{ij} \in \mathbb{R}^D$$

- Verso β :

$$\frac{\partial \mathcal{L}}{\partial \beta_j} = \sum_{i=1}^N \frac{\partial \mathcal{L}}{\partial \tilde{H}_{ij}} \in \mathbb{R}^D$$

2. Gradiente rispetto all'input normalizzato $\hat{H}$

Poiché:

$$\tilde{H}_{ij} = \gamma_j \hat{H}_{ij} + \beta_j$$

si ha:

$$\frac{\partial \mathcal{L}}{\partial \hat{H}_{ij}} = \frac{\partial \mathcal{L}}{\partial \tilde{H}_{ij}} \cdot \gamma_j$$

Forma compatta:

$$\frac{\partial \mathcal{L}}{\partial \hat{H}} = \frac{\partial \mathcal{L}}{\partial \tilde{H}} \odot \gamma \in \mathbb{R}^{N \times D}$$

3. Gradiente rispetto a H

Ora dobbiamo tornare da $\hat{H}$ a H .

Ricordiamo:

$$\hat{H}_{ij} = \frac{H_{ij} - \mu_i}{\sigma_i + \epsilon}$$

Il gradiente finale per ogni riga i (dimensione D) è:

$$\frac{\partial \mathcal{L}}{\partial H_{ij}} = \frac{1}{\sigma_i} \left(\frac{\partial \mathcal{L}}{\partial \hat{H}_{ij}} - \frac{1}{D} \sum_{k=1}^D \frac{\partial \mathcal{L}}{\partial \hat{H}_{ik}} - \hat{H}_{ij} \cdot \frac{1}{D} \sum_{k=1}^D \frac{\partial \mathcal{L}}{\partial \hat{H}_{ik}} \hat{H}_{ik} \right)$$

- Dimensioni:
- $\frac{\partial \mathcal{L}}{\partial \hat{H}} \in \mathbb{R}^{N \times D}$
- Risultato:

$$\frac{\partial \mathcal{L}}{\partial H} \in \mathbb{R}^{N \times D}$$

Riassunto backward LayerNorm

- Verso γ :

$$\frac{\partial \mathcal{L}}{\partial \gamma_j} = \sum_{i=1}^N \frac{\partial \mathcal{L}}{\partial \hat{H}_{ij}} \cdot \hat{H}_{ij} \in \mathbb{R}^D$$

- Verso β :

$$\frac{\partial \mathcal{L}}{\partial \beta_j} = \sum_{i=1}^N \frac{\partial \mathcal{L}}{\partial \hat{H}_{ij}} \in \mathbb{R}^D$$

- Verso H :

$$\frac{\partial \mathcal{L}}{\partial H_{ij}} = \frac{1}{\sigma_i} \left(\frac{\partial \mathcal{L}}{\partial \hat{H}_{ij}} - \frac{1}{D} \sum_{k=1}^D \frac{\partial \mathcal{L}}{\partial \hat{H}_{ik}} - \hat{H}_{ij} \cdot \frac{1}{D} \sum_{k=1}^D \frac{\partial \mathcal{L}}{\partial \hat{H}_{ik}} \hat{H}_{ik} \right) \in \mathbb{R}^{N \times D}$$

Backward – Cross Attention

Forward (richiamo)

$$Z = \text{Attention}(Q, K, V)W_o + b_o$$

con:

- $Q = LW_Q + b_Q \in \mathbb{R}^{N \times d_k}$
- $K = XW_K + b_K \in \mathbb{R}^{M \times d_k}$

- $V = XW_V + b_V \in \mathbb{R}^{M \times d_v}$
- $Z \in \mathbb{R}^{N \times d_v}$

Backward - Flusso dei gradienti

1. Output projection

Già fatto:

$$\frac{\partial \mathcal{L}}{\partial Z}, \frac{\partial \mathcal{L}}{\partial W_o}, \frac{\partial \mathcal{L}}{\partial b_o}$$

2. Scaled Dot-Product Attention

$$\frac{\partial \mathcal{L}}{\partial V} = A^\top \frac{\partial \mathcal{L}}{\partial Z}, \frac{\partial \mathcal{L}}{\partial Q} = \frac{\partial \mathcal{L}}{\partial S} K / \sqrt{d_k}, \frac{\partial \mathcal{L}}{\partial K} = \left(\frac{\partial \mathcal{L}}{\partial S} \right)^\top Q / \sqrt{d_k}$$

3. Proiezioni lineari

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial W_Q} &= L^\top \frac{\partial \mathcal{L}}{\partial Q}, & \frac{\partial \mathcal{L}}{\partial b_Q} &= \sum_i \frac{\partial \mathcal{L}}{\partial Q_i}, & \frac{\partial \mathcal{L}}{\partial L} &= \frac{\partial \mathcal{L}}{\partial Q} W_Q^\top \\ \frac{\partial \mathcal{L}}{\partial W_K} &= X^\top \frac{\partial \mathcal{L}}{\partial K}, & \frac{\partial \mathcal{L}}{\partial b_K} &= \sum_i \frac{\partial \mathcal{L}}{\partial K_i}, & \frac{\partial \mathcal{L}}{\partial X} \Big|_K &= \frac{\partial \mathcal{L}}{\partial K} W_K^\top \\ \frac{\partial \mathcal{L}}{\partial W_V} &= X^\top \frac{\partial \mathcal{L}}{\partial V}, & \frac{\partial \mathcal{L}}{\partial b_V} &= \sum_i \frac{\partial \mathcal{L}}{\partial V_i}, & \frac{\partial \mathcal{L}}{\partial X} \Big|_V &= \frac{\partial \mathcal{L}}{\partial V} W_V^\top \end{aligned}$$

Riassunto Cross-Attention Backward

- Il gradiente si propaga da $Z \rightarrow$ attenzione $\rightarrow$ proiezioni lineari.
- Verso latenti L :

$$\frac{\partial \mathcal{L}}{\partial L} = \frac{\partial \mathcal{L}}{\partial Q} W_Q^\top$$

- Verso input X :

$$\frac{\partial \mathcal{L}}{\partial X} = \frac{\partial \mathcal{L}}{\partial X} \Big|_K + \frac{\partial \mathcal{L}}{\partial X} \Big|_V$$

- Verso pesi W_Q, W_K, W_V e bias relativi: aggiornati con outer product tra input e gradienti.

Back Backward - Input Embedding

Forward (richiamo)

All'inizio il Perceiver prende l'input X_{raw} (es. immagine, audio, testo) e lo passa in un embedding lineare o una piccola convoluzione per portarlo nello spazio latente D :

$$X = X_{\text{raw}} W_E + b_E$$

- $X_{\text{raw}} \in \mathbb{R}^{M \times C}$ (M = numero token/pixel/patch, C = canali originali)
- $W_E \in \mathbb{R}^{C \times D}$ (matrice embedding)
- $b_E \in \mathbb{R}^D$ (bias embedding)
- $X \in \mathbb{R}^{M \times D}$ (embedding usato per K, V)

Backward

Dopo aver fatto il backward della cross-attention, abbiamo già ottenuto:

$$\frac{\partial \mathcal{L}}{\partial X} \in \mathbb{R}^{M \times D}$$

Ora risaliamo.

- Verso W_E :

$$\frac{\partial \mathcal{L}}{\partial W_E} = X_{\text{raw}}^T \frac{\partial \mathcal{L}}{\partial X} \in \mathbb{R}^{C \times D}$$

- Verso b_E :

$$\frac{\partial \mathcal{L}}{\partial b_E} = \sum_{i=1}^M \frac{\partial \mathcal{L}}{\partial X_i} \in \mathbb{R}^D$$

- Verso input grezzo X_{raw} :

$$\frac{\partial \mathcal{L}}{\partial X_{\text{raw}}} = \frac{\partial \mathcal{L}}{\partial X} W_E^T \in \mathbb{R}^{M \times C}$$

D Riassunto Input Embedding Backward

- Verso W_E : aggiorna la matrice embedding

$$\frac{\partial \mathcal{L}}{\partial W_E} = X_{\text{raw}}^T \frac{\partial \mathcal{L}}{\partial X}$$

- Verso b_E : accumula gli errori

$$\frac{\partial \mathcal{L}}{\partial b_E} = \sum_i \frac{\partial \mathcal{L}}{\partial X_i}$$

- Verso input grezzo: distribuisce il gradiente all'array iniziale

$$\frac{\partial \mathcal{L}}{\partial X_{\text{raw}}} = \frac{\partial \mathcal{L}}{\partial X} W_E^T$$

Perceiver - Forward & Backward Pass (schema riassuntivo)

Forward Pass

1. Input grezzo

$$X_{\text{raw}} \in \mathbb{R}^{M \times C}$$

2. Input Embedding

$$X = X_{\text{raw}} W_E + b_E \in \mathbb{R}^{M \times D}$$

3. Cross-Attention

- Queries dai latenti: $Q = L W_Q + b_Q \in \mathbb{R}^{N \times D}$
- Keys e Values dall'input:

$$K = XW_K + b_K \in \mathbb{R}^{M \times D}$$

$$V = XW_V + b_V \in \mathbb{R}^{M \times D}$$

- Attenzione:

$$Z = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \in \mathbb{R}^{N \times D}$$

- Proiezione output:

$$Z' = ZW_o + b_o \in \mathbb{R}^{N \times D}$$

4. Residual + LayerNorm + MLP (ripetuto T volte nei blocchi latenti)

$$L_{\text{final}} \in \mathbb{R}^{N \times D}$$

5. Pooling

$$h = \frac{1}{N} \sum_{i=1}^N L_{\text{final},i} \in \mathbb{R}^D$$

6. Classificatore lineare + softmax

$$z = hW_c + b_c \in \mathbb{R}^K, p = \text{softmax}(z) \in \mathbb{R}^K$$

7. Loss (cross-entropy)

$$\mathcal{L} = - \sum_{k=1}^K y_k \log p_k$$

Backward Pass

1. Loss → Softmax → Logits

$$\frac{\partial \mathcal{L}}{\partial z} = p - y \in \mathbb{R}^K$$

2. Classificatore lineare

$$\frac{\partial \mathcal{L}}{\partial h} = (p - y)W_c^T \in \mathbb{R}^D$$

$$\frac{\partial \mathcal{L}}{\partial W_c} = h^T(p - y) \in \mathbb{R}^{D \times K}, \frac{\partial \mathcal{L}}{\partial b_c} = p - y \in \mathbb{R}^K$$

3. Pooling

$$\frac{\partial \mathcal{L}}{\partial L_{\text{final},i}} = \frac{1}{N} \frac{\partial \mathcal{L}}{\partial h} \in \mathbb{R}^D$$

4. Latent Transformer Block (Residual + MLP + Self-Attention)

- Residual: gradiente copiato nei due rami
- MLP: aggiorna W_1, W_2, b_1, b_2 e propaga a L
- LayerNorm: aggiorna γ, β e propaga a input normalizzato
- Self-Attention: stesso backward della cross-attention ma con Q, K, V tutti dai latenti

5. Cross-Attention

$$\frac{\partial \mathcal{L}}{\partial Q}, \frac{\partial \mathcal{L}}{\partial K}, \frac{\partial \mathcal{L}}{\partial V}$$

- Verso output projection W_o, b_o

- Verso softmax dei punteggi $S = QK^T / \sqrt{d_k}$
 - Verso Q, K, V lineari
6. Proiezioni lineari
- Queries (latenti):

$$\frac{\partial \mathcal{L}}{\partial W_Q} = L^T \frac{\partial \mathcal{L}}{\partial Q}, \frac{\partial \mathcal{L}}{\partial L} = \frac{\partial \mathcal{L}}{\partial Q} W_Q^T$$

- Keys e Values (input):

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial W_K} &= X^T \frac{\partial \mathcal{L}}{\partial K}, \frac{\partial \mathcal{L}}{\partial X} \Big|_K = \frac{\partial \mathcal{L}}{\partial K} W_K^T \\ \frac{\partial \mathcal{L}}{\partial W_V} &= X^T \frac{\partial \mathcal{L}}{\partial V}, \frac{\partial \mathcal{L}}{\partial X} \Big|_V = \frac{\partial \mathcal{L}}{\partial V} W_V^T \\ \frac{\partial \mathcal{L}}{\partial X} &= \frac{\partial \mathcal{L}}{\partial X} \Big|_K + \frac{\partial \mathcal{L}}{\partial X} \Big|_V \end{aligned}$$

7. Input Embedding

$$\frac{\partial \mathcal{L}}{\partial W_E} = X_{\text{raw}}^T \frac{\partial \mathcal{L}}{\partial X}, \frac{\partial \mathcal{L}}{\partial b_E} = \sum_i \frac{\partial \mathcal{L}}{\partial X_i}, \frac{\partial \mathcal{L}}{\partial X_{\text{raw}}} = \frac{\partial \mathcal{L}}{\partial X} W_E^T$$

In sintesi

- Forward: l'input grezzo $\rightarrow$ embedding $\rightarrow$ cross-attention (Q da latenti, K/V da input) $\rightarrow$ latent transformer $\rightarrow$ pooling $\rightarrow$ classificatore $\rightarrow$ softmax $\rightarrow$ loss.
- Backward: la loss risale in senso inverso, aggiornando classifier, pooling, transformer, attenzioni, proiezioni lineari, embedding e infine gli input originali.

Schema Riassuntivo Forward/Backward

Step	Forward	Backward
1. Input grezzo	$X_{raw} \in \mathbb{R}^{M \times C}$	$\delta X_{raw} = \delta X W_E^\top \in \mathbb{R}^{M \times C}$
2. Input Embedding	$X = X_{raw} W_E + b_E$ $W_E \in \mathbb{R}^{C \times D}, b_E \in \mathbb{R}^D$ $\Rightarrow X \in \mathbb{R}^{M \times D}$	$\partial W_E = X_{raw}^\top \delta X \in \mathbb{R}^{C \times D}$ $\partial b_E = \sum_i \delta X_i \in \mathbb{R}^D$
3. Cross-Attention	$Q = L W_Q + b_Q \in \mathbb{R}^{N \times D}$ $K = X W_K + b_K \in \mathbb{R}^{M \times D}$ $V = X W_V + b_V \in \mathbb{R}^{M \times D}$ $S = \frac{Q K^\top}{\sqrt{D}} \in \mathbb{R}^{N \times M}$ $A = \text{softmax}(S) \in \mathbb{R}^{N \times M}$ $Z = A V \in \mathbb{R}^{N \times D}$ $Z' = Z W_O + b_O \in \mathbb{R}^{N \times D}$ - Residual: $L^{(1)} = L + Z'$	- Residual: $\delta L = \delta L^{(1)} + \delta Q W_Q^\top$ - Output proj.: $\partial W_O = Z^\top \delta Z', \delta Z = \delta Z' W_O^\top$ $\delta V = A^\top \delta Z$ $\delta A = \delta Z V^\top$ - Softmax: $\delta S = \delta A \odot (A - A \odot A)$ $\delta Q = \delta S K / \sqrt{D}$ $\delta K = \delta S^\top Q / \sqrt{D}$ - Proiezioni lineari: $\partial W_Q, \partial W_K, \partial W_V$ aggiornati
4. Latent Transformer Block ($\times T$)	LayerNorm 1: $\tilde{L} = LN(L)$ Self-Attention: come Cross-Attn ma solo sui latenti $\Rightarrow Z'' \in \mathbb{R}^{N \times D}$ Residual: $L' = L + Z''$ LayerNorm 2: $\tilde{L}' = LN(L')$ MLP: $H = \tilde{L}' W_1 + b_1 \in \mathbb{R}^{N \times D_{ff}}$ $H' = \sigma(H)$ $F = H' W_2 + b_2 \in \mathbb{R}^{N \times D}$ Residual: $L_{final} = L' + F$	Residuals: ogni split $\rightarrow$ gradiente va a entrambi i rami. LayerNorm: $\delta L = \frac{1}{\sqrt{\sigma^2 + \epsilon}} \left(\delta \tilde{L} - \frac{1}{N} \sum \delta \tilde{L} - \frac{L - \mu}{\sigma^2} \sum \delta \tilde{L} (L - \mu) \right)$ $\partial \gamma = \sum \delta \tilde{L} \odot \hat{L}, \quad \partial \beta = \sum \delta \tilde{L}$ Self-Attn: backward come Step 3, con input latenti. MLP: $\delta F = \delta L_{final}$ $\partial W_2 = H'^\top \delta F, \partial b_2 = \sum \delta F$ $\delta H' = \delta F W_2^\top$ $\delta H = \delta H' \odot \sigma'(H)$ $\partial W_1 = \tilde{L}'^\top \delta H, \partial b_1 = \sum \delta H$ $\delta \tilde{L}' = \delta H W_1^\top$
5. Pooling	$h = \frac{1}{N} \sum_i L_{final,i} \in \mathbb{R}^D$	$\delta L_{final,i} = \frac{1}{N} \delta h$
6. Classificatore + Softmax	$z = h W_c + b_c, W_c \in \mathbb{R}^{D \times K}, b_c \in \mathbb{R}^K$ $p = \text{softmax}(z) \in \mathbb{R}^K$	$\partial W_c = h^\top (p - y)$ $\partial b_c = p - y$ $\delta h = (p - y) W_c^\top$
7. Loss	$\mathcal{L} = - \sum_{k=1}^K y_k \log p_k$	$\delta z = p - y$

Il processo di addestramento di un Perceiver

L'addestramento di una rete neurale come il **Perceiver** si basa sull'iterazione di tre fasi fondamentali: **forward pass**, **backward pass** e **aggiornamento dei pesi**.

Tali operazioni vengono eseguite su sottoinsiemi del dataset (batch) e ripetute per più cicli completi (epoche).

- **Dataset**

È l'insieme dei dati su cui addestriamo il modello.

- Nel caso di ImageNet → circa 1.200.000 immagini etichettate in 1000 classi.
- Formalmente:

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\} \quad |D| = N$$

Con x_i : immagine i -esima, y_i : classe associata i -esima

- **Batch**

Invece di prendere tutte le N immagini insieme, le dividiamo in piccoli gruppi (batch).

- Esempio: batch size $m = 256$.
- Ogni batch B contiene m coppie (x_i, y_i) .
- Il modello fa forward + backward + aggiornamento pesi su ciascun batch.

- **Epoca**

Una epoca è un giro completo sul dataset.

- Numero di batch per epoca =

$$\frac{N}{m}$$

Se $N = 1.200.000, m = 256 \rightarrow$ circa 4687 batch per epoca.

- Dopo un'epoca il modello ha visto tutti i dati una volta.

Fine del Forward Pass

Per ogni batch (es. 256 immagini):

1. Ogni immagine passa attraverso: embedding → cross-attention → latent transformer → pooling → classificatore
2. Il modello produce un vettore di probabilità (softmax) per ognuna delle 256 immagini.

$$p_i \in \mathbb{R}^C, \quad C = 1000 \text{ classi di ImageNet}$$

3. La loss (cross-entropy) viene calcolata come media sul batch:

$$\mathcal{L}(B) = -\frac{1}{m} \sum_{i=1}^m \sum_{c=1}^C y_{i,c}^{true} \log(p_{i,c})$$

La doppia sommatoria è presente perché stiamo calcolando la Loss sulle m immagini presenti nel batch.

Output del forward: Predizioni e Loss numerica $\mathcal{L}$

Fine del Backward Pass

Dal valore della loss:

- Si calcolano i gradienti di tutti i parametri W del modello.
- Per ogni peso otteniamo:

$$g_W = \nabla_W \mathcal{L}(B)$$

Output del backward:

- Gradienti per ogni layer (embedding, cross-attention, latent transformer, classificatore).

Ma i pesi non sono ancora stati modificati.

Ottimizzazione

Il backward calcola i gradienti ($\nabla_W \mathcal{L}$), cioè le derivate della loss rispetto ai pesi, ma i gradienti da soli non aggiornano i pesi: servono solo come informazione. L'ottimizzatore **prende i gradienti e decide come modificare i pesi per ridurre la loss**.

Formula generale SGD (Stochastic Gradient Descent)

$$W \leftarrow W - \eta \cdot \nabla_W \mathcal{L}$$

dove:

- W = peso del modello,
- η = learning rate,
- $\nabla_W \mathcal{L}$ = gradiente della loss rispetto a W .

Caratteristiche:

- Aggiorna i pesi direttamente seguendo il gradiente.
- È semplice ed efficiente, ma:
 - sensibile alla scelta del learning rate,
 - può oscillare o convergere lentamente,
 - non adatta il passo per ogni parametro.

Usato spesso con varianti: SGD con momentum, che aggiunge una memoria dei gradienti passati.

Adam (Adaptive Moment Estimation)

Adam introduce la stima di momenti dei gradienti → media (primo momento) e varianza (secondo momento).

Formula:

Primo momento (media mobile dei gradienti)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_W \mathcal{L}$$

- m_t : stima della media dei gradienti al tempo t .
- $\nabla_W \mathcal{L}$: gradiente della loss rispetto ai pesi W .
- β_1 : fattore di decadimento (tipicamente 0.9) → controlla quanto del gradiente passato viene mantenuto.

Secondo momento (varianza dei gradienti)

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)(\nabla_W \mathcal{L})^2$$

- v_t : stima della varianza (quadrato dei gradienti).
- β_2 : fattore di decadimento (tipicamente 0.999) → più vicino a 1 = più memoria del passato.

Correzione del bias (aggiorniamo questi valori perché inizialmente i valori usati tendono a sottostimare i valori reali):

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Aggiornamento finale:

$$W_{t+1} = W_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Caratteristiche:

- Ogni parametro ha un learning rate adattivo.
- Converge più velocemente e stabilmente di SGD.
- Richiede più memoria perché memorizza m_t e v_t .
- Parametri tipici: $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$.

È l'ottimizzatore più usato nei modelli deep learning.

Esempio numerico su Adam

Immaginiamo di voler aggiornare un solo peso W .

Parametri iniziali:

- Peso: $W_0 = 0.5$ Learning rate: $\eta = 0.1$ $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$

Gradienti calcolati dal backward:

- al passo 1: $\nabla_W \mathcal{L} = 0.3$
- al passo 2 : $\nabla_W \mathcal{L} = 0.2$

Passo 1

1. Primo momento:

$$m_1 = \beta_1 m_0 + (1 - \beta_1) \nabla_W \mathcal{L} = 0.9 \cdot 0 + 0.1 \cdot 0.3 = 0.03$$

Si parte con $m_0 = 0$ e $v_0 = 0$ perché non ci sono gradienti iniziali.

2. Secondo momento:

$$v_1 = \beta_2 v_0 + (1 - \beta_2)(\nabla_W \mathcal{L})^2 = 0.999 \cdot 0 + 0.001 \cdot (0.3^2) = 0.00009$$

3. Correzioni del bias:

$$\hat{m}_1 = \frac{m_1}{1 - \beta_1^1} = \frac{0.03}{0.1} = 0.3$$

$$\hat{v}_1 = \frac{v_1}{1 - \beta_2^1} = \frac{0.00009}{0.001} = 0.09$$

4. Aggiornamento:

$$W_1 = W_0 - \eta \cdot \frac{\hat{m}_1}{\sqrt{\hat{v}_1} + \epsilon} = 0.5 - 0.1 \cdot \frac{0.3}{\sqrt{0.09}} = 0.5 - 0.1 \cdot \frac{0.3}{0.3} = 0.5 - 0.1 = 0.4$$

Passo 2

Gradiente: $\nabla_W \mathcal{L} = 0.2$

1. Primo momento:

$$m_2 = 0.9 \cdot 0.03 + 0.1 \cdot 0.2 = 0.047$$

2. Secondo momento:

$$v_2 = 0.999 \cdot 0.00009 + 0.001 \cdot (0.2^2) = 0.00008991 + 0.00004 = 0.00012991$$

3. Correzioni:

$$\hat{m}_2 = \frac{m_2}{1 - \beta_1^2} = \frac{0.047}{1 - 0.81} = \frac{0.047}{0.19} \approx 0.247$$

$$\hat{v}_2 = \frac{v_2}{1 - \beta_2^2} = \frac{0.00012991}{1 - 0.998001} \approx \frac{0.00012991}{0.001999} \approx 0.065$$

4. Aggiornamento:

$$W_2 = W_1 - 0.1 \cdot \frac{0.247}{\sqrt{0.065}} = 0.4 - 0.1 \cdot \frac{0.247}{0.255} = 0.4 - 0.1 \cdot 0.968 \approx 0.303$$

LAMB (Layer-wise Adaptive Moments)

LAMB è una variante di Adam progettata per il large batch training (es. batch size > 32k).

Passaggi:

1. Stima dei momenti (come Adam):

$$m_t, v_t \rightarrow \Delta W_t = \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

2. Calcolo del trust ratio per layer:

$$r_t = \frac{\|W_t\|}{\|\Delta W_t\|}$$

3. Aggiornamento finale:

$$W_{t+1} = W_t - \eta \cdot r_t \cdot \Delta W_t$$

Caratteristiche:

- Mantiene la direzione adattiva di Adam.
- Normalizza gli step rispetto alla scala dei pesi di ogni layer → bilanciamento.
- Permette di usare batch enormi senza instabilità.
- Usato con successo per allenare BERT e Perceiver su dataset molto grandi.

Parametri tipici: stessi di Adam, ma con learning rate schedulato e warmup iniziale.

Weight Sharing nel Perceiver

Il Perceiver è costruito come una sequenza ricorsiva di:

$$\text{Cross Attention} \rightarrow \text{Latent Transformer Block} \rightarrow \text{Cross Attention} \rightarrow \dots$$

Ora, invece di avere nuovi set di pesi per ogni blocco (come avviene nei Transformer standard), il **Perceiver riutilizza lo stesso set di pesi in tutti i livelli della rete.**

1. Cross Attention

Il meccanismo che mappa l'input (X_{emb}) nei latenti (L) ha un solo set di matrici di proiezione:

$$W_q, W_k, W_v, W_o$$

Queste matrici vengono riutilizzate identiche in ogni chiamata di cross attention.

Quindi il Cross Attention non "impara" cose diverse ad ogni livello, ma fa sempre la stessa trasformazione parametrica, applicata ricorsivamente.

2. Latent Transformer Block

Anche il blocco di self-attention + MLP sui latenti usa sempre lo stesso set di pesi, quindi le matrici della self-attention:

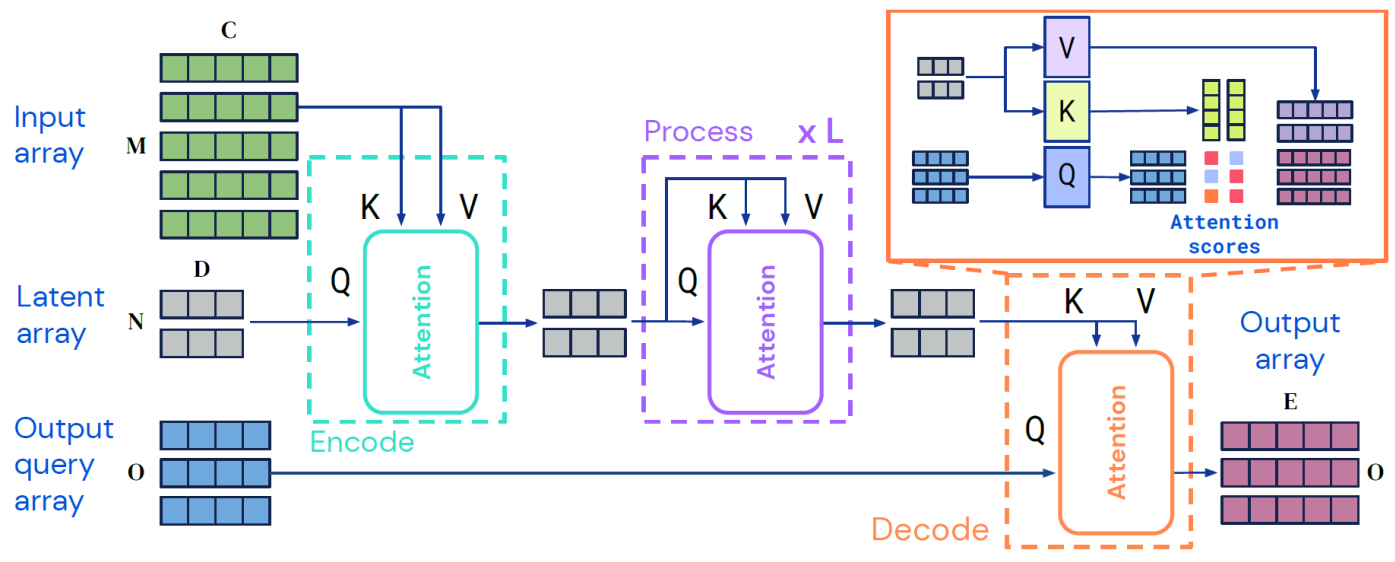
$$W_q, W_k, W_v, W_o$$

e i pesi delle due proiezioni del feed-forward (es. W_1, W_2) sono condivisi in tutti i livelli.

Perché questa scelta?

- Riduce drasticamente il numero di parametri (il modello non esplode di dimensioni).
- Trasforma il Perceiver in una rete ricorsiva/profondamente iterata, più simile a un RNN profondo che a un Transformer classico.
- Garantisce che i latenti vengano raffinati progressivamente dagli stessi operatori, invece di ogni volta da operatori diversi.

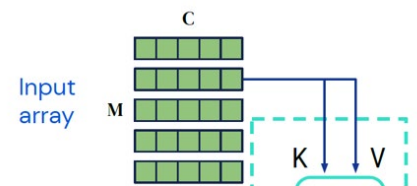
Architettura del Perceiver IO



Input array $X \in \mathbb{R}^{M \times C}$

L'input array è una matrice:

$$X \in \mathbb{R}^{M \times C} \quad X = \{x_1, x_2, \dots, x_M\} \quad x_i \in \mathbb{R}^C$$



Esempi

• Linguaggio - GLUE benchmark (senza tokenizzazione)

Abbiamo un testo, di questo ogni carattere viene convertito in uno o più byte con valore intero tra 0 e 255.

Ogni x_i (cioè ogni byte) viene rappresentato concatenando due parti:

$$x_i = [f_i || p_i]$$

dove:

1. $f_i \in \mathbb{R}^{128} = E[b_i]$ embedding del byte
 - Esistono 256 possibili byte (0-255).
 - Si costruisce una matrice di embedding $E \in \mathbb{R}^{256 \times 128}$.
 - Ogni riga di E è un vettore di 128 numeri che rappresenta un byte.
 - Quando arriva un byte b_i , si prende la riga corrispondente di E , $f_i = E[b_i]$
 - Risultato: un vettore di 128 numeri.
2. $p_i \in \mathbb{R}^{64} =$ positional encoding Fourier
 - Il modello deve sapere dove si trova il byte nella sequenza (inizio, metà, fine...).
 - Per questo si usa un encoding sinusoidale multiscala (Fourier features).
 - Ogni posizione $i \rightarrow$ vettore di 64 numeri.

Infine si fa la concatenazione:

$$x_i = [f_i || p_i] \in \mathbb{R}^{128+64} = \mathbb{R}^{192}$$

Per esempio per la parola Ciao:

1. Conversione in byte UTF-8

Dato il testo, lo convertiamo in byte UTF-8:

$$\mathbf{b} = (b_0, b_1, \dots, b_{M-1}), \quad b_i \in \{0, \dots, 255\}$$

Per "Ciao": $M = 4$ e $(b_0, b_1, b_2, b_3) = (67, 105, 97, 111)$

Carattere	Byte (decimale)
C	67
i	105
a	97
o	111
Quindi $M = 4$.	

Riga 0 = "C"

Riga 1 = "i"

Riga 2 = "a"

Riga 3 = "o"

$$X = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} \in \mathbb{R}^{4 \times 192}$$

2. Embedding del byte ($f_i \in \mathbb{R}^{128}$)

Ogni byte viene usato come **indice del vettore corrispondente della matrice embedding** $E \in \mathbb{R}^{256 \times 128}$.

$$f_i = E[b_i], \quad f_i \in \mathbb{R}^{128}$$

Per "Ciao"

- $b_0 = 67 \rightarrow f_0 = E[67] \in \mathbb{R}^{128}$ *(ovvero il vettore corrispondente è la riga 67 di E)*
- $b_1 = 105 \rightarrow f_1 = E[105] \in \mathbb{R}^{128}$
- $b_2 = 97 \rightarrow f_2 = E[97] \in \mathbb{R}^{128}$
- $b_3 = 111 \rightarrow f_3 = E[111] \in \mathbb{R}^{128}$

3. Positional encoding ($p_i \in \mathbb{R}^{64}$)

Gli embedding dei byte f_i da soli **non contengono informazione sulla posizione**. Per ciascuna posizione $i = 0, 1, 2, 3$, calcoliamo un vettore di 64 dimensioni con sinusoidi:

$$p_i = \left[\sin\left(\frac{i}{\lambda_1}\right), \cos\left(\frac{i}{\lambda_1}\right), \sin\left(\frac{i}{\lambda_2}\right), \cos\left(\frac{i}{\lambda_2}\right), \dots \right]$$

- Posizione 0 $\rightarrow p_0 \in \mathbb{R}^{64}$
- Posizione 1 $\rightarrow p_1 \in \mathbb{R}^{64}$
- Posizione 2 $\rightarrow p_2 \in \mathbb{R}^{64}$
- Posizione 3 $\rightarrow p_3 \in \mathbb{R}^{64}$

4. Vettore finale per ogni byte

Ogni elemento dell'input è:

$$x_i = [f_i \| p_i] \in \mathbb{R}^{192}$$

Quindi:

- "C" $\rightarrow x_0 \in \mathbb{R}^{192}$
- "i" $\rightarrow x_1 \in \mathbb{R}^{192}$

- "a" $\rightarrow x_2 \in \mathbb{R}^{192}$
- "o" $\rightarrow x_3 \in \mathbb{R}^{192}$

5. Forma dell'array X

L'input array finale per la parola "Ciao" è:

$$X \in \mathbb{R}^{4 \times 192}$$

Esempio – ImageNet

$$X \in \mathbb{R}^{M \times C}$$

- $M = 224 \times 224 = 50176 \rightarrow$ numero di pixel (un token per pixel)
- $C = 3 + 128 = 131 \rightarrow$ feature per pixel (valori di colore + posizione)

Embedding del pixel (f_i)

- Ogni pixel ha tre valori RGB normalizzati.

$$f_i = (R, G, B) \in \mathbb{R}^3$$

Positional encoding (p_i)

- Le coordinate del pixel (x, y) vengono codificate con Fourier features (sinusoidi multiscala).

$$p_i \in \mathbb{R}^{128}$$

Vettore finale per ogni pixel

- Si concatenano colore e posizione:

$$x_i = [f_i || p_i] \in \mathbb{R}^{131}$$

Input array finale

- Ogni riga = un pixel codificato.
- L'immagine completa diventa:

$$X \in \mathbb{R}^{50176 \times 131}$$

Esempio – Video (Kinetics-700)

Nel benchmark Kinetics-700, ogni dato è una **clip video** (breve sequenza di secondi).

Il modello deve classificare l'azione contenuta nel video (es. correre, cucinare, ballare...).

Un video è una sequenza di **frame 2D** nel tempo $\rightarrow$ quindi diventa un **volume 3D** (spazio $\times$ tempo).

$$X \in \mathbb{R}^{M \times C}$$

- $M = H \times W \times T$

H = altezza frame (es. 224)

W = larghezza frame (es. 224)

T = numero di frame totali nel video (es. 16 o 32)

esempio: $224 \times 224 \times 16 \approx 802,816$ token

spesso ridotto a $\sim 50,000$ token tramite campionamento/sottocampionamento

- $C = 3 + 128 = 131$

3 valori di colore (R, G, B)

128 valori da positional encoding Fourier

Step 1- Embedding del voxel (f_i)

Ogni voxel è descritto dal colore:

$$f_i = (R, G, B) \in \mathbb{R}^3$$

- R, G, B = valori di colore normalizzati in $[0,1]$.
- Quindi f_i è un vettore di 3 numeri reali.

Esempi:

- Pixel bianco $\rightarrow f_i = (1,1,1)$
- Pixel nero $\rightarrow f_i = (0,0,0)$
- Pixel grigio (128,128,128) $\rightarrow f_i = (0.5,0.5,0.5)$

Step 2 - Positional encoding (p_i)

L'informazione del colore da sola non dice dove si trova il voxel.

Serve un encoding delle coordinate:

- $x \in [0, W - 1]$ = posizione orizzontale (colonna del pixel)
- $y \in [0, H - 1]$ = posizione verticale (riga del pixel)
- $t \in [0, T - 1]$ = indice del frame

Queste tre coordinate (x, y, t) vengono trasformate in un vettore sinusoidale multiscala:

$$p_i = [\sin(x/\lambda_1), \cos(x/\lambda_1), \dots, \sin(y/\lambda_j), \cos(y/\lambda_j), \sin(t/\lambda_k), \cos(t/\lambda_k)] \in \mathbb{R}^{128}$$

- Ogni λ è una frequenza diversa.
- Si usano molte frequenze $\rightarrow$ combinando seno e coseno si ottengono 128 numeri in totale.
- Così due voxel con stesso colore ma in posizioni diverse avranno vettori diversi.

Step 3 - Vettore finale del voxel (x_i)

Si concatenano colore e posizione:

$$x_i = [f_i || p_i] \in \mathbb{R}^{3+128} = \mathbb{R}^{131}$$

Quindi:

- parte "contenuto" = 3 numeri (colore)

- parte "dove si trova" = 128 numeri (posizione spaziotemporale)
- totale = 131 numeri per ogni voxel

Step 4 - Input array finale (X)

- Ogni voxel diventa una riga di X .
- Numero totale di voxel:

$$M = H \times W \times T$$

Per un video tipico ($224 \times 224, 16$ frame):

$$M = 224 \times 224 \times 16 \approx 802,816$$

Per motivi di efficienza si usano strategie di sottocampionamento, così si riduce a circa 50.000 voxel.

Forma finale:

$$X \in \mathbb{R}^{M \times 131}, M \approx 50,000$$

Esempio – Audio (waveform grezzo)

L'audio è un **segnale continuo nel tempo**, che viene campionato in una sequenza di valori (ampiezze). Ogni campione temporale diventa un **token** x_i

Step 1 - Embedding del campione (f_i)

- L'audio viene campionato, ad esempio a 16 kHz (16.000 campioni al secondo).
- Ogni campione è un valore reale dell'ampiezza della waveform, normalizzato in $[-1, 1]$.
- Questo valore è preso direttamente come embedding:

$$f_i \in \mathbb{R}^1$$

Esempi:

- Silenzio $\rightarrow f_i = 0$
- Massima ampiezza $\rightarrow f_i = 1$ o -1

Step 2 - Positional encoding (p_i)

- L'ampiezza da sola non dice quando nel tempo è stato registrato il campione.
- Ogni campione ha una posizione temporale:

$$t \in [0, M - 1]$$

dove M = numero totale di campioni.

Per distinguere i campioni, la posizione viene codificata con sinusoidi multiscala (Fourier features):

$$p_i = [\sin(t/\lambda_1), \cos(t/\lambda_1), \sin(t/\lambda_2), \cos(t/\lambda_2), \dots] \in \mathbb{R}^{64}$$

- Totale: 64 numeri.

Questo dà al modello nozione di ordine temporale.

Step 3 - Vettore finale per ogni campione

Concatenazione:

$$x_i = [f_i || p_i] \in \mathbb{R}^{1+64} = \mathbb{R}^{65}$$

- 1 numero = ampiezza
- 64 numeri = posizione temporale
- totale = 65 numeri per ogni campione audio

Step 4 - Input array finale (X)

- Ogni campione audio diventa una riga di X .
- Se prendiamo 1 secondo di audio a 16 kHz:

$$M = 16,384 \text{ (circa)}$$

Forma finale:

$$X \in \mathbb{R}^{16384 \times 65}$$

Esempio - Input multimodale (Vision + Language, VQA)

Nel compito **Visual Question Answering (VQA)** l'input è composto da:

1. un'immagine (es. una foto),
2. una domanda in linguaggio naturale (testo).

Domanda:

👉 "What color is the cat?"

Immagine:

👉 una foto RGB 224×224

Il modello riceve entrambi come un unico **input array** X

$$X \in \mathbb{R}^{M \times C}$$

- $M = M_{\text{img}} + M_{\text{text}}$
= numero di pixel dell'immagine + numero di byte della domanda
- $C = C_{\text{base}} + d_m$
dove d_m è la dimensione dell'embedding di modalità (nel paper $d_m = 16$).

Step 1 - Immagine

Ogni pixel dell'immagine (ad esempio 224×224):

$$f_i^{\text{img}} = (R, G, B) \in \mathbb{R}^3$$

Coordinate spaziali $(x, y) \rightarrow$ Fourier features:

$$p_i^{\text{img}} \in \mathbb{R}^{128}$$

Embedding di modalità (per dire che è "immagine"):

$$m^{\text{img}} \in \mathbb{R}^{16}$$

Concatenazione:

$$x_i^{img} = [f_i^{img} \parallel p_i^{img} \parallel m^{img}] \in \mathbb{R}^{3+128+16} = \mathbb{R}^{147}$$

Totale per tutta l'immagine:

$$M_{img} = H \times W \Rightarrow X^{img} \in \mathbb{R}^{M_{img} \times 147}$$

Step 2 - Testo

La domanda testuale viene convertita in byte UTF-8.

Ogni byte $b_j \in \{0, \dots, 255\}$:

Embedding del byte:

$$f_j^{text} \in \mathbb{R}^{128}$$

Posizione nella sequenza → Fourier features:

$$p_j^{text} \in \mathbb{R}^{64}$$

Embedding di modalità (per dire che è "testo"):

$$m^{text} \in \mathbb{R}^{16}$$

Concatenazione:

$$x_j^{text} = [f_j^{text} \parallel p_j^{text} \parallel m^{text}] \in \mathbb{R}^{128+64+16} = \mathbb{R}^{208}$$

Totale per la domanda:

$$M_{text} = \text{numero di byte} \Rightarrow X^{text} \in \mathbb{R}^{M_{text} \times 208}$$

Step 3 - Array multimodale finale

Concatenazione immagine + testo:

$$X = \begin{bmatrix} X^{img} \\ X^{text} \end{bmatrix}, X \in \mathbb{R}^{(M_{img} + M_{text}) \times C}$$

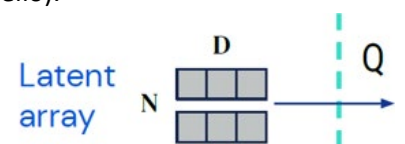
dove:

- $C = 147$ per i token immagine,
- $C = 208$ per i token testuali.

Latent Array:

È una **matrice di vettori latenti** $Z \in \mathbb{R}^{N \times D}$ **interamente appresa** (parametri del modello).

- **N** = numero di latenti (slot/ "unità di memoria").
- **D** = dimensione del canale/embedding dei latenti



Questi vettori **non derivano** direttamente dall'input e **non hanno un legame spaziale** con token/pixel specifici: sono una base compatta che il modello usa per riassumere l'input.

Caso 1 - Testo (GLUE, byte-level)

- Input: sequenza di byte UTF-8 $\rightarrow M = \text{lunghezza in byte (es. 2048)}$.
- Ogni byte: embedding 128 + posizione 64 $\rightarrow C = 192$.
- Input array: $X \in \mathbb{R}^{2048 \times 192}$.
- Latenti: $Z \in \mathbb{R}^{512 \times 512}$.

I 512 latenti leggono i 2048 byte.

Caso 2 - Immagini (ImageNet)

- Input: immagine $224 \times 224 = 50.176$ pixel.
- Ogni pixel: colore RGB (3) + posizione (128) $\rightarrow C = 131$.
- Input array: $X \in \mathbb{R}^{50176 \times 131}$.
- Latenti: $Z \in \mathbb{R}^{512 \times 512}$

I 512 latenti leggono da 50.176 pixel.

Caso 3 - Video (Kinetics-700)

- Input: volume $224 \times 224 \times 16 \approx 802.000$ voxel.
- Ogni voxel: colore RGB (3) + posizione (x,y,t) Fourier (128) $\rightarrow C = 131$.
- Input array (sottocampionato): $X \in \mathbb{R}^{50000 \times 131}$.
- Latenti: $Z \in \mathbb{R}^{512 \times 512}$.

I 512 latenti leggono un sottoinsieme dei 50.000 voxel.

Caso 4 - Audio (waveform)

- Input: 1s audio a 16kHz $\rightarrow 16.384$ campioni.
- Ogni campione: ampiezza (1) + posizione Fourier (64) $\rightarrow C = 65$.
- Input array: $X \in \mathbb{R}^{16384 \times 65}$.
- Latenti: $Z \in \mathbb{R}^{512 \times 512}$.

I 512 latenti leggono sequenze di campioni.

Caso 5 - Multimodale (VQA: immagine + testo)

- Input:
- immagine $224 \times 224 = 50.176$ pixel $\rightarrow C = 147$ (RGB+posizione+modalità).
- domanda testuale (es. 20 byte) $\rightarrow C = 208$ (byte+posizione+modalità).
- Input array: concatenazione dei due $\rightarrow X \in \mathbb{R}^{(50176+20) \times C}$.
- Latenti: $Z \in \mathbb{R}^{512 \times 512}$.

I 512 latenti leggono sia pixel che testo.

Encode Module

L'Encode Module è il blocco di **cross-attention** che trasferisce l'informazione dall'**Input array X** al **Latent array Z**.

- **Idea:** i latenti (Q) leggono i contenuti dall'input (K, V).
- **Output:** un nuovo stato latente Z^1 con la stessa forma di partenza $N \times D$, ma arricchito dei dati dell'input.

Input del blocco

- Latent array iniziale

$$Z^{(0)} \in \mathbb{R}^{N \times D}$$

- Input array

$$X \in \mathbb{R}^{M \times C}$$

1. Proiezioni lineari

$$\begin{aligned} Q &= Z^{(0)} W_Q & W_Q &\in \mathbb{R}^{D \times d_k}, & Q &\in \mathbb{R}^{N \times d_k} \\ K &= X W_K & W_K &\in \mathbb{R}^{C \times d_k}, & K &\in \mathbb{R}^{M \times d_k} \\ V &= X W_V & W_V &\in \mathbb{R}^{C \times d_v}, & V &\in \mathbb{R}^{M \times d_v} \end{aligned}$$

2. Attention Scores

$$S = \frac{QK^T}{\sqrt{d_k}} \quad S \in \mathbb{R}^{N \times M}$$

3. Softmax (pesi di attenzione)

$$A_{ij} = \frac{e^{S_{ij}}}{\sum_{m=1}^M e^{S_{im}}} \quad A \in \mathbb{R}^{N \times M}$$

4. Aggregazione

$$O = AV \quad O \in \mathbb{R}^{N \times d_v}$$

5. Proiezione lineare finale

$$\tilde{Z} = OW_O \quad W_O \in \mathbb{R}^{d_v \times D}, \tilde{Z} \in \mathbb{R}^{N \times D}$$

6. Residual Connection (post-attention)

$$Z_a = Z^{(0)} + \tilde{Z} \in \mathbb{R}^{N \times D}$$

7. Layer Normalization (post-attention)

$$\hat{Z} = \text{LN}(Z_a) \in \mathbb{R}^{N \times D}$$

8. Feed Forward (MLP a 2 layer)

- Espansione:

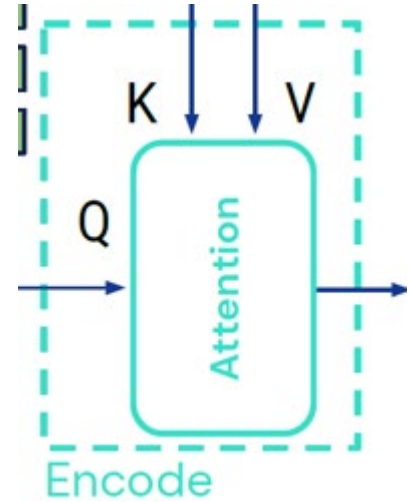
$$H = \hat{Z} W_1 + b_1, W_1 \in \mathbb{R}^{D \times 4D}, H \in \mathbb{R}^{N \times 4D}$$

- Attivazione (GELU/ReLU):

$$H' = \sigma(H) \in \mathbb{R}^{N \times 4D}$$

- Compressione:

$$F = H' W_2 + b_2, W_2 \in \mathbb{R}^{4D \times D}, F \in \mathbb{R}^{N \times D}$$



9. Residual Connection (post-MLP)

$$Z_b = \hat{Z} + F \in \mathbb{R}^{N \times D}$$

10. Layer Normalization (post-MLP)

$$Z^{(1)} = \text{LN}(Z_b) \in \mathbb{R}^{N \times D}$$

Output del blocco Encode

$$Z^{(1)} \in \mathbb{R}^{N \times D}$$

Caso 1 - Testo (GLUE, byte-level)

- **Input:** sequenza di byte UTF-8
- **Dimensioni:** $M = L_{\text{byte}}$ (nel paper fino a 2048), $C = 128 + 64 = 192$
- **Forme:**

$$\begin{aligned} X &\in \mathbb{R}^{M \times 192} \\ K &\in \mathbb{R}^{M \times d_k}, V \in \mathbb{R}^{M \times d_v} \\ Q &\in \mathbb{R}^{512 \times d_k} \text{ (se } N = 512\text{)} \\ S, A &\in \mathbb{R}^{512 \times M} \\ Z^{(1)} &\in \mathbb{R}^{512 \times 512} \end{aligned}$$

- **Cosa fa:** i latenti pesano byte e posizioni (Fourier 1D) per condensare pattern della sequenza in 512 vettori compatti.

Caso 2 - Immagini (ImageNet)

- **Input:** immagine 224×224 RGB
- **Dimensioni:** $M = 224 \cdot 224 = 50,176$, $C = 3 + 128 = 131$
- **Forme:**

$$\begin{aligned} X &\in \mathbb{R}^{50176 \times 131} \\ K &\in \mathbb{R}^{50176 \times d_k}, V \in \mathbb{R}^{50176 \times d_v} \\ Q &\in \mathbb{R}^{512 \times d_k} \\ S, A &\in \mathbb{R}^{512 \times 50176} \\ Z^{(1)} &\in \mathbb{R}^{512 \times 512} \end{aligned}$$

- **Cosa fa:** i latenti leggono pixel + posizioni 2D (Fourier) e si specializzano su frequenze/regioni (bordi, texture, struttura globale).

Caso 3 — Video (Kinetics-700)

- **Input:** volume $H \times W \times T$ (es. $224 \times 224 \times 16$)
- **Dimensioni:** $M_{\text{raw}} = HWT \approx 802,816 \rightarrow$ sottocampionato a $M \approx 50,000$; $C = 3 + 128 = 131$
- **Forme:**

$$\begin{aligned} X &\in \mathbb{R}^{50000 \times 131} \\ K &\in \mathbb{R}^{50000 \times d_k}, V \in \mathbb{R}^{50000 \times d_v} \\ Q &\in \mathbb{R}^{512 \times d_k} \\ S, A &\in \mathbb{R}^{512 \times 50000} \\ Z^{(1)} &\in \mathbb{R}^{512 \times 512} \end{aligned}$$

- Cosa fa: integra spazio+tempo (Fourier 3D). Latenti diversi catturano movimento, coerenze temporali e contesto statico.

Caso 4 - Audio (waveform)

- **Input:** waveform campionata (es. $\sim 16k$ campioni/s)
- **Dimensioni:** per ~ 1 s $\rightarrow M \approx 16,000$; $C = 1 + 64 = 65$
- **Forme:**

$$\begin{aligned} X &\in \mathbb{R}^{M \times 65} \\ K &\in \mathbb{R}^{M \times d_k}, V \in \mathbb{R}^{M \times d_v} \\ Q &\in \mathbb{R}^{512 \times d_k} \\ S, A &\in \mathbb{R}^{512 \times M} \\ Z^{(1)} &\in \mathbb{R}^{512 \times 512} \end{aligned}$$

- **Cosa fa:** latenti "sintonizzano" porzioni temporali e frequenze implicite (grazie alle Fourier 1D) per riassumere il segnale.

Caso 5 - Multimodale (VQA: immagine + testo)

- **Input:** pixel immagine + byte domanda concatenati
- **Dimensioni per modalità:**
 - **Immagine:** $M_{\text{img}} = 50176$, $C_{\text{img}} = 3 + 128 + 16 = 147$
 - **Testo:** $M_{\text{text}} = L_{\text{byte}}$, $C_{\text{text}} = 128 + 64 + 16 = 208$
 - **Totale:** $M = M_{\text{img}} + M_{\text{text}}$. In pratica si proietta ogni modalità a un canale comune C^* prima dell'Encode.
- **Forme (dopo allineamento canali):**

$$X \in \mathbb{R}^{(M_{\text{img}} + M_{\text{text}}) \times C^*}$$

K, V con la stessa M ;

$$Q \in \mathbb{R}^{512 \times d_k};$$

$$S, A \in \mathbb{R}^{512 \times M};$$

$$Z^{(1)} \in \mathbb{R}^{512 \times 512}$$

- **Cosa fa:** i latenti fondono contenuto visivo e linguistico; l'attenzione sfrutta anche il modality embedding per collegare parole a regioni d'immagine.

Process Module (Latent Transformer / Latent Attention)

Input del blocco

- Latent array aggiornato dall'Encode:

$$Z^{(1)} \in \mathbb{R}^{N \times D}$$

Proiezioni lineari

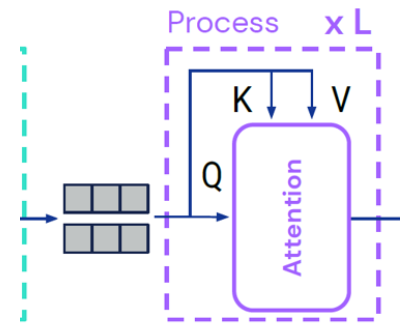
$$Q = Z^{(1)} W_Q, Q \in \mathbb{R}^{N \times d_k}$$

$$K = Z^{(1)} W_K, K \in \mathbb{R}^{N \times d_k}$$

$$V = Z^{(1)} W_V, V \in \mathbb{R}^{N \times d_v}$$

- $W_Q, W_K, W_V \in \mathbb{R}^{D \times d_k}$.

Stavolta Query, Key e Value provengono tutti dai latenti stessi.



Attention Scores

$$S = \frac{QK^T}{\sqrt{d_k}} S \in \mathbb{R}^{N \times N}$$

Ogni latente misura la similarità con tutti gli altri

Softmax (pesi di attenzione)

$$A_{ij} = \frac{e^{S_{ij}}}{\sum_{m=1}^N e^{S_{im}}}, A \in \mathbb{R}^{N \times N}$$

Distribuzione di attenzione tra latenti (quanto un latente guarda gli altri)

Aggregazione

$$O = AV O \in \mathbb{R}^{N \times d_v}$$

Ogni latente raccoglie informazione da tutti gli altri.

Proiezione lineare finale

$$\tilde{Z} = OW_O, W_O \in \mathbb{R}^{d_v \times D}, \tilde{Z} \in \mathbb{R}^{N \times D}$$

Residual Connection (post-attention)

$$Z_a = Z^{(1)} + \tilde{Z}, Z_a \in \mathbb{R}^{N \times D}$$

Layer Normalization (post-attention)

$$\hat{Z} = \text{LN}(Z_a), \hat{Z} \in \mathbb{R}^{N \times D}$$

Feed Forward (MLP a 2 layer)

- Espansione:

$$H = \hat{Z}W_1 + b_1, W_1 \in \mathbb{R}^{D \times 4D}, H \in \mathbb{R}^{N \times 4D}$$

- Attivazione (GELU/ReLU):

$$H' = \sigma(H), H' \in \mathbb{R}^{N \times 4D}$$

- Compressione:

$$F = H'W_2 + b_2, W_2 \in \mathbb{R}^{4D \times D}, F \in \mathbb{R}^{N \times D}$$

Residual Connection (post-MLP)

$$Z_b = \hat{Z} + F, Z_b \in \mathbb{R}^{N \times D}$$

Layer Normalization (post-MLP)

$$Z^{(2)} = \text{LN}(Z_b), Z^{(2)} \in \mathbb{R}^{N \times D}$$

Output del blocco Process

$$Z^{(2)} \in \mathbb{R}^{N \times D}$$

- Stessa forma dei latenti in input ($N \times D$, es. 512×512).
- Contenuto aggiornato: i latenti hanno comunicato tra loro, scambiandosi informazione.
- Questo blocco si ripete L volte:

$$Z^{(L)} = \text{Process}^L(Z^{(1)})$$

Caso 1 - Testo (GLUE, byte-level)

- **Input iniziale Encode:** $X \in \mathbb{R}^{M \times 192}, M \leq 2048$.
- **Dopo Encode:** $Z^{(1)} \in \mathbb{R}^{512 \times 512}$.
- **Process:**

$$Q, K, V \in \mathbb{R}^{512 \times d_k}.$$

$$S, A \in \mathbb{R}^{512 \times 512}.$$

$$Z^{(2)} \in \mathbb{R}^{512 \times 512}.$$

- Cosa fa: i 512 latenti scambiano informazione tra loro → alcuni si specializzano in pattern di caratteri, altri in sintassi globale.

Caso 2 - Immagini (ImageNet)

- **Input iniziale Encode:** $X \in \mathbb{R}^{50176 \times 131}$.
- **Dopo Encode:** $Z^{(1)} \in \mathbb{R}^{512 \times 512}$.
- **Process:**

$$Q, K, V \in \mathbb{R}^{512 \times d_k}.$$

$$S, A \in \mathbb{R}^{512 \times 512}.$$

$$Z^{(2)} \in \mathbb{R}^{512 \times 512}.$$

- Cosa fa: i latenti mettono in relazione feature visive (bordi, texture, regioni) scambiandosi info → emergono strutture globali.

Caso 3 - Video (Kinetics-700)

- **Input iniziale Encode:** $X \in \mathbb{R}^{50000 \times 131}$ (sottocampionato).
- **Dopo Encode:** $Z^{(1)} \in \mathbb{R}^{512 \times 512}$.
- **Process:**

$$Q, K, V \in \mathbb{R}^{512 \times d_k}.$$

$$S, A \in \mathbb{R}^{512 \times 512}.$$

$$Z^{(2)} \in \mathbb{R}^{512 \times 512}.$$

- Cosa fa: latenti specializzati su movimento e frame statici interagiscono → si combinano in rappresentazioni spaziotemporali più ricche.

Caso 4 - Audio (waveform)

- **Input iniziale Encode:** $X \in \mathbb{R}^{M \times 65}$, $M \approx 16000$.
- **Dopo Encode:** $Z^{(1)} \in \mathbb{R}^{512 \times 512}$.
- **Process:**

$$Q, K, V \in \mathbb{R}^{512 \times d_k}.$$

$$S, A \in \mathbb{R}^{512 \times 512}.$$

$$Z^{(2)} \in \mathbb{R}^{512 \times 512}.$$

- Cosa fa: latenti che hanno letto frammenti di segnale (tempo/frequenze) si influenzano tra loro → emergono pattern fonetici o ritmici globali.

Caso 5 - Multimodale (VQA: immagine + testo)

- **Input iniziale Encode:**
Immagine: $X_{img} \in \mathbb{R}^{50176 \times 147}$.
Testo: $X_{txt} \in \mathbb{R}^{M_{txt} \times 208}$.
- **Concatenati e proiettati** → $X \in \mathbb{R}^{(M_{img} + M_{txt}) \times C^*}$.
- **Dopo Encode:** $Z^{(1)} \in \mathbb{R}^{512 \times 512}$.
- **Process:**

$$Q, K, V \in \mathbb{R}^{512 \times d_k}.$$

$$S, A \in \mathbb{R}^{512 \times 512}.$$

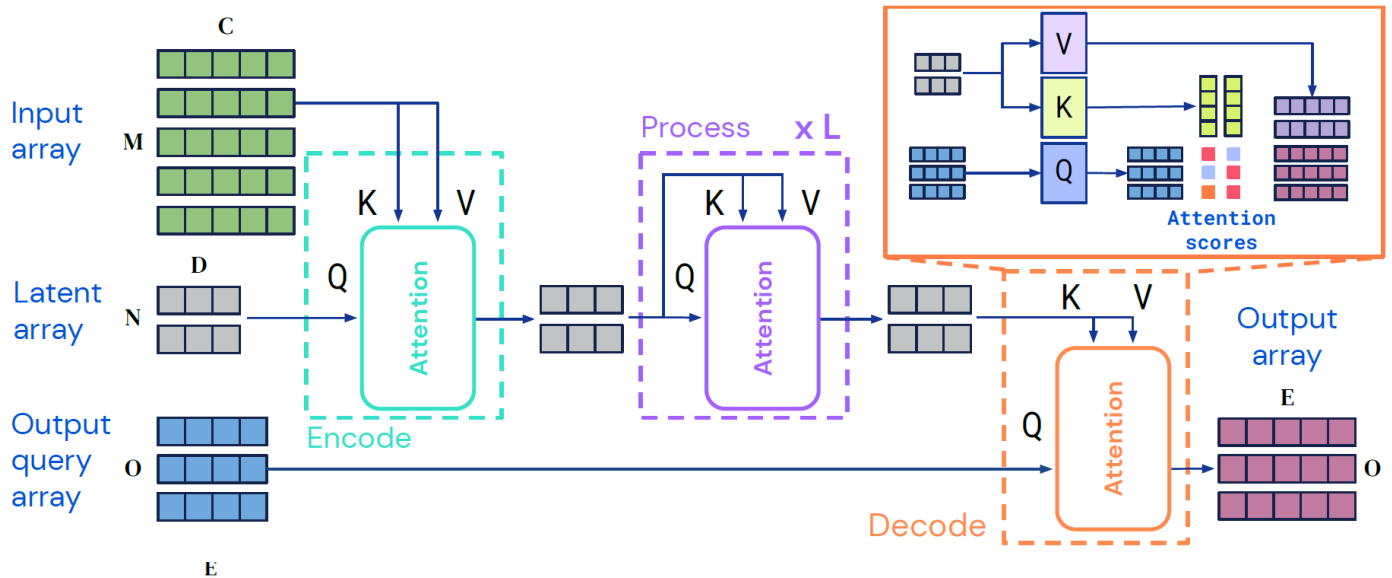
$$Z^{(2)} \in \mathbb{R}^{512 \times 512}.$$

- Cosa fa: i latenti scambiano info multimodali → collegano regioni visive a parole rilevanti, integrando i due canali.

Output del Process

$$Z^{(L)} \in \mathbb{R}^{N \times D}$$

- Stessa forma (512×512).
- Contenuto raffinato e arricchito dopo L layer di self-attention.
- Rappresenta la memoria interna finale che verrà usata dal Decode.



Ricapitolando

Input array: sequenza di token (immagine, testo, audio...)

$$X \in \mathbb{R}^{M \times C}$$

Latent array : spazio compatto dove il modello elabora l'informazione

$$Z \in \mathbb{R}^{N \times D}$$

Dopo L layer di process (self-attention), abbiamo un array latente raffinato:

$$Z^{(L)} \in \mathbb{R}^{N \times D}$$

Output Query Array

È una matrice di query $Y^{(0)} \in \mathbb{R}^{O \times E}$, dove:

- O = numero di query, cioè quante "domande" vogliamo fare ai latenti (per esempio una nel caso di classificazione)
- E = dimensione iniziale delle query (può variare a seconda di come le costruiamo)

$$E = \text{dim}(\text{posizione}) + \text{dim}(\text{tipo}) + \text{dim}(\text{modalità}) + \text{dim}(\text{contesto (opzionale)})$$

Ogni riga di $Y^{(0)}$ è un vettore che specifica:

- Che tipo di informazione vogliamo leggere dai latenti e dove/per chi vogliamo questa informazione es. posizione spaziale, temporale, categoria.

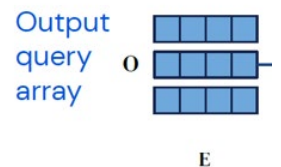
Le query sono quindi una guida per il Decode: invece di leggere tutto il contenuto latente, chiediamo solo ciò che serve.

Cosa contiene una query $y_i^{(0)}$

È un descrittore ottenuto in genere per concatenazione di blocchi (tutti vettori):

$$y_i^{(0)} = [\underbrace{\text{posizione}}_{\text{dim}=P} \parallel \underbrace{\text{tipo di output}}_{\text{dim}=T} \parallel \underbrace{\text{modalità}}_{\text{dim}=M} \parallel \underbrace{\text{contesto opzionale}}_{\text{dim}=C}]$$

$$E = P + T + M + C$$



Ogni parte è un blocco informativo e contiene un tipo di segnale diverso che aiuta il modello a interpretare la query e a decidere dove guardare nei latenti e cosa leggere.

Questi blocchi possono essere **learned** (parametri) o **deterministici** (Fourier); in entrambi i casi ricevono gradiente tramite il decode.

- **Posizione:** coord. spaziali/temporali/indice di token (spesso con Fourier features multiscala).

Il modello deve sapere a quale punto dell'output la query si riferisce, se per esempio a un pixel, a un token di testo o a un frame temporale.

a) **Deterministico**

Si codifica la posizione con una funzione matematica fissa, che non ha parametri:

- **Fourier Features:** si mappano le coordinate (x, y, t, indice) in sinusoidi multiscala:

$$p(i) = [\sin(2\pi Bx), \cos(2\pi Bx), \sin(2\pi By), \cos(2\pi By), \dots]$$

Dove B rappresenta diverse frequenze spaziali o temporali.

b) **Learned**

Si assegna a ogni posizione un vettore trainabile (una riga di una tabella di embedding).

$$p(i) = E_{\text{pos}}[i], \quad E_{\text{pos}} \in \mathbb{R}^{N_{\text{pos}} \times P}$$

- **Tipo di output:** embedding che distingue cosa stai chiedendo (es. classificazione vs segmentazione vs testo).

Indica che tipo di informazione stai chiedendo al modello per esempio:

- Dammi una classificazione
- Dammi un token di testo (decodifica)
- Dammi la segmentazione di un pixel

È sempre un embedding trainabile (learned):

$$t = E_{\text{type}}[\text{'classification'}], \quad T \approx 16$$

Ogni tipo ha un proprio vettore che il modello può aggiornare durante l'allenamento.

- **Modalità:** embedding che indica la modalità dell'output (immagine, testo, audio, ...).

Specifica da quale dominio stai generando l'output, diventa cruciale nei modelli multimodali perché i latenti contengono informazioni miste.

Embedding trainabile (learned):

$$m = E_{\text{mod}}[\text{'image'}], \quad M \approx 16$$

Tutti gli output appartenenti alla stessa modalità condividono lo stesso vettore.

Esempio:

In VQA (Vision + Language), le query della risposta testuale avranno un embedding di modalità "text", mentre i latenti provengono da immagini + testo.

Questo aiuta l'attenzione a "capire" che deve restituire parole, non feature visive.

- **Contesto opzionale:** vettori che riassumono parti dell'input utili a guidare la decodifica (es. nel multimodale).

Trasporta **informazioni aggiuntive** che aiutano la query a essere più specifica.

È opzionale, ma utile nei compiti dove l'output dipende da un input complesso o da un'altra modalità.

Allineamento ai latenti: poiché i latenti hanno dimensione D , tutte le query $Y^{(0)}$ verranno proiettate con W_q nello spazio di Q per il decode

$$Q = Y^{(0)}W_q \in \mathbb{R}^{O \times D}, \quad W_q \in \mathbb{R}^{E \times D}$$

(semplice proiezione lineare "traduttrice" da spazio query E allo spazio latenti D).

1. Testo - GLUE (byte-level, classificazione)

Task: Compito di classificazione testuale: per esempio determinare se una frase è positiva o negativa. L'output è una sola etichetta per tutto il testo.

Idea: chiedo una sola decisione globale sul testo; non serve posizione.

- O : 1 (un'unica predizione globale).
- $y^{(0)}$: tipo di output (classificazione) + modalità (testo). (Posizione non necessaria: l'output non è per token.)
- $E: T + M \rightarrow$ tipico $16 + 16 = 32$.
- $Y^{(0)}: \mathbb{R}^{1 \times 32}$.
- (Proiezione) $Q: Y^{(0)}W_q \in \mathbb{R}^{1 \times D}$.

La query chiede:

“Riassumi il significato complessivo del testo e dimmi la sua classe.”

Il decoder risponderà con un vettore di logits (es. $[0.8, 0.2] \rightarrow$ positivo).

È l'esatto equivalente del *token [CLS]* dei Transformer, ma qui è una *query di output* esterna ai latenti.

2. Immagini - ImageNet (classificazione)

Task: Classificare un'intera immagine in una delle 1000 classi ImageNet.

Idea: una sola etichetta per tutta l'immagine; niente coordinate in query.

- O : 1 (categoria globale).
- $y^{(0)}$: tipo di output (classificazione) + modalità (immagine).
- $E: T + M = 16 + 16 = 32$.
- $Y^{(0)}: \mathbb{R}^{1 \times 32}$.
- (Proiezione) $Q: Y^{(0)}W_q \in \mathbb{R}^{1 \times D}$

La query rappresenta:

“Riassumi le feature visive dell'immagine e dammi la classe principale.”

La cross-attention tra questa query e i latenti (che rappresentano tutti i pixel codificati) fa sì che il modello estragga un'unica risposta aggregata da tutto il contenuto visivo.

3. Video - Kinetics (classificazione azione)

Task: Classificare l'azione principale di un video (es. “correre”, “bere”, “saltare”).

Idea: riassumo l'intero video in una classe; nessuna posizione temporale nell'output.

- O : 1 (azione globale).
- $y^{(0)}$: tipo di output (classificazione) + modalità (video).
- $E: T + M = 16 + 16 = 32$.
- $Y^{(0)}: \mathbb{R}^{1 \times 32}$.
- (Proiezione) $Q: \mathbb{R}^{1 \times D}$.

“Guarda tutti i frame e sintetizza quale azione viene svolta.”

Il decoder aggrega informazioni temporali già presenti nei latenti e restituisce una label tra le 700 classi di Kinetics.

Anche qui, una sola query è sufficiente per un output globale.

4. Audio - waveform (classificazione clip)

Task: Riconoscere il contenuto di un clip audio (es. parola, comando, tipo di suono).

L'audio viene trattato come una sequenza di campioni, ma l'output è una classe globale.

Idea: una classe per l'intero clip; niente posizione nell'output.

- O : 1 (classe del clip).
- $y^{(0)}$: tipo di output (classificazione) + modalità (audio).
- $E: T + M = 16 + 16 = 32$.
- $Y^{(0)}: \mathbb{R}^{1 \times 32}$.
- (Proiezione) $Q: \mathbb{R}^{1 \times D}$.

“Analizza la clip completa e restituisci la categoria sonora.”

Il modello combina informazioni temporali dai latenti (che rappresentano l'intera waveform) per fornire una classificazione finale (es. “comando: stop”, “rumore: applauso”).

5. Multimodale - VQA (immagine + testo, risposta testuale)

Task: Data un'immagine e una domanda in linguaggio naturale, generare una risposta testuale

(es. “What color is the car?” → “Red”)

Idea: devo generare una sequenza di token; ogni query chiede il j -esimo token.

- $O: L$ (numero massimo di token in output).
- $y^{(0)}$ (per ciascun token):
posizione del token (positional/Fourier) + tipo di output (testo) + modalità (testo) + (opz.) contesto (embedding della domanda).
- $E: P + T + M(+C) \rightarrow$ tipico ≈ 128 (es. 64 pos + 16 tipo + 16 modalità + 32 contesto).
- $Y^{(0)}: \mathbb{R}^{L \times 128}$.
- (Proiezione) $Q: \mathbb{R}^{L \times D}$.

Ogni riga di $Y^{(0)}$ è una “query token”:

- Query 1 → “Dammi la **prima parola** della risposta.”
- Query 2 → “Dammi la **seconda parola**, sapendo cosa ho già detto e cosa chiede la domanda.”
- ... e così via.

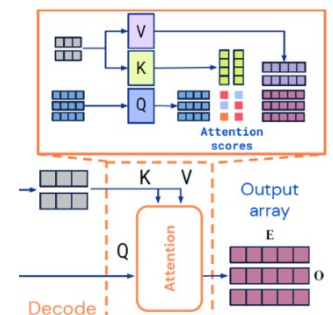
Il decoder utilizza le informazioni multimodali nei latenti (immagine + testo) e, grazie al contesto, produce token coerenti con la domanda.

Caso	O	P	T	M	C	E	Tipo di output	Descrizione intuitiva
Testo (GLUE)	1	0	16	16	0	32	Classificazione	“Riassumi il testo → dammi la label”
Immagini (ImageNet)	1	0	16	16	0	32	Classificazione	“Riassumi i pixel → dammi l’oggetto”
Video (Kinetics)	1	0	16	16	0	32	Classificazione	“Riassumi i frame → dammi l’azione”
Audio (waveform)	1	0	16	16	0	32	Classificazione	“Riassumi i campioni → dammi il suono”
VQA (multimodale)	L	64	16	16	32	128	Sequenza testuale	“Genera i token della risposta alla domanda”

Blocco Decode

Il Decode riceve due “sorgenti” di informazione:

- le **Output Queries** (ciò che chiediamo),
- i **Latenti finali** (ciò che il modello ha compreso sull’input).



Output query Array $Y^{(0)}$

Simbolo	Nome	Provenienza	Significato	Dimensione
$Y^{(0)}$	Output Query Array	Definito dal task (es. 1 query per classificazione, L per multimodale)	Specifica cosa vogliamo leggere dai latenti (posizione, tipo di output, modalità, contesto)	$\mathbb{R}^{O \times E}$

Latent Array finale $Z^{(L)}$

Simbolo	Nome	Provenienza	Significato	Dimensione
$Z^{(L)}$	Latent Array dopo il Process	Risultato del blocco Process (self-attention sui latenti)	Contiene la rappresentazione compressa dell’intero input (immagine, testo, audio, ecc.)	$\mathbb{R}^{N \times D}$

Partendo da questi due elementi dobbiamo ottenere Q, K e V , che possono essere ottenuti mediante proiezione lineare con le seguenti matrici di peso trainabili:

$$\begin{aligned}
 Q &= Y^{(0)} W_Q & W_Q &\in \mathbb{R}^{E \times (h \cdot d_k)} \Rightarrow Q \in \mathbb{R}^{O \times (h \cdot d_k)} \\
 K &= Z^{(L)} W_K & W_K &\in \mathbb{R}^{D \times (h \cdot d_k)} \Rightarrow K \in \mathbb{R}^{N \times (h \cdot d_k)} \\
 V &= Z^{(L)} W_V & W_V &\in \mathbb{R}^{D \times (h \cdot d_v)} \Rightarrow V \in \mathbb{R}^{N \times (h \cdot d_v)}
 \end{aligned}$$

Perché usare $h \cdot d_k$ e non semplicemente D ?

Perché l’attenzione è **multi-head**:

- invece di una singola attenzione grande, si usano h attenzioni indipendenti in parallelo;
- ognuna ha dimensione ridotta d_k o d_v ;

Questo migliora la **capacità di rappresentare pattern diversi** contemporaneamente (es. attenzione a posizioni, colori, parole, suoni, ecc.).

Nel paper abbiamo le seguenti dimensioni:

Parametro	Valore
$E = D = 512$	dimensione modello
$h = 8$	8 teste
$d_k = d_v = 64$	dimensione interna per testa
Quindi $h \cdot d_k = h \cdot d_v = 512$	totale equivalente a D
Da cui:	
$ \begin{aligned} W_Q &\in \mathbb{R}^{512 \times 512} \\ W_K &\in \mathbb{R}^{512 \times 512} \\ W_V &\in \mathbb{R}^{512 \times 512} \\ Q &\in \mathbb{R}^{O \times 512}, K, V \in \mathbb{R}^{N \times 512} \end{aligned} $	

Calcolo dell'attenzione nel Decode

Per ogni testa di attenzione, calcoliamo:

$$S = \frac{QK^T}{\sqrt{d_k}}$$

- $Q \in \mathbb{R}^{O \times d_k}$
- $K \in \mathbb{R}^{N \times d_k}$

$$\rightarrow S \in \mathbb{R}^{O \times N}$$

Ogni riga di S rappresenta una query (cioè un elemento dell'output). Il prodotto scalare misura la somiglianza: se Q e K sono simili, il valore S_{ij} è alto $\rightarrow$ quella parte del latente sarà considerata importante per quella query.

La divisione per $\sqrt{d_k}$ serve solo per mantenere i valori in un range numerico stabile (evita che la softmax dopo "esploda").

Una volta ottenuto S , dobbiamo creare **la matrice di attenzione A** , che userai per pesare i Value V .

$$A = \text{softmax}(S) \quad A \in \mathbb{R}^{O \times N}$$

- O = numero di query (una riga per ogni elemento di output)
- N = numero di latenti (una colonna per ogni latente)

Ogni riga di A contiene i pesi di attenzione della query corrispondente. Come sappiamo con la softmax trasformiamo ogni riga di S (che contiene punteggi grezzi) in pesi normalizzati compresi tra 0 e 1, che sommano a 1.

$$A_{ij} = \frac{e^{S_{ij}}}{\sum_{n=1}^N e^{S_{in}}}$$

- **Valore alto** $\rightarrow$ la query guarda **molto** quel latente.
- **Valore basso** $\rightarrow$ la query lo considera **poco**.

Ora che abbiamo $A = \text{softmax}(S)$, dobbiamo usare questi pesi di attenzione per leggere le informazioni dai latenti.

Questo avviene calcolando:

$$O = AV$$

Simbolo	Dimensione	Provenienza
A	$O \times N$	pesi di attenzione (una riga per query, una colonna per latente)
V	$N \times d_v$	vettori dei latenti (contenuto informativo)
O	$O \times d_v$	risultato per query

Ogni riga di O rappresenta la risposta del modello per una query:

$$O_i = \sum_{j=1}^N A_{ij} V_j$$

In parole semplici:

La query i legge dai latenti una "media pesata" dei loro vettori V_j , dove i pesi A_{ij} dicono quanto ciascun latente è importante per quella query.

- **Multi-head attention**

Il calcolo $O = AV$ avviene indipendentemente per ogni testa di attenzione.

Ogni testa t produce il proprio:

$$O^{(t)} = A^{(t)} V^{(t)} \text{ con } O^{(t)} \in \mathbb{R}^{O \times d_v}$$

Poi tutti gli $O^{(t)}$ vengono concatenati:

$$O^{(\text{concat})} = [O^{(1)} \parallel O^{(2)} \parallel \dots \parallel O^{(h)}] \Rightarrow O^{(\text{concat})} \in \mathbb{R}^{O \times (h \cdot d_v)}$$

- **Proiezione finale**

Per riportare il risultato nello spazio comune dei latenti (dimensione D):

$$\tilde{Y} = O^{(\text{concat})} W_O$$

$$W_O \in \mathbb{R}^{(h \cdot d_v) \times D}, \quad \tilde{Y} \in \mathbb{R}^{O \times D}$$

Questo passaggio finale:

- fonde le informazioni provenienti da tutte le teste,
- ricompone un'unica rappresentazione per ciascuna query, compatibile con la dimensione modello D .

Quindi quello che otteniamo da questa fase della cross-attention sarà:

$$\tilde{Y} \in \mathbb{R}^{O \times D}$$

Residual connection + Layer Normalization nel Decode

Ora che abbiamo $\tilde{Y}$, cioè il risultato della **cross-attention**, dobbiamo **integrare questa nuova informazione** con la rappresentazione iniziale delle query $Y^{(0)}$.

Questa fase serve per:

1. **conservare** le informazioni originali delle query (non perdere ciò che definisce il tipo di output),
2. **aggiungere** le nuove informazioni lette dai latenti,
3. **stabilizzare** i valori numerici (normalizzazione).

Le due matrici che dobbiamo sommare sono:

Simbolo Forma

Significato

$Y^{(0)}$ $O \times E$ Query di partenza (specificano cosa chiediamo al modello)

$\tilde{Y}$ $O \times D$ Risultato della cross-attention (ciò che abbiamo letto dai latenti)

$$Y^{(\text{proj})} = Y^{(0)} W_{\text{align}}$$

$$W_{\text{align}} \in \mathbb{R}^{E \times D} \Rightarrow Y^{(\text{proj})} \in \mathbb{R}^{O \times D}$$

Questa è una semplice proiezione lineare (matrice di pesi trainabile) che “traduce” le query nello stesso spazio dei latenti.

Somma residua (residual connection)

Ora possiamo sommare i due contributi:

$$Y^{(\text{sum})} = Y^{(\text{proj})} + \tilde{Y}$$

Interpretazione:

- $Y^{(\text{proj})} \rightarrow$ rappresenta ciò che la query **era inizialmente** (cosa stai chiedendo).
- $\tilde{Y} \rightarrow$ rappresenta **la risposta** ottenuta dai latenti tramite la cross-attention.
- Sommando, ottieni una rappresentazione che **unisce la domanda e la risposta**: il modello mantiene il significato originale della query ma lo arricchisce con le informazioni lette.

Questa somma è detta **connessione residua** perché “salta” il blocco di attenzione e si somma alla sua uscita, migliorando la stabilità dell'apprendimento (evita che la rete dimentichi l'ingresso).

Normalizzazione (Layer Normalization)

Subito dopo la somma, si applica una Layer Normalization:

$$Y^{(\text{norm})} = \text{LN}(Y^{(\text{sum})})$$

Per ogni riga (ogni query):

$$\mu = \frac{1}{D} \sum_{j=1}^D Y_{ij}^{(\text{sum})}, \sigma^2 = \frac{1}{D} \sum_{j=1}^D (Y_{ij}^{(\text{sum})} - \mu)^2$$
$$Y_{ij}^{(\text{norm})} = \gamma \frac{Y_{ij}^{(\text{sum})} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

- γ, β sono parametri trainabili (scaling e shift).
- ϵ è una piccola costante per evitare divisioni per zero.

Perché serve:

- evita che i valori crescano o si riducano troppo durante le iterazioni di training;

- rende il flusso del gradiente più stabile;
- mantiene le diverse query comparabili tra loro (stessa scala).

Feed-Forward (MLP) + Residual + LayerNorm

Questa parte ha la stessa logica che c'è in ogni Transformer block (Encode, Process, Decode): dopo la parte di attenzione, si applica una trasformazione neurale indipendente su ciascun token (qui: ciascuna query).

1. Input

$$Y^{(\text{norm})} \in \mathbb{R}^{O \times D}$$

- ogni riga è una query arricchita e normalizzata dopo la cross-attention;
- dobbiamo ora farla passare in un piccolo MLP per aumentarne la capacità di rappresentazione non lineare.

2. Struttura del Feed-Forward (MLP a due layer)

$$\text{MLP}(x) = \sigma(xW_1 + b_1)W_2 + b_2$$

Dove:

- $W_1 \in \mathbb{R}^{D \times 4D}$
- $W_2 \in \mathbb{R}^{4D \times D}$
- σ = funzione di attivazione (nel Perceiver IO è GELU, a volte ReLU)

Step per step:

1. Espansione:

$$H = Y^{(\text{norm})} W_1 + b_1 \Rightarrow H \in \mathbb{R}^{O \times 4D}$$

→ aumenta la capacità (più neuroni, $4 \times$ più grande).

2. Attivazione non lineare:

$$H' = \sigma(H)$$

→ introduce non linearità, permettendo di modellare relazioni più complesse.

3. Compressione:

$$F = H'W_2 + b_2 \Rightarrow F \in \mathbb{R}^{O \times D}$$

→ riduce di nuovo alla dimensione modello D .

L'architettura Perceiver IO si basa sul **Perceiver**, che ha ottenuto la sua generalità cross-dominio assumendo che il suo input sia un semplice array di byte bidimensionale: un insieme di elementi (che potrebbero essere pixel o patch nella visione, caratteri o parole nel linguaggio, o una forma di embedding, appreso o meno), ognuno descritto da un vettore di caratteristiche.

Il modello, quindi:

1. **Codifica delle informazioni sull'array di input:** Si parte da un array di input, che può essere qualsiasi tipo di dato strutturato, come immagini, audio, testo, o una combinazione di questi. Il modello si propone di processare questi input complessi senza richiedere una specifica architettura di rete neurale progettata per il tipo di dati specifico. Invece, cerca di universalizzare il processo di apprendimento su vari tipi di dati.
2. **Utilizzando un numero minore di vettori di caratteristiche latenti:** Il concetto di "caratteristiche latenti" si riferisce a rappresentazioni astratte ad alta dimensione che il modello

apprende durante l'addestramento. Queste rappresentazioni sono "latenti" perché, a differenza dei dati di input originali, non sono direttamente osservabili ma sono invece costruite dalla rete per catturare le informazioni essenziali degli input. Il Perceiver IO mira a ridurre la dimensione dell'input in una forma più gestibile mantenendo l'essenziale delle informazioni attraverso queste caratteristiche latenti.

3. **Utilizzando l'attenzione di stile Transformer:** I Transformers sono un tipo di architettura per le reti neurali che si basa su meccanismi di attenzione per pesare l'importanza relativa di diverse parti dell'input nel contesto di un determinato task. L'attenzione permette al modello di focalizzarsi su parti specifiche dell'input, migliorando la capacità di elaborazione e interpretazione. Nel contesto del Perceiver IO, questo meccanismo è utilizzato per analizzare e interpretare le caratteristiche latenti.
4. **Elaborazione iterativa e aggregazione finale fino a una categoria:** Dopo aver codificato l'input in vettori di caratteristiche latenti e averli elaborati tramite attenzione, il modello procede attraverso fasi di elaborazione iterativa. Questo significa che le informazioni vengono processate ripetutamente, potenzialmente migliorando la rappresentazione latente ad ogni iterazione. Questo processo iterativo continua fino a quando le caratteristiche latenti non sono aggregate in un'output finale, che può essere la classificazione in una categoria, la produzione di una sequenza di testo, ecc.

Piuttosto che produrre una singola categoria, Perceiver IO mira ad avere lo stesso livello di generalità rispetto ai suoi output rispetto a quanto ha il Perceiver rispetto ai suoi input: cioè, dovrebbe produrre array di output arbitrari. Possiamo prevedere ogni elemento dell'array di output utilizzando un altro modulo di attenzione interrogando l'array latente utilizzando un vettore di caratteristiche di query unico per l'elemento di output desiderato. In altre parole, definiamo un array di query con lo stesso numero di elementi dell'output desiderato. Le query possono essere progettate manualmente, embedding appresi o una semplice funzione dell'input. Esse fanno attenzione ai latenti per produrre un array di output della forma desiderata.

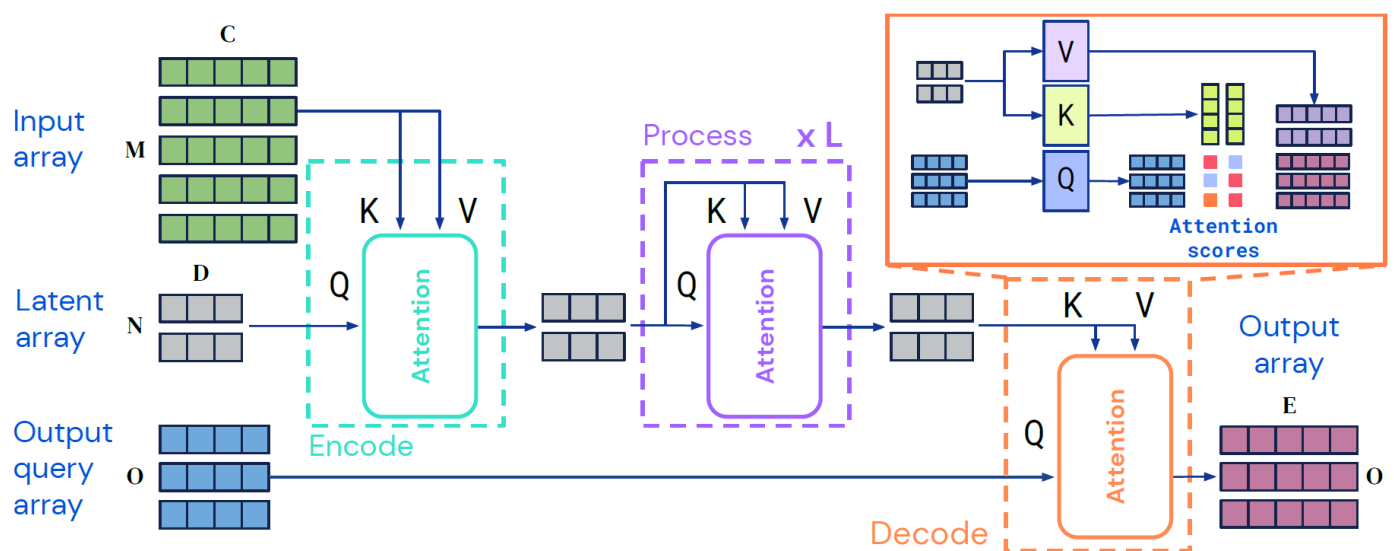


Figura 2: L'architettura Perceiver IO. Perceiver IO mappa array di input arbitrari su array di output arbitrari in un processo agnostico al dominio. La maggior parte del calcolo avviene in uno spazio latente il cui dimensionamento è tipicamente inferiore rispetto agli input e agli output, il che rende il processo computazionalmente gestibile anche per input e output molto grandi.

Codifica, processo e decodifica

Input Array

Qui rappresentiamo l'array di input multidimensionale. Ad esempio, potrebbe essere un'immagine, un file audio o qualsiasi altro tipo di dato strutturato. Le dimensioni di questo array sono indicate da $\mathbf{M} \times \mathbf{C}$, dove $\mathbf{M}$ potrebbe rappresentare il numero di elementi nell'input (pixel, campioni temporali, token di testo, ecc.) e $\mathbf{C}$ le loro caratteristiche (canali di colore, frequenza, incastonamenti di parole, ecc.).

Latent Array

L'array latente è una rappresentazione compressa degli input. Piuttosto che avere la stessa dimensione degli input, è di dimensioni $\mathbf{N} \times \mathbf{D}$, dove $\mathbf{N}$ è il numero di latenti (generalmente molto più piccolo di $\mathbf{M}$) e $\mathbf{D}$ è la dimensione delle caratteristiche latenti.

Output Query Array

Questo rappresenta l'array di query per l'output. Può essere considerato come un insieme di "domande" che il modello lancia per capire cosa cercare nell'array latente per generare l'output desiderato. Le dimensioni sono $\mathbf{O} \times \mathbf{E}$, dove $\mathbf{O}$ è il numero di query e $\mathbf{E}$ è la dimensione delle loro caratteristiche.

Processo di Attenzione e Codifica

- **Encode:** L'array di input e l'array latente sono prima sottoposti a un processo di attenzione. Per l'input, si calcolano i vettori chiave (K) e valore (V) per ogni elemento di $\mathbf{M}$. Per l'array latente, si calcola un vettore di query (Q).
- **Attenzione:** Un modulo di attenzione calcola i punteggi di attenzione tra Q e K, e usa questi punteggi per ponderare i valori V. Questo processo codifica le informazioni rilevanti dall'array di input nell'array latente.
- **xL:** Questo processo viene eseguito L volte, indicando il numero di iterazioni di elaborazione, potenziando l'array latente ad ogni passaggio.

Processo di Attenzione e Decodifica

- **Decode:** Un processo simile viene utilizzato per decodificare l'array latente per ottenere l'output. Si genera un nuovo set di vettori chiave (K) e valore (V) dall'array latente, mentre i vettori query (Q) provengono dall'output query array.
- **Attenzione:** Anche qui l'attenzione calcola i punteggi tra Q e K e utilizza questi per ponderare V, producendo così l'output array.

Output Array

- **E:** Questo array rappresenta il risultato del processo di decodifica e contiene le informazioni che sono state richieste dall'output query array.

In questa architettura, i processi di attenzione permettono al Perceiver IO di elaborare input di grandi dimensioni riducendoli a un formato gestibile senza perdere informazioni critiche. L'elaborazione iterativa attraverso l'attenzione consente di affinare i dettagli e le relazioni nell'array latente, migliorando la qualità dell'output finale.

Iniziamo **codificando** mediante un modulo di attenzione che mappa gli array di input $x \in \mathbb{R}^{(M \times C)}$ in array in uno spazio latente $z \in \mathbb{R}^{(N \times D)}$. Successivamente, **processiamo** i latenti z applicando una serie di moduli che prendono in ingresso e restituiscono array in questo spazio latente. Infine, **decodifichiamo** applicando un modulo di attenzione che mappa array latenti su array di output $y \in \mathbb{R}^{(O \times E)}$. M , C , O ed E sono proprietà dei dati del task e possono essere molto grandi (Tab. 5), mentre N e D sono iperparametri e possono essere scelti per rendere il calcolo del modello gestibile. Seguendo il design del Perceiver, implementiamo ciascuno dei componenti dell'architettura utilizzando moduli di attenzione di stile Transformer.

Ogni modulo applica un'operazione di attenzione globale query-key-value (**QKV**) seguita da un percettore multi-layer (**MLP**). Come consueto nelle architetture di stile Transformer, applichiamo il MLP in modo indipendente a ciascun elemento della dimensione indice. Sia l'encoder che il decoder prendono in ingresso due array, il primo utilizzato come input per le reti chiave e valore del modulo, e il secondo utilizzato come input per la rete di query del modulo. L'output del modulo ha la stessa dimensione indice (lo stesso numero di elementi) dell'input di query.

L'architettura Perceiver IO si basa su primitive simili a quelle nei Transformers. Perché i Transformers non sono sufficienti? I Transformers scalano molto male sia in termini di calcolo che di memoria. Poiché i Transformers distribuiscono moduli di attenzione omogeneamente in tutta la sua architettura, utilizzando l'intero input per generare query e chiavi ad ogni livello. Questo significa che ogni livello scala quadraticamente in termini di calcolo e memoria, il che rende impossibile applicare i Transformers su dati ad alta dimensionalità come le immagini senza qualche forma di pre-elaborazione. Anche in domini come il linguaggio, dove i Transformers eccellono, spesso è necessaria una pre-elaborazione (ad esempio, tokenizzazione) per scalare oltre brevi sequenze di input. Perceiver IO utilizza l'attenzione in modo non omogeneo mappando gli input in uno spazio latente, elaborando in quel tempo latente e decodificando in uno spazio di output. Perceiver IO non ha una dipendenza quadratica dalla dimensione di input o output: i moduli di attenzione dell'encoder e del decoder dipendono linearmente dalla dimensione di input e output (rispettivamente), mentre l'attenzione latente è indipendente dalle dimensioni di input e output. A causa della riduzione corrispondente dei requisiti di calcolo e memoria, Perceiver IO scala a input e output molto più grandi. Mentre i Transformers sono tipicamente utilizzati in contesti con dati pre-elaborati per contenere al massimo alcune migliaia di dimensioni, mostriamo buoni risultati su domini con centinaia di migliaia di dimensioni.

Questa architettura può essere applicata a input di qualsiasi forma o layout spaziale, inclusi input o output con diverse strutture spaziali (ad esempio, suono e video). In contrasto con gli spazi latenti tipicamente utilizzati nella visione, il latente non condivide esplicitamente la struttura (spaziale o altrimenti) degli input. Per decodificare queste informazioni, le interroghiamo utilizzando l'attenzione incrociata.

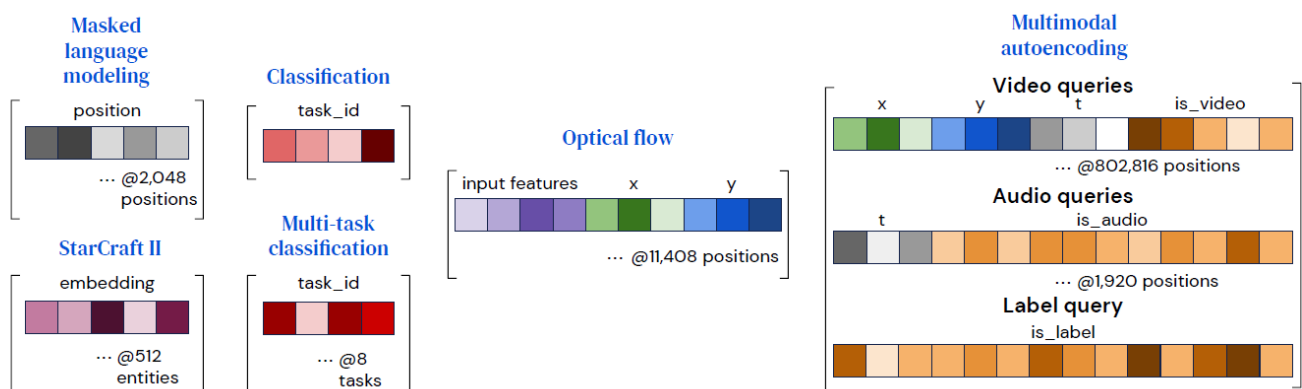


Figure 3: Costruiamo query con caratteristiche specifiche dell'output per produrre output con semantica diversa. Per impostazioni in cui ogni punto di output differisce solo nella sua posizione, come nel linguaggio, può essere utilizzato un embedding di posizione. Le caratteristiche di input per l'output target possono anche essere utilizzate per le query, sia da sole (come per StarCraft II) che insieme alle caratteristiche di posizione (come per il flusso). Per impostazioni multi-{task, modality} utilizziamo un embedding per ogni {task, modality} invece di ogni posizione. Un singolo embedding appreso è sufficiente per compiti di classificazione semplici, come ImageNet. Per compiti con output eterogenei come l'autoencoding multimodale, le caratteristiche specifiche per alcune query (come la posizione xy) possono essere combinate con embedding di modalità, che riempiono anche gli embedding a lunghezza fissa.

Decodifica della rappresentazione latente con un array di query

Il nostro obiettivo è produrre un array di output finale di dimensioni $O \times E$, dato una rappresentazione latente di dimensioni $N \times D$. Otteniamo un output di questa dimensione interrogando il decoder con un array di dimensione indice O . Per catturare la struttura dello spazio di output, utilizziamo query contenenti le informazioni appropriate per ciascun punto di output, ad esempio la sua posizione spaziale o la sua modalità.

Costruiamo le query combinando (concatenando o aggiungendo) un insieme di vettori in un vettore di query contenente tutte le informazioni rilevanti per uno dei O output desiderati. Questo processo è analogo al modo in cui le informazioni di posizione vengono utilizzate per interrogare funzioni implicite come NeRF (Mildenhall et al., 2020). Illustriamo la struttura della query per i compiti che consideriamo qui in Fig. 3. Per compiti con output semplici, come la classificazione, queste query possono essere riutilizzate per ogni esempio e possono essere apprese da zero. Per output con una struttura spaziale o sequenziale, includiamo un encoding di posizione (ad esempio un encoding di posizione appreso o una caratteristica di Fourier) che rappresenta la posizione da decodificare nell'output. Per output con una struttura multi-task o multimodale, apprendiamo una singola query per ogni compito o per ogni modalità: queste informazioni consentono alla rete di distinguere una query di compito o modalità dalle altre, così come gli encoding di posizione consentono all'attenzione di distinguere una posizione da un'altra. Per altri compiti, l'output dovrebbe riflettere il contenuto dell'input nella posizione della query: ad esempio, per il flusso troviamo utile includere la caratteristica di input nel punto interrogato, e per StarCraft II utilizziamo le informazioni sull'unità per associare l'output del modello all'unità corrispondente. Troviamo che anche le caratteristiche di query molto semplici possono produrre buoni risultati, suggerendo che il processo di attenzione latente sia in grado di apprendere ad organizzare le informazioni rilevanti in modo facile da interrogare.

Ogni punto di output dipende solo dalla sua query e dall'array latente, permettendoci di decodificare gli output in parallelo. Questa proprietà ci permette di ammortizzare l'addestramento del modello su dataset di dimensioni di output molto grandi. Ad esempio, Kinetics consiste di etichette, voxel video e campioni audio che insieme arrivano a oltre 800.000 punti (Tab. 5), che è proibitivamente costoso da decodificare in una sola volta, anche con scalabilità lineare. Invece, campioniamo in modo casuale l'array di output durante il tempo di addestramento e calcoliamo la perdita su un sottoinsieme accessibile di punti. Al momento del test, generiamo gli output per batch per produrre l'intero array di output.

FINE NOTE

INIZIO

APPENDICE

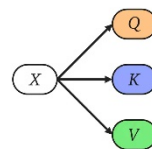
Appendice

Differenza tra Self-attention e Cross-attention

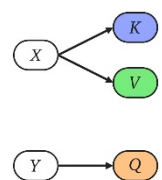
La **self-attention** è un meccanismo in cui Query (Q), Key (K) e Value (V) **provengono tutti dallo stesso insieme di input.**

- Serve a far sì che ogni elemento della sequenza (o ogni latente) **interagisca con gli altri** per arricchire la propria rappresentazione.

Self Attention



Cross Attention



- È quindi un'operazione **intra-spazio**: lo stesso array si auto-analizza e si rielabora.

Input I : sequenza di token (parole o patch immagine).

$$I \in \mathbb{R}^{M \times D}$$

Si calcolano direttamente:

$$Q = IW_Q, \quad K = IW_K, \quad V = IW_V$$

Attenzione:

$$A = \text{Softmax}\left(\frac{QK^T}{\sqrt{D}}\right) \in \mathbb{R}^{M \times M}$$

Output:

$$Z = A \cdot V \in \mathbb{R}^{M \times D}$$

(nel caso dei transformer con M molto grande il calcolo diventa proibitivo)

La **cross-attention** è un meccanismo in cui **Query (Q) proviene da un insieme (es. i latenti)** e **Key (K), Value (V) provengono da un altro insieme (es. l'immagine/token di input)**.

- Serve a permettere a un insieme (latenti) di **interrogare un altro insieme** (input esterno) e di incorporarne l'informazione.
- È quindi un'operazione **inter-spazio**: un array aggiorna la propria rappresentazione "guardando" un altro array.

Formula:

$$A = \text{Softmax}\left(\frac{QK^T}{\sqrt{D}}\right), \quad Z = A \cdot V$$

con Q dai latenti L , e K, V dall'immagine I .

Differenza tra Single-Head Cross-Attention e Multi-Head Cross-Attention nel Perceiver

1. Single-Head Cross-Attention

1. Proiezioni lineari

$$Q = LW^Q \quad (512 \times 512 \rightarrow 512 \times 512)$$
$$K = XW^K, \quad V = XW^V \quad (50176 \times 512 \rightarrow 50176 \times 512)$$

2. Attenzione

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{512}}\right) \quad (512 \times 50176)$$

3. Output

$$\text{Out} = AV \quad (512 \times 512)$$

👉 Ogni latente interroga l'intera immagine con un'unica mappa di attenzione.

2. Multi-Head Cross-Attention (h = 8)

1. Divisione embedding

$$d_k = \frac{D}{h} = \frac{512}{8} = 64$$

2. Proiezioni per ogni testa i

$$Q_i = LW_i^Q \quad (512 \times 512 \rightarrow 512 \times 64)$$
$$K_i = XW_i^K, \quad V_i = XW_i^V \quad (50176 \times 512 \rightarrow 50176 \times 64)$$

3. Attenzione per ogni testa

$$A_i = \text{softmax}\left(\frac{Q_i K_i^\top}{\sqrt{64}}\right) \quad (512 \times 50176)$$
$$\text{head}_i = A_i V_i \quad (512 \times 64)$$

4. Concatenazione e proiezione finale

$$\text{Out} = \text{Concat}(\text{head}_1, \dots, \text{head}_8) W^O \quad (512 \times 512)$$

👉 Ogni latente interroga l'immagine con 8 mappe di attenzione diverse → viste parallele su pattern differenti (colore, bordi, texture, struttura globale).

Quel W^O che compare alla fine della Multi-Head Attention è la matrice di output projection. Vediamo bene cosa significa e che ruolo ha.

◆ Definizione di W^O

Dopo aver calcolato tutte le teste:

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i) \in \mathbb{R}^{N \times d_k}$$

Concatenano i risultati:

$$H = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \in \mathbb{R}^{N \times (h \cdot d_k)}$$

Poiché:

$$h \cdot d_k = D$$

abbiamo $H \in \mathbb{R}^{N \times D}$.

A questo punto si applica una proiezione lineare con matrice W^O :

$$\text{Out} = HW^O, \quad W^O \in \mathbb{R}^{D \times D}$$

◆ A cosa serve W^O

1. Ricombinare le teste

- Ogni testa produce una rappresentazione parziale (es. bordi, colore, posizione).
- La concatenazione è solo un "accostamento".
- W^O permette di **mescolare e fondere** queste informazioni in un'unica rappresentazione coerente nello spazio di dimensione D .

2. Mantenere la dimensione costante

- Input e output della Multi-Head devono restare nella stessa dimensione D (nel Perceiver: 512).
- Senza W^O , l'output sarebbe solo una concatenazione, priva di trasformazione finale.

3. Parametri appresi

- W^O è una matrice di peso **appresa tramite backpropagation**.
- Inizializzata (es. Xavier uniform), si aggiorna durante il training.
- Serve a decidere **come pesare le informazioni provenienti dalle varie teste**.

3. Confronto diretto

Aspetto	Single-Head	Multi-Head (h=8)	
Dimensione Q	512×512	$8 \times 512 \times 64$	
Dimensione K,V	50176×512	$8 \times 50176 \times 64$	
Pesi di attenzione	1 matrice 512×50176	8 matrici 512×50176	
Output testa	512×512	$8 \times 512 \times 64 \rightarrow \text{concat} \rightarrow 512 \times 512$	
Capacità	Una sola prospettiva	Più prospettive parallele (pattern diversi)	

◆ Differenza concettuale

- Le matrici $W^Q, W^K, W^V \rightarrow$ servono a **proiettare** l'input in spazi diversi per calcolare l'attenzione.
- La matrice $W^O \rightarrow$ serve a **ricombinare** i risultati delle teste e riportarli nello spazio embedding originale

◆ Nel Perceiver (ImageNet)

- $D = 512$
- $h = 8$ teste
- Ogni testa $\rightarrow d_k = 64$
- Dopo la concat: $H \in \mathbb{R}^{N \times 512}$
- Moltiplicazione con $W^O \in \mathbb{R}^{512 \times 512} \rightarrow$ output finale ancora $N \times 512$

✓ In breve:

Il W^O è il peso di uscita della Multi-Head Attention.

Serve a **mescolare le informazioni delle varie teste** e a **ridurre tutto nella dimensione originale D** .

Multi-Head Attention (MHA) in generale

In un layer Transformer (o Perceiver), non usiamo una sola attenzione ma più teste parallele.

- Ogni testa ha i suoi pesi W_Q, W_K, W_V e calcola:

$$Z_h = \text{Softmax}\left(\frac{Q_h K_h^T}{\sqrt{D_h}}\right) V_h$$

con $D_h = D/H$ (dimensione ridotta se ci sono H teste).

- Poi i risultati di tutte le teste vengono concatenati e proiettati con un altro peso W_O :

$$Z = \text{Concat}(Z_1, \dots, Z_H) W_O$$

Quindi la MHA è una struttura architetturale che moltiplica la capacità del modello di guardare in più "spazi" o "prospettive".

Cosa cambia sostanzialmente con tipi di input diversi nel Perceiver?

Sostanzialmente, ciò che cambia è solo il **modo in cui l'input grezzo viene trasformato in un array di byte bidimensionale (la matrice $M \times C$)** prima di essere processato dal modello. Il cuore dell'architettura Perceiver, ovvero

il meccanismo di cross-attenzione che astrae l'input in un array latente di dimensione fissa e i successivi blocchi Transformer, **rimane quasi identico**.

Questa è la vera forza del Perceiver: disaccoppiare l'elaborazione principale dalla specificità dell'input. Il modello non ha bisogno di un'architettura completamente diversa per ogni tipo di dato, ma solo di una strategia di "appiattimento" e codifica posizionale iniziale.

Vediamo le differenze specifiche per ogni caso.

Come cambiano le matrici e gli step successivi

1. Immagine

- **Matrice di Input ($M \times C$):** Per un'immagine, la trasformazione è molto diretta. L'immagine, con le sue dimensioni spaziali (altezza H , larghezza W) e i suoi canali di colore (es. 3 per RGB), viene "appiattita".
 - **M (numero di elementi):** Diventa $H * W$. Ogni pixel è un elemento dell'array di input.
 - **C (dimensione di ogni elemento):** Corrisponde ai canali del pixel (es. 3 per i valori R, G, B) a cui si aggiunge una codifica posizionale (positional encoding) per preservare l'informazione spaziale. Questa codifica è fondamentale, altrimenti il modello vedrebbe solo un "sacco di pixel" senza sapere dove si trovavano.
 - **Esempio:** Un'immagine 224x224 RGB diventa una matrice $M \times C$ di $(224*224) \times (3 + D_{pos})$, dove $M = 50.176$ e D_{pos} è la dimensione della codifica posizionale.
- **Matrice Latente ($N \times D$):** Questa matrice è **indipendente dall'input**. La sua dimensione è un iperparametro del modello che tu definisci a priori.
 - **N (numero di vettori latenti):** È fisso (es. 512). Questo determina la "larghezza di banda" computazionale del modello. È molto più piccolo di M .
 - **D (dimensione di ogni vettore latente):** È anch'esso fisso (es. 1024).
- **Step Successivi:**
 - **Cross-Attention:** L'array latente ($N \times D$), che è inizializzato casualmente, "guarda" l'enorme array di input ($M \times C$) e, tramite la cross-attenzione, estrae le informazioni più salienti, "comprimendole" in sé stesso. L'array latente funge da *query*, mentre l'array di input funge da *key* e *value*.
 - **Self-Attention (Transformer Blocks):** L'array latente $N \times D$, ora "carico" di informazioni dall'immagine, viene processato da una serie di blocchi Transformer standard (self-attention). Qui, i vettori latenti interagiscono tra loro per affinare e integrare le informazioni estratte.
 - **Classificazione Finale:** L'output del blocco Transformer (ancora di dimensione $N \times D$) viene mediato (average pooling) lungo la dimensione N per ottenere un singolo vettore di dimensione D . Questo vettore viene poi dato in input a una testa di classificazione (un semplice layer lineare) per produrre i **logits** finali.

2. Video

- **Matrice di Input ($M \times C$):** Un video è una sequenza di frame (immagini) nel tempo. L'appiattimento include anche la dimensione temporale.
 - **M (numero di elementi):** Diventa $T * H * W$, dove T è il numero di frame.
 - **C (dimensione di ogni elemento):** Come per l'immagine, include i canali del pixel (es. 3) più una codifica posizionale. Crucialmente, questa codifica deve rappresentare non solo la posizione (x, y) nel frame, ma anche la posizione temporale t .
 - **Esempio:** Un video di 16 frame 224x224 diventa una matrice $M \times C$ di $(16*224*224) \times (3 + D_{pos_3D})$, con $M = 802.816$.
- **Matrice Latente ($N \times D$):** **Non cambia**. È sempre la stessa matrice di dimensione fissa $N \times D$ definita come iperparametro.

- **Step Successivi: Identici al caso dell'immagine.** Il meccanismo di cross-attenzione non si "preoccupa" del fatto che l'input provenga da un'immagine o da un video. Semplicemente, interroga un array di byte molto più grande, che grazie alla codifica posizionale contiene implicitamente l'informazione spazio-temporale.

3. Nuvola di Punti (Point Cloud)

- **Matrice di Input ($M \times C$):** Una nuvola di punti è un insieme di punti in uno spazio 3D.
 - **M (numero di elementi):** È semplicemente il numero di punti nella nuvola.
 - **C (dimensione di ogni elemento):** Per ogni punto, le feature sono le sue coordinate (x, y, z). A queste si possono aggiungere altre informazioni come il colore (R, G, B) o la normale. A differenza delle immagini, qui non è necessaria una codifica posizionale esplicita, perché le coordinate (x, y, z) sono già l'informazione posizionale stessa.
 - **Esempio:** Una nuvola di 4096 punti con coordinate e colori diventa una matrice $M \times C$ di 4096×6 .
- **Matrice Latente ($N \times D$):** Ancora una volta, **non cambia**. La sua dimensione è fissa.
- **Step Successivi:** Esattamente gli stessi. La matrice latente $N \times D$ usa la cross-attenzione per interrogare la matrice di punti $M \times C$ e distillare l'informazione rilevante. Il resto del processo fino ai logits è invariato.

Tabella Riassuntiva

Tipo di Input	Matrice di Input ($M \times C$)	Matrice Latente ($N \times D$)	Note
Immagine	$M = H * W, C = \text{Canali} + \text{Pos. Encoding 2D}$	Fissa, $N \times D$ (iperparametro)	La struttura spaziale è codificata.
Video	$M = T * H * W, C = \text{Canali} + \text{Pos. Encoding 3D}$	Fissa, $N \times D$ (iperparametro)	La struttura spazio-temporale è codificata.
Nuvola di Punti	$M = N. \text{Punti}, C = \text{Coord. } (x,y,z) [+ \text{Colori...}]$	Fissa, $N \times D$ (iperparametro)	La posizione è data dalle coordinate stesse.

In sintesi, l'eleganza del Perceiver sta nel confinare la complessità della multimodalità a un unico, semplice step di preelaborazione, mantenendo il "cervello" del modello (i blocchi di attenzione) agnostico e riutilizzabile per qualsiasi tipo di dato.